
quod

Project documentation — a Prolog/Brahms P2P
node over QUIC

Yan Guiborat

30 June 2026

Contents

quod	4
Architecture	4
Build & run	5
Configuration	5
Deploy (Docker + Nomad)	6
Status / next steps	6
How quod stores and shares knowledge — a plain-language guide	7
1. The shape of the problem	7
2. The one idea everything is built on	8
3. Ontologies that reach into each other	9
4. Reading something means watching it	10
5. Who's allowed to change what	10
6. How a network of computers agrees on changes	11
7. The history is a permanent, browsable record	12
8. Reading is cheap — it doesn't go through the committee	13
9. Surviving crashes now, surviving lies later	13
10. After a change is agreed: reactions	14
11. The physics side route	14
12. What's settled, and what's still open	15
13. Where the building starts	15
Content Layer Design — Proved Scopes over QUIC	17
0. The one idea everything hangs on	17
1. The core reframe — content is two things, not one	18
2. The central primitive — the staged proved scope	20
3. The inter-ontology graph	21
4. The :: operator	23
5. Commit semantics	24
6. Consensus — where, and how much	26
7. Transports — no broker	28
8. Machinery to port from bbsvx	29
9. Boundary to onia	29
10. Proposed phasing	30
11. Open questions	30
12. Open issues from the design review	31
13. The committee and the history — how an ontology is ordered and stored	32
14. The event system — how a committed change notifies what reacts	37
Ordering layer — Phase 1 build spec (quod_ledger / quod_prolog)	39
Module map	39
Table of contents	40

Shared header include/quod_ledger.hrl (canonical types + records)	40
1. quod_ledger state machine	42
2. Wire protocol over quod_link	54
3. Persistence (quod_ledger_store)	58
4. quod_prolog apply + prove	62
5. quod_ns_sup lifecycle + membership	68
6. Build order + test plan	73
7. Open questions / out of scope for Phase 1	76
Identity / signing milestone — kickoff brief	78
Kickoff prompt (paste this)	78
Why now — the motivation (don't lose this)	78
Definition of done	79
Current state — accurate pointers	79
Open architectural decisions — take a position, I'll comment	80
Constraints (must honor)	81
Read order	81
Verification (end state)	81
Deferred work — central registry	82
1. Identity / signing milestone — the big gate	82
2. Transport hardening (hostile-net)	83
3. Raft membership (M3 / M4 fast-follows)	83
4. Reader/subscriber arc — the path to “millions read root”	84
5. Parked (deliberately — don't reopen without a reason)	84
M1 content-layer — code review findings	85
Real bugs (live in M1)	85
Correctness subtleties — dormant in M1, pinned for M2 (concurrency) .	86
Robustness — mostly M2-coupled	86
Efficiency — fine at M1 scale, M2	86
Cleanup	87
By design / accepted	87
M2 content-layer — code review findings	88
Real bugs (live in M2)	88
Robustness — production-hardening beyond M2's loopback fault model . .	89
Lower-severity / latent — fixed opportunistically	89
Second pass — max-effort /code-review (10 angles, 2026-06-27)	90
Rejected (verified false alarms)	91
Module reference	92
Application & supervision	92
QUIC transport	92
Liveness handshake (link_up requires a peer ACK)	94

Membership (Brahms gossip)	95
A round	95
Byzantine handling	96
Sample validation (failure detector)	96
How it works	98
Windowing	98
Scope of the guarantee	99
Key conventions	100
Example	100
Namespaces	101
Prolog engine & proving	102
Content layer (ledger & diff)	105
Milestones	105
append → commit → apply is deadlock-free (load-bearing)	106
Channel-match hazard	106
Identity & signing	109
What is stored	109
The cert and mutual TLS	110
Observability	111

quod

A **Prolog/Brahms** P2P node over **QUIC** — no broker, no EMQX.

Transport is QUIC via the pure-Erlang [quic](#) library (no NIF, no msquic). Membership is [Brahms](#) byzantine-resistant peer sampling, one instance per **namespace** (ontology). This re-bases onbrater's L1: where onbrater used an in-VM MQTT broker, quod uses QUIC streams + Brahms gossip for fan-out.

Architecture

```

quod_brahms (gen_statem, one per namespace Ns) — view V + min-wise sampler
  |   gossips on channel Ns; open_link / quod_link:send / monitor(LinkPid)
  ▼
quod_quic (gen_server, singleton) ————— server + dialer + authority
  |   one connection per peer (no dial race); pure-Erlang QUIC
  ▼
quod_conn (one process per peer) ————— OWNS the QUIC connection
  |   demuxes {stream_data, StreamId, ..} to the right link
  ▼
quod_link (one process per (peer, channel)) ——— framing + publish + send
  |   publishes to {channel, Channel}; its death is the disconnect signal

```

module	role
quod_brahms	per-namespace membership statem (push/pull/reconstruct rounds)
quod_brahms_sampler	secret-keyed min-wise uniform sampler (HMAC-SHA256)
quod_quic	QUIC server + dialer + one-connection-per-peer authority
quod_conn	per-peer connection owner; routes stream data to links
quod_link	per-(peer, channel) stream: header handshake + length-prefixed frames
quod_reg	gproc nomenclature ({conn,NodeId}, {channel,Ns}, {quod_brahms,Ns}, ..)
quod_app	env-driven boot (config from the orchestrator)

Identity. A node's id is its **Ed25519 public key** (node_id), generated on first boot and persisted; the address {Host, Port} is demoted to a resolvable routing hint. The first frame on a stream is a header announcing the opener's {Pubkey, Addr} + channel, and **mutual TLS** binds the connection to that key (quic:peercert/1 must match the claimed pubkey). The committee is identified by pubkeys; Brahms discovery still works in addresses (it reads the Addr from the header). (*No-identity/test*

boots use the address as the id, transitionally.) Per-message/block signing + quorum certificates are Phase B.

Message contract. A consumer of channel Ns:

```
quod_reg:subscribe({channel, Ns}),
receive {quod_message, {Peer, LinkPid}, Ns, Payload} -> ... end, %% inbound gossip
quod_link:send(LinkPid, Reply),                               %% reply on the same stream
erlang:monitor(process, LinkPid) %% -> {'DOWN', ...} is the disconnect
```

Build & run

Pure Erlang — no C toolchain, fast build.

```
./scripts/gen-cert.sh      # once: self-signed dev cert in priv/certs/
rebar3 compile
rebar3 shell               # QUIC listener on :14567
```

Join a namespace from the shell, or via env (see below):

```
quod_brahms:start_namespace(<<"ont:test">>, #{
  node_id    => {"127.0.0.1", 14567},
  seed_peers => [{"127.0.0.1", 14568}, {"127.0.0.1", 14569}]}).
quod_brahms:view(<<"ont:test">>).    %% the converged peer view
quod_brahms:sample(<<"ont:test">>).  %% a uniform sample
```

rebar3 eunit (unit) and rebar3 ct (real loopback QUIC) cover the stack.

Configuration

A release reads its config from the environment (so a node is configured by its orchestrator, no custom sys.config):

env var	effect
QUOD_PORT	QUIC listen_port (and the port of this node's node_id)
QUOD_NODE_IP	node_id = {QUOD_NODE_IP, QUOD_PORT} — the dialable id
QUOD_NAMESPACE	ontology namespace to join on boot
QUOD_SEEDS	space/comma-separated ip:port bootstrap peers

Prometheus metrics are served at GET /metrics on metrics_port (default 14568): quod_up and per-namespace quod_brahms_{view_size,sample_size,links,rounds}.

+Q is not optional in a container The BEAM sizes its port table from `ulimit -n`. Container runtimes default `nofile` to $\sim 1e9$, which preallocates **~ 1.5 GB** of `port_table.config/vm.args` caps it with `+Q 65536` (KB-sized table). Without it a node uses ~ 2 GB instead of ~ 100 MB.

Deploy (Docker + Nomad)

```
docker build -t quod:0.2.1 .                # multi-stage; cached deps layer
docker tag quod:0.2.1 <registry>/quod:0.2.1 && docker push <registry>/quod:0.2.1
nomad job run deploy/quod.nomad             # 3 nodes, host net, static UDP 14567
```

`deploy/quod.nomad` pins one instance per server, host-network, a static UDP port (so seeds are predictable), a static seed list (each node filters itself out), and a Consul service. `quod` is **stateless** (in-memory view/sampler, cert in the image) — no CSI volume.

Status / next steps

- **Done:** pure-Erlang QUIC transport, Brahms membership per namespace, env boot, Docker/Nomad deploy. Verified full-mesh on a 3-node cluster (~ 100 MB/node).
- **Next:** the application layer — wire `erlog` so a namespace's gossip carries **ontology facts / distributed Prolog queries**, not just node ids.
- **Deferred:** PUSH over QUIC datagrams; per-block quorum certificates + signed changes (Phase B); Brahms gossiping the address hint (sparse-seed move-survival).

How quod stores and shares knowledge — a plain-language guide

quod lets many computers share collections of facts and keep them in sync, with no central server. This guide explains how, end to end, with no special background assumed. The detailed decisions and trade-offs live in a separate document, `content-layer-design.md`. The step-by-step plan for building the first part is in `ordering-layer-spec.md`.

Two words you'll see throughout:

- An **ontology** is a collection of facts and rules — *a cat is an animal, animals have legs*, and so on. It's quod's unit of shared knowledge.
- These are written in **Prolog**, a language where you state facts and rules and then ask questions, and the system works out the answer from what it knows.

1. The shape of the problem

We want a network of computers to share ontologies. Two things make this hard:

1. **There's no central server.** The computers find each other and agree among themselves.
2. **Content comes in two very different kinds**, which need opposite treatment. Some of it — the categories things fall into (a dog is a kind of animal), who owns what, the overall structure of the world — changes rarely, but everyone must agree on it exactly. Other content — where things are, how they're moving — changes constantly, and a lost update doesn't matter because a fresh one arrives a split-second later. We'll call this fast-moving kind **physics**.

These two kinds travel on separate routes:

	Structural facts	Physics
Changes	rarely	constantly
A lost update	must never happen	is fine — the next one fixes it
Route	careful, ordered, agreed	quick, send-and-forget

Most of this guide is about the careful route. Physics gets a short section near the end.

One honest wrinkle up front: "fast-changing" and "physics" are *not* the same thing. A few facts — who's holding the sword, whether the door is open — change often *and* must be agreed on exactly. Those take the careful route and pay its cost; only truly throwaway state (position, motion) takes the quick one. We come back to this in section 12.

2. The one idea everything is built on

Here's the move the whole design rests on: **don't try to predict what a question will do — run it and watch what it touches.**

Why you can't predict: in Prolog, asking a question can quietly *change* things. A rule, while working out an answer, can add or remove facts. For example, a rule answering "find me a free meeting room" might *book* the room it finds — the looking and the changing are tangled together. And a question can reach across into other ontologies. So you genuinely can't tell, before running it, whether a question will change anything or which ontologies it'll touch. You only find out by running it.

So we run it and watch. The question runs against a private throwaway copy (so the real data isn't touched until we're sure), and we record two things:

- **what it read** — the facts it looked at, and
- **what it changed** — the facts it added or removed.

Together, these two are the question's *footprint* — the marks it left. Once the question finishes, its nature is obvious:

- **It changed nothing** → it was just a *read*. Hand back the answer; done.
- **It changed something** → it's a *change* that has to be recorded and agreed.

This "run it, then look at the footprint" approach is what lets everything else — links between ontologies, change notifications, agreement between computers — be *one* method instead of many separate ones.

There's one more twist. Between running a change on the throwaway copy and making it official, the world can move — someone else may have changed a fact this change relied on. So at the moment of making it official, the system re-checks that everything the change *read* is still true. If it isn't, the change is rejected and the asker simply tries again. (This is why recording "what it read" matters as much as "what it changed.")

A small practical note: when a question does change facts, those changes are made as a clean final step *after* the answer is found, not scattered through the search. That keeps things tidy and matches how Prolog already works.

3. Ontologies that reach into each other

Think of each ontology as a separate notebook of facts. Most of the time a question stays inside one notebook. To reach another, you write `::`.

```
animals::diet(dog, D)
```

That reads: “work this out over in the *animals* ontology.” (D is a blank for the system to fill in — so this is asking “what does a dog eat?”)

By default a name is local; `::` is the only way to cross into another notebook. That keeps things simple — there's no single master list of which ontology owns which word, and no confusion when two ontologies happen to use the same name.

There's a second, smaller use of `::`. A *link* between two ontologies — “my_dog is a kind of the dog that lives in the animals notebook” — is just a fact that *points* across a boundary:

```
isa(my_dog, animals::dog)
```

Notice this is a stored fact, not a question — it just *names* something in another notebook. The system only actually reaches across when it needs something it doesn't have at home. Asking “is my_dog a dog?” is answered right here from the local fact; but asking “what does my_dog eat?” has to hop over to *animals*, because the answer lives there. So a link and a `::` question are the same idea at different sizes: a link names a thing across the boundary, and following it becomes a question when you actually need what's over there.

Because these links are just facts, the **web of connections between ontologies builds itself** as questions run. We don't draw the map by hand — it emerges from use, and we keep a directory of it for two jobs: knowing where to send a cross-notebook question, and knowing who to tell when something changes (next section).

4. Reading something means watching it

Here's a nice payoff from recording "what a question read." When the *pets* notebook reads a fact that lives in the *animals* notebook, *pets* automatically becomes *interested* in that fact — without having to say so. So if *animals* later changes it, the system already knows exactly who to tell: everyone whose recent questions read it.

You never declare "please notify me when this changes" — **reading it is subscribing to it**. That gives change notifications across the whole network essentially for free, and the kind of link tells the system where to send each one.

(The honest catch: today the system tracks interest in broad *groups* of facts, not single ones — for example, all "door" facts together rather than one specific door. So a change to one door can ping everyone watching any door. For rarely-changing things that's harmless; making it fact-by-fact precise is a later refinement.)

5. Who's allowed to change what

A fact belongs to the ontology that holds it, and **only that ontology may change it**. Each ontology is the one that decides its own permissions and runs its own rules when something is added or changed (adding a fact can trigger other rules, fill in default values, and so on). *(Today the permission checks are stubbed — computers trust each other's stated names; real checks arrive with the trust-the-stranger work in section 9. This section describes the intended shape.)*

That has a clean consequence: to change something in another ontology, you can't just reach in and overwrite it — you have to **ask the owner to do it**, through the owner's own rules. "Inventory, please add this sword" — not "I'll just stick this sword in your inventory."

For now we keep it simple: a question may **read** across ontologies freely, but may only **change** facts in its own. Changing facts across ontologies — moving a sword out of one notebook's chest and into another's bag — is genuinely harder: you must make sure the sword isn't lost or duplicated if something fails halfway. **And this is not an exotic case — it's the common one**. Every trade, pickup, and crafting recipe in a game moves something between ontologies. So it matters a lot; it's just the next big thing to build, not a corner case. When we add it, the safe default will be **all or nothing** — both the take and the give happen, or neither does.

6. How a network of computers agrees on changes

When several computers hold the same ontology, they must agree on the **order** of changes, so they all end up identical. (Order matters: if one computer adds a rule and another removes it, the final result depends on which happened first.) The mechanism is a small **committee**.

The committee

Each ontology is run by a small group of computers — say 3 or 5 — that holds it and agrees on every change.

- One member is **in charge** at a time. It takes incoming changes, bundles each into a numbered record — a **block** — and sends that block to the others. A block becomes official once **more than half** the committee has written it down.
- If the one in charge dies or goes quiet, the others **pick a new one among themselves** in a second or two. No human involved.

Why “more than half”? Because any two “more than half” groups must share at least one computer — and that shared computer won't write down two conflicting blocks. So the committee can never end up agreeing to two different versions of history. That's the whole safety argument, in one sentence.

The clever part: a committee of **one computer** is just “one computer in charge” — the simple setup you'd run on your laptop. Grow the *same* committee to 3 or 5 and it survives some members dying, with no change to the code. So “simple now” and “robust later” aren't two designs — they're one design, sized to taste.

(For the curious: this is a streamlined version of Raft, a standard agreement recipe used by many databases. We build our own lean version rather than add a large piece of outside software.)

Who sits on the committee is part of the ontology

Committee membership is itself stored as facts in the ontology, and a computer that wants to join is let in only if it passes a “may I join?” test the ontology defines. The committee is deliberately **stable** — it changes only on purpose (a member joins or leaves, and that itself is recorded as a block). That stability is exactly what lets it keep an exact record.

Finding the computers vs. running the ontology

Two layers, doing different jobs:

- **Discovery** — a background layer (we call it Brahms) where every computer casually passes along which others it has seen lately, like rumors spreading through a crowd. It's how computers *find* each other. It doesn't hold ontologies or order anything.
- **The committee** — the small, fixed group that actually holds one ontology and agrees on its changes.

So discovery *finds* candidates; the committee is the small subset that actually runs the show. Being one of the many computers out there is not the same as being on a committee.

7. The history is a permanent, browsable record

Keep two things separate:

- **The current facts** — the ontology as it is right now, what questions read.
- **The history** — the ordered list of every change ever made (the blocks). If you start from nothing and apply every change in order, you get back the current facts.

We keep the **full history, forever, and make it browsable** — a permanent record, like a logbook you can page through on a web page. It's saved to disk on the committee members. (Normally quod saves nothing to disk; the history is the one deliberate exception, because everything else can be rebuilt from it. The rule is: *save the history; rebuild the rest from it.*) Trimming old history to save space is an option we can add later; by default we keep everything.

For now the history lives only on the committee members. Later, an ontology will be able to name extra computers that should also keep a full copy — again, just by stating it as a fact.

8. Reading is cheap — it doesn't go through the committee

A natural worry: if a popular ontology — say *root*, the basic one everyone builds on — is run by just 3 computers, does every lookup pile onto those 3?

No — because **the committee is only the authority for *changing* an ontology**. Reading it needs nothing more than *a copy of the current facts*, and copies can be everywhere:

1. **Your own cached copy** — for things that rarely change (like the category structure), a computer keeps its own copy and is only pinged on the rare change. In normal running, a lookup in *root* is answered right on the asking computer, with no network at all.
2. **A nearby copy** — one short step away to a neighbouring computer, not all the way to the committee.
3. **The committee itself** — only when you need the guaranteed-latest value, which is rare.

So the 3 committee computers only carry that ontology's *changes*, and a foundational ontology like *root* is barely ever changed. If some *other* ontology turns out to be changed heavily, the fix is to split it into smaller ontologies, each with its own committee — more committees sharing the load, all working at the same time.

9. Surviving crashes now, surviving lies later

The committee starts out able to survive computers that **crash** — die, freeze, or fall behind. That's the right level of safety inside one trusted operator's cluster, and it's the simpler thing to build.

Some ontologies — ones shared between parties who don't trust each other — will eventually need to survive computers that **lie**, sending different stories to different members. That's a heavier kind of agreement, but it slots in underneath *without changing anything above it*: the way changes are proposed and applied stays the same; only the committee's internal way of reaching agreement changes. So each ontology can pick its level of trust, and we can add the stronger kind later without disturbing the rest. (This is also when real identity and permission checks arrive — see section 5.)

10. After a change is agreed: reactions

Once a change is official, three things happen, in order:

1. **The facts change** — the add or remove is applied.
2. **Summaries update** — anything the system keeps that's calculated from the facts (quick-lookup tables, the "who's watching what" list, and later things like a visual view of the world) is recomputed. This happens immediately, before anyone is told, so that by the time you're notified everything lines up.
3. **Reactions happen** — rules that say "when X happens, do Y" now run, messages go out, screens update. This is the outward-facing part, and it happens only for a genuinely *new* change.

That last point carries the one rule that's easy to get wrong: **when a computer replays old history** — catching up after falling behind, or rebuilding after a restart — **it must redo the fact changes but must NOT redo the reactions**. Otherwise a computer catching up on a thousand old changes would re-send a thousand old messages. (An earlier, related system handled this badly and only worked by chance; here the rule is explicit.)

Reactions, like everything else, are just facts: "when this kind of change happens, run this." This reaction system — together with the cross-ontology notifications from section 4 — is the next thing to build once the core is solid. The first build already handles the narrow, safe version of it.

11. The physics side route

Fast-moving state — positions, motion, anything that updates moment to moment — does **not** go through the committee or the history. That would be far too slow, and it doesn't need to be exact. Instead:

- Each moving thing has one computer in charge of it — its **owner** — that calculates how it moves. (Same "only the owner may change it" idea as section 5, just applied to a moving object instead of a fact.) Every other computer keeps a lightweight read-only copy that just mirrors the owner.
- Updates spread by quick, send-and-forget broadcasts. One that gets lost is simply overwritten by the next one a moment later.

So physics is a fast, lossy route running alongside the careful one. The only time the two meet is when something owned by one computer affects something owned by another — say a push from one area shoves an object that belongs to a different area — and that has to go politely through the owner, the same "ask the owner" rule as section 5.

12. What's settled, and what's still open

Settled:

- Run a question, watch what it read and changed; that footprint is the basic building block the whole system uses.
- A change is re-checked at the moment it's made official; if something it relied on changed underneath it, it's rejected and retried.
- :: to reach another ontology; reading across is allowed now, changing across is deferred.
- Reading a fact subscribes you to it — that's the notification system.
- Each ontology is run by a small committee that agrees on an ordered list of changes; one computer grows to several with the same code.
- The full change history is kept permanently and is browsable.
- Reads are served by cheap copies and caches, not the committee, so popular ontologies don't bottleneck.
- Survive crashes first; survive lies later, behind the same interface.
- Physics is a separate fast, lossy route.

Still open (and honestly so):

- **Changing facts across ontologies** — the move-it-without-losing-or-duplicating problem. It's common (every trade and pickup), so it matters; it's deferred, and the safe **all-or-nothing** version comes next.
- **Fast and exact** — some game-state changes (who holds the sword, is the door open) are both frequent and must-be-agreed, so today they pay the careful route's cost. Whether they deserve a third, faster route is still open.
- **Identity** — today computers trust each other's stated names; real signatures and permissions come with the survive-lies work. The first build runs in trusted, single-operator mode.
- **Reading two ontologies at once** can catch each at a slightly different instant, so they may not perfectly line up. We accept that for now.
- **Notification precision** — today we notify about whole groups of facts at once, not single facts.

13. Where the building starts

The first thing being built is the careful route's foundation: for each ontology, the committee that keeps the ordered list of changes, plus the part that applies

those changes and answers questions. That has a detailed, ready-to-build plan in `ordering-layer-spec.md`. Everything else here — cross-ontology links, notifications, the physics route, surviving lies — builds on top of it.

Content Layer Design — Proved Scopes over QUIC

Status: design notes, pre-implementation. **Date:** 2026-06-25. **Scope:** the ontology/content layer that sits on top of quod's working membership + transport (Brahms + QUIC). Today quod is *all* membership; this document is the plan for making the mesh carry the actual ontology.

This is a thinking document, not a spec. It records the decisions we reached and the questions still open. Inspiration is taken from onia (Architecture-H), bbsvx, and onbrater, but deliberately **simpler and broker-free**.

New here? Read `content-layer.md` first — a plain-language overview of the whole design. This document is the detailed decisions, trade-offs, and caveats behind it.

Reviewed (devil's-advocate + architect). Several claims below were overstated; they now carry inline *Caveat (review)* notes, and §12 consolidates the real gaps and the decisions still owed. The biggest: the ordering layer (§6) is *named but not designed* — that is the next design task.

0. The one idea everything hangs on

Everything is a *staged proved scope*.

When the network proves a goal, it runs the goal optimistically against a staging overlay. The goal returns its **bindings** (the solution — the variable values) up to the calling ontology, and leaves behind two things:

- **read-set** — the facts (and the ontologies) the proof *looked at*.
- **write-set** — the facts the proof *changed*.

(Backtracking yields the *next* bindings on demand; the read-set and write-set keep growing across solutions — see §5 *Backtracking*.)

That pair — the closure of everything the proof touched — is the **proved scope**. It is the unit the network re-checks and agrees on. Relations, cross-ontology calls, conflict detection, notification, and consensus are all *views over the proved scope*, not separate subsystems.

The agreed pillars:

1. **Proved scope is the central primitive.** (§2)

2. **The inter-ontology graph is the central standing structure** — produced by proved scopes, consumed for routing and notification. (§3)
3. **:: is explicit**, one operator with two positions. (§4)
4. **Commit is uniform *within an ontology***: a scope commits iff *its owning* transaction commits. Cross-ontology atomicity is **opt-in, not default** (§5).
5. **Every fact write is an ordered transaction in its own namespace's log** — per-namespace ordering, never one global chain; cross-ontology atomicity is opt-in over just the touched set. (§6)
6. **No broker**. Facts and scope dialogs ride reliable QUIC streams; physics rides QUIC datagrams. (§7)

1. The core reframe — content is two things, not one

Content is **durable structural facts** *and* **dynamic state (physics)**. They want opposite transports and opposite guarantees, so they must not share a path.

	Durable structural facts	Dynamic state (physics)
Examples	isa, instance_of, have_attribute, ACL, admission	positions, velocities, any
Change rate	rare	constant
Loss tolerance	none (must converge)	high (next tick fixes it)
onia treats it as	committed kb (D)	in-memory derived state
Transport	reliable, ordered (QUIC stream)	lossy, fast (QUIC datagram)

onia already keeps physics **out** of the durable log. So “physics spread across all ontologies” is not a fact-store problem — it is a separate fast lane (§7).

*Caveat (review): “change rate: rare” is true for *schema* facts (isa, attribute definitions) but **false** for *instance / gameplay state* (“who holds the sword”, “is the door open”) — those change fast *and* must be agreed, so they pay full ordering latency (§6) and can't ride the lossy physics lane. There is no “fast *and* agreed” lane yet. See §12.*

What bbsvx already learned about physics

bbsvx has a working physics layer; its lessons shape ours:

- **Owner / ghost authority.** Each `physical_space` has exactly **one authority node** (the node that created it — sticky, anti-hijack). The authority simulates full bodies in a real engine (Rapier); every other node holds read-only **ghosts** (kinematic, positions set from broadcasts); static bodies are owned everywhere. *No rollback* — the authority is correct by definition, ghosts converge. Forces/impulses/joints apply only on the authority: a node can't shove an entity it doesn't own.
- **Lossy fast broadcast, not ordered.** State spreads via Plumtree eager-push (~1–5 ms), **not** EPTO total order (~85 ms). Deliberately lossy: the authority broadcasts *all* owned states at ~30 Hz, batched with collisions, deduped by source+timestamp; an old batch is overwritten within ~33 ms, so dropping it is free. This validates our datagram fast-lane (§7).
- **Reads off the critical path.** Position/velocity reads hit an **ETS handle to the engine** directly (`body_position/2 ...`), never a `gen_server` call — a latency fix worth copying.
- **Tick is authority-local, unsynchronised.** 30 Hz on the authority; ghosts passively receive. No cross-node tick sync → temporary divergence is accepted (convergence over consistency).

Pain points they hit, and the fixes — all worth pre-empting:

Problem	Fix
Physics process tied to a 5 s-watchdogged worker, died with it	dedicated persistent supervisor
Brutal-kill left stale ETS blocking world re-create	self-heal: clear stale entry at init
Delayed tick → huge Δt → bodies tunnel through thin colliders	clamp $\Delta t \leq 100$ ms
Runaway bodies drift to infinity, burn CPU	hard position bound (± 1000 m)
Broadcast bounce-back reprocessed	record own batch source+ts, dedup

The cross-ontology wrinkle for us: a moving entity has a single authority, so cross-ontology physics (a force from space A on a body owned by space B) can't be a local write — it must be a request to B's authority. That is exactly a `::` call into B (§4) and it must respect B's ownership. So physics is a fast *lane*, but cross-ontology physics interactions still route through the proved scope. (Open question in §11.)

2. The central primitive — the staged proved scope

Why you can't classify a goal up front

In Prolog you cannot know, before running a goal, whether proving it will assert/retract somewhere in its derivation — and once it crosses ontologies you cannot know *which* ontologies it will touch either. Both are discovered only by running it. So routing by a predicted “read vs write” type is impossible.

So decide *after* execution (optimistic concurrency control)

Run the goal against a staging overlay, let it touch whatever it touches, and capture its read-set and write-set. The classification falls out of running it:

- **wrote nothing** → it was a read → validate the read-set by attestation → done, fast (§6).
- **wrote something** → it is a transaction → append it to the namespace's ordered log (§5, §6).

Staging is prototyped (port from bbsvx)

erlog's DB interface is a behaviour; the DB handle is threaded through `#est{}`, so a proof can run against a swapped-in backend. bbsvx already builds the staging layers on top of this (see §8). Per-proof copy-on-write + commit/drop is **prototyped** there — a pattern to port. *Caveat (review)*: §8 itself lists semantics bbsvx left open, so “port” includes resolving those (§12).

External side effects (sending a message, a physics push — onia's **E** category) cannot be rolled back, so they must be **deferred until after commit**. onia already does this.

What a fact is (and what the read/write-set hold)

- **A fact** is a Prolog clause {Head, Body} (Body empty for a plain fact). Its **identity is its content** — {Head, Body} only. Asserting the same content twice dedups; retract is by content. (bbsvx confirms: dedup/conflict hashing strips everything but Head+Body.)
- **Per-fact metadata = a pointer to the log transaction that asserted it.** That transaction already carries who emitted it (caller namespace), when, the order (log index), and the signature — so we do *not* copy bbsvx's namespace **provenance chain** into the fact. bbsvx needed that chain only because it had

no per-namespace ordered log and forwarded facts through subscription federation; we have the log, which gives per-fact provenance for free. (Add the chain back only if we ever cache foreign facts.)

- **write-set** = the differ's ordered op-log of asserts/retracts (each tied to the committing transaction).
- **read-set** = the differ's per-functor content hashes — what the proof looked at, for conflict-check and notification. *Caveat (review)*: this is **per-predicate** (a whole functor hashed as one), **not per-fact** — so two writes to *different* facts of the same predicate falsely conflict, and one fact change notifies *every* reader of that predicate. Per-fact granularity is a decision owed (§12).

Reads: a goal emitted from a caller ontology

A goal is never free-floating — it carries the **caller namespace** (the emitting ontology), the way onia carries its subject. So the read API is just:

```
prove(TargetNs, Goal, CallerNs) -> bindings
```

A read **is** a proved scope with an empty write-set: proved against the local converged log, with **bindings returned to the caller** (not auto-asserted into it). CallerNs is first-class even in MVP — it is what the read-set→notification index (§3) and :: ACL key on. The agent/client machinery (wielding, sessions) is onia's L3 concern; quod only exposes prove. **Cross-namespace reads see the target's latest committed state, recording the log height read** (§3) — no snapshots to retain. *Caveat (review)*: reading two namespaces (B then C) can **mix moments** — B may move between the reads — so a cross-namespace read is **not** a consistent snapshot across ontologies (read skew). Tolerated for MVP; flagged in §12.

3. The inter-ontology graph

Relations and :: calls are the same thing at different sizes

- A **relation** like `isa(cat, animal)` that crosses a boundary is *a fact proven across that boundary* — a tiny proved scope (read one thing, write nothing).
- A **:: call** is *the same move, bigger* — prove a whole goal across the boundary.

Same mechanism, different size. There is no separate “relation network” and “cross-ontology call” feature — there is one primitive.

The graph emerges; we index it

The graph's **arcs are proved scopes**; its **labels are the relation functors**. We do not author it as a separate map — proving *populates* it (via cross-ontology reads), and we maintain it as an **index/cache** for two purposes:

- **Routing (pull):** to send a `::` call, look up which node hosts the target (see *Finding the target node* below).
- **Notification (push):** the **read-set is the subscription**. When a fact changes, the nodes to notify are exactly the proved scopes that *read* it. You never declare a subscription — reading B's fact *is* subscribing to it. The relation label gives differentiated routing. (This is the “notify others quickly when something happens to an ontology” requirement — *caveat (review)*: today it fires **per predicate**, not per fact, see §2/§12.)

Because relations can be *derived* (proven by rules, not just stored), the graph is a **runtime trace we cache**, not a static authored structure. Same “run it to find the scope” principle as §2.

Finding the target node

The graph gives you the target *namespace*; you still need a *node* that hosts it. Three cases:

1. **You're a member of the target namespace** → you already hold its Brahms membership, and the nodes in that view *are* the nodes hosting it. Pick one (a write/scope contacts one; a read picks $f+1$ for attestation, §6). *Caveat (review)*: being in the Brahms view $\neq$ hosting the fact log — membership and the replica set aren't necessarily the same, so this needs defining (§12).
2. **You're not a member** → you need a directory: namespace → some hosting nodes. The clean, dog-fooding answer is that **the directory is itself an ontology** — a well-known root namespace every node joins, carrying low-rate `hosts(Ns, Node)` facts merged as a CRDT (last-writer-wins, periodically republished), read from a hot ETS cache. This is exactly bbsvx's `bbsvx_contact_service` (namespace → {host,port}, gossiped ~60 s, LWW). Routing is then just another proved scope over the root ontology.
3. **Cold / unknown** → ask any peer “who hosts Ns?” and let it answer from its directory; once you reach one hosting node you learn the target's Brahms view and refresh from there.

So: **Brahms view when you're in the namespace; a CRDT contact-ontology when you're not**. No new mechanism — the directory is content like everything else. *Caveat (review)*: the LWW directory is the one **AP/lossy** piece (concurrent

host registrations can be dropped), and "routing is just a proved scope" hides a cold-start bootstrap cycle (case 3 is the real entry point). See §12.

Two layers, two lifetimes — it is *not* QUIC-connections-between-ontologies

Layer	What it is	Lifetime
Logical edge	a relation fact (qualified name)	durable — replicated with the ontology
Physical traversal	a QUIC stream that walks the edge	ephemeral — one scope-prove (the)

An ontology is *not* a node — it is hosted by a **set** of nodes. To walk edge $A \rightarrow B$, a node holding A opens a **per-proof stream** (`quod_link`) on the **already-standing per-peer connection** (`quod_conn`, kept alive by Brahms) to a node holding B. Connections are between nodes and reused; the graph itself is content, not wires. This stream replaces bbsvx's WebSocket `scope_ws`.

4. The `::` operator

Explicit, not implicit

Default: the next fact/goal is **local** to the current ontology. `::` is the **only** way to reach another ontology. This avoids needing a global "who owns this predicate" directory and the name-clash problems implicit routing would have.

One operator `Ns::Term`, two positions

- **Goal position** — `..., animals::diet(dog, D), ...` → "prove this goal in `animals`." Qualifies the **predicate**. (This is bbsvx today.)
- **Argument position** — `isa(my_dog, animals::dog)` → "dog lives in `animals`; references to it resolve there on demand." Qualifies the **name**. This is what turns cross-ontology edges into **first-class data** (the graph of §3).

Name-position reduces to goal-position

Argument-position `::` is a *convention on top of* goal-position: a small fixed set of **link-following predicates** (`isa`, `instance_of`, `have_attribute`, ...) notice a `::-` tagged argument and re-issue themselves as a goal-position call into that ontology. **One mechanism**, not two; both bottom out in the staged proved scope.

When a foreign name triggers a trip

The rule: **a foreign-tagged name causes a cross-ontology trip only when one of the link-following predicates has it as its *subject* and actually runs.** Storing the fact, unifying it, passing it around — all stay local and free. It is one clause per relation:

```
have_attribute(Ns::Class, Attr, Val) :- Ns::have_attribute(Class, Attr, Val).
isa(Ns::Class, Super)                :- Ns::isa(Class, Super).
```

Properties:

- **Lazy** — the hop happens only when that rule runs (when the foreign property is actually needed), never when the name merely appears.
- **Cheap** — no global “where does this live?” check on every term; only these few predicates look, and only at their subject.
- **Self-limiting** — if the fetched thing itself points to a third ontology, the same predicate hops again, walking the trail; the existing `:: depth-limit + cycle guard` caps it.
- **Splits the example right** — `isa(my_dog, animals::dog)` answers “is my_dog a dog?” from the local fact (no hop), but “what does my_dog eat?” goes through `have_attribute`, meets `animals::dog` as subject, and hops.

Which predicates are link-followers is itself the “differentiated routing by relation” of §3: each relation decides how (and whether) it follows.

Phasing

Build **goal-position first** (simple, already working in `bbsvx`). Add **argument-position** (the link-following clauses above) when the relation graph needs first-class edges. Same operator, not a fork.

5. Commit semantics

Uniform rule (one line):

A scope commits **if and only if** the transaction that opened it commits.

- **self** is the degenerate case: the scope's DB is your own, so writes are visible to you immediately and commit with you. (bbsvx special-cased self only to dodge a `gen_statem:call-to-self` deadlock — an implementation detail, not a semantic difference, so nothing principled to preserve.)
- **local / remote**: the foreign write is the *owner's* transaction — see *Foreign writes* below.

Foreign writes: the owner executes — ACL forces this

Concrete case: a rule in **GAME** moves a sword from a chest into a player's bag, which lives in **INVENTORY** — a different ontology, on different nodes.

GAME can't splice that fact into INVENTORY directly, because **INVENTORY owns its own access control and its own write-side logic** (who may write, plus the effects, defaults, and derivations a write triggers — *onia's effect/4*, inherited *have_attribute*, ...). Authorising *and* running that logic is INVENTORY's job, so a foreign write is necessarily **a request the owner executes through its own rules** ("INVENTORY, add this sword"), not a ready-made diff. ACL is the visible reason; the general one is that the owner's write path is its own program. (bbsvx already works this way: `::` runs the goal on the *target's* actor, in the target's overlay, under the target's ACL.)

This collapses the earlier "two ways" — there is really **one write mechanism: the owner executes**. What stays open is only **commit-coupling**:

- **Independent**: each ontology commits its own writes separately. GAME commits "removed from chest"; INVENTORY commits "added to bag." Simple, but a crash/ timeout between the two **loses or duplicates the sword**. Fine only for genuinely independent effects.
- **Coupled**: several owner-executed writes wrapped in one all-or-nothing step — a real distributed commit across *only* the touched ontologies (2PC, or BFT for the untrusted slice). No out-of-step window; needs the coordination protocol.

Caveat (review): the doc originally made **independent the default** and called coupled "rare." That is backwards — the sword move *is* the conserve-a-resource case (transfers, trades, crafting), which is the **common** case and needs coupled. And "healed by compensation" has **no general construction** for effect-bearing Prolog writes (a write fires rules/effects others may have acted on). **Decision owed (§12)**: default to coupled for conserved resources; define the coupling protocol (the MVP avoids it only by forbidding foreign writes).

MVP: allow only **own-ontology writes + cross-ontology reads**. Reads commit nothing on the target, so there is no cross-node commit to build yet — the whole problem is deferred until foreign writes are actually needed.

Backtracking

A :: call is not one-shot — the caller can backtrack into it for the next solution. The mechanism (bbsvx and onia §16 agree):

- The **cursor lives on the answering side** — the choice points (#erlog{ } state) stay on the target, addressed by a unique **InvocationId** per :: call.
- The caller drives it with **next** (onia's dialog room: prove / next / close). Each next advances that cursor and returns the **next bindings** (the binding-return of §0). When the caller's engine backtracks into the :: choice point it sends a next; close (or the stream dying) ends it.

Caveat (review): an open cursor holds the **full interpreter state** on the answering node until close/stream-death. Without per-caller caps + an idle timeout that is a memory-leak / DoS surface — cursor lifecycle is a decision owed (§12).

Backtracking and staged writes — we stay ISO Prolog-compliant. Verified in erlog: the choice point (#cp{ }) captures bindings + variable counter but **not the db**, so backtracking does *not* undo assert/retract — they persist, exactly as standard Prolog (and bbsvx/onia) already behave. We adopt that as-is: **no savepoints, no db-rewind machinery** — the db behaviour module stays simple.

The only consequence (a write from an abandoned search branch persists in the diff) is standard Prolog and is the rule author's concern, handled the usual way: **writes are effects applied *after* the solution is found, not interleaved with the search** (onia's effect model). Then backtracking only ever rewinds bindings — which erlog does natively — and there is nothing to undo. Savepoints stay in the back pocket only if we ever want true tentative-write-and-backtrack, which the MVP does not. (One ISO fine-print to confirm is erlog's *logical update view* conformance — whether an assert is visible to a call already iterating the same predicate — but that's an erlog detail, not a quod design decision.)

6. Consensus — where, and how much

Reframing “must be Byzantine at the transaction level”

Agreed on the *goal* (trustworthy results in an untrusted network); disagreed that the *mechanism* is “everything through one global blockchain.” The mechanism matches the operation:

Operation	Mechanism
Any fact write	append to the owning namespace's ordered log (single sequence)
Goal results (reads)	quorum attestation (f+1 matching signed answers), or recomputation
Cross-ontology atomic (rare)	coordinate the touched namespaces' logs (2PC / BFT)
Physics	datagram, last-writer-wins, no log

The key move: **every fact write is an ordered transaction within its namespace**. Each ontology is its own per-namespace ordered log of diffs; every node applies the same diffs in the same order, so retraction, deletion, and "is this absent?" are all well-defined **with no CRDT machinery** — the log order *is* the tie-break. The proved-scope's read-set is validated *at the write's log position* (did anything it read change since it ran? — if so, abort/retry), which gives serializability **within a namespace**. *Caveat (review)*: it does **not** compose across namespaces (a proof reading B then C can mix moments — read skew), and because the read-set is **per-predicate** (§2) conflicts are detected per predicate, not per fact. There is also no contention story (backoff, retry cap) for hot facts. See §12.

This works because the rate is right:

- The facts that change **fast** (who holds the sword, is the door open) are exactly the ones nodes **must agree on** — they need ordering anyway; a CRDT's "fast path" would be unsafe for them.
- The facts that are **cheap/monotonic** (schema: isa, attribute definitions) change **rarely**, so ordering them costs nothing.
- The genuinely high-frequency stuff — physics — is already on the separate lossy datagram lane (§7), off the log entirely.

So there is no fact class that wants a CRDT fast-path, and we **drop CRDT for the fact store**. (CRDT/LWW is still fine for *soft routing state* like the contact directory of §3 — that is not the fact store.)

Caveat (review): two honest consequences. **(1)** The middle bullet's "fast facts need ordering anyway" means those facts pay full ordering latency and we have no "fast *and* agreed" lane — the "ordering is free" framing only holds for the genuinely-rare schema facts. **(2)** Dropping CRDT is a **CAP choice**: an ordered log is **CP**, so under a network partition the side that can't reach the sequencer/quorum **cannot write** (and may not read latest) — that namespace is unavailable there. Stated, not hidden. The monotonic schema subset is a legitimate AP/CRDT candidate worth reconsidering (§12).

What ordering buys, and how heavy it must be

A per-namespace total order = **arbitration of conflicting writes** + a clean, replayable, auditable history — which is why bbsvx (EPTO) and onia (Tendermint) both built on an ordered log, and why Prolog *needs* it: retract + negation-as-failure make order carry meaning, so a CRDT (which converges to *a* set, not the intended derivation) is the wrong substrate.

How heavy the orderer is varies **per namespace**:

- **Single sequencer per namespace** — simplest; fine for a dev / single-operator cluster.
- **Consensus (Tendermint or lighter)** — when operators distrust each other, for the security-critical namespaces. Byzantine.

Caveat (review) — this is the doc's biggest gap. The single sequencer is a **single point of failure and crash-fault-only**: it can equivocate, censor, or halt the namespace — the exact faults Brahms exists to resist. It is also **undesigned**: *which* node sequences, how it's chosen, how it fails over, how non-sequencer nodes replicate and apply in order, and where the log is persisted are all unspecified. And "harden to consensus later" is a **commit-path rewrite, not a config knob** (single-sequencer needs no agreement, view-change, or quorum-certificates; BFT needs all of them). **Designing this is the next task (§12).**

7. Transports — no broker

Everything rides the existing QUIC mesh (quod_quic / quod_conn / quod_link); nothing reintroduces a broker (vs onbrater's in-VM ejabberd/MQTT, which also brought a GPL cliff).

- **Facts / scope dialogs** → reliable, ordered **QUIC streams** (quod_link). A scope dialog is one stream per prove invocation; the closed stream *is* the "requester left, cancel" signal that MQTT needed a session hook for.
- **Physics / dynamic state** → **QUIC datagrams** (send_dgram, already noted in the QUIC-optimizations memo). Unreliable, last-writer-wins, no log. This is the "UDP" instinct — placed on the dynamic half, where loss is fine.

The earlier "MQTT vs UDP" question resolves to: **neither** — reliable QUIC streams for facts/scope, QUIC datagrams for physics, both over the same connection.

8. Machinery to port from bbsvx

The staged-proved-scope model already exists in prototype in bbsvx, in layers over erlog's DB behaviour:

Module	Role
bbsvx_erlog_db_local_prove	shadow overlay; asserts/retra
bbsvx_erlog_db_differ	records every write as a diff
bbsvx_erlog_db_federated	cross-ontology reads (subscri
bbsvx_actor_ontology scope_prove / scope_commit / scope_drop	the cross-ontology transaction
bbsvx_common_predicates pred_cross_ontology_call	the :: handler: self / local / r

Port, don't reinvent — but resolve the tensions bbsvx left open (read-only comment vs writes-via-scope; self/local/remote semantics; explicit :: vs the implicit federated read path) using the decisions in §4-§6.

9. Boundary to onia

Decision: **quod-native API first**, not a strict clone of onia's L1 boundary. Later, expose an adapter matching `onbrater_host:prove/2,3 + submit_event/2 + {onbrater_event, Ns, ...}` so onia plugs in as a git dep behind its narrow boundary (onia §11/§15).

One known leak: onia's Index (a deterministic *cluster-wide* order on events) assumes the ordering layer (§6). Until that exists, a local per-node monotonic index is fine for a dev/single-operator cluster — flag it, don't pretend it's cluster-wide.

Identity stance. *Updated (identity milestone A.3, 2026-06-30):* a node's `node_id` is now its **Ed25519 pubkey** (the address is a routing hint), and connections are bound to that key via **mutual TLS** — the keypair lands, so identity is no longer stubbed. Still **stubbed until Phase B**: per-change **signatures** (`#transaction.sig`), the per-block **quorum certificate**, and signed attestation. So ACL-enforced foreign writes (§5) and quorum attestation (§6) — which need *signed* changes, not just an authenticated transport — remain gated on Phase B (§12). The single-operator trust model holds in the interim.

10. Proposed phasing

1. **Single-node content engine** — wire erlog per namespace (fills the quod_prolog TODO in quod_sup.erl); staged proved-scope proving (port local_prove + differ); a **per-namespace ordered log** (single sequencer to start) so every node applies the same diffs in the same order; local goal-position :: *Caveat (review)*: as scoped this exercises **no distribution** (remote :: is Phase 2), runs single-operator with signing stubbed (§9), and its hardest dependency — the ordered log itself — is **undesignated** (§6/§12) and must be designed before this ships. The genuinely distributed, useful-to-onia milestone is the *end of Phase 2*, not Phase 1. (The ordered log is now **sketched in §13** — per-namespace Raft.)
2. **Inter-ontology graph** — index cross-ontology reads; read-set-driven notification; remote :: over QUIC streams (replace scope_ws); argument-position :: (the link-following clauses of §4).
3. **Physics lane** — QUIC datagram broadcast for dynamic state, LWW, no log.
4. **Hardening** — per-namespace sequencer → consensus (Tendermint or lighter) where trust demands; quorum attestation for reads; cross-ontology atomic coupling for the rare cases that need it.

11. Open questions

Settled this session (folded into §2/§4/§5/§6):

- **Fact representation + read-set/write-set** — content-identity facts + a pointer to the asserting log transaction; write-set = op-log, read-set = per-functor content hashes (§2).
- **:: argument-position** — link-following predicates; trip only when one has the foreign name as its subject (§4).
- **Reads** — prove(TargetNs, Goal, CallerNs) → bindings; a read is a proved scope with an empty write-set (§2).
- **Backtracking over staged writes** — stay ISO Prolog-compliant (asserts persist on backtrack, as erlog/bbsvx/onia already do); no savepoints, db module stays simple; speculative writes handled by the effects-after discipline (§5).
- **Cross-namespace read consistency** — read the target's latest commit, record the height; no snapshots (§2/§3).
- **Consensus model** — per-namespace ordered log; CRDT dropped for the fact store (§6).

Still open — all deferrable past the MVP:

- **Cross-node coupled commit** — the rare atomic case: 2PC vs small consensus over the touched ontologies, coordinator-failure handling, crash vs Byzantine. *Phase 4.*
 - **Attestation quorum** — $f+1$ vs $2f+1$; can the relation label set the policy per edge? *Phase 4.*
 - **Physics** — adopt owner/ghost authority (§1); how a cross-ontology force reaches the owner without per-tick commit latency; authority handoff. *Phase 3.*
 - **onia boundary convergence** — when to add the `onbrater_host` adapter. *When onia needs it.*
-

12. Open issues from the design review

A devil's-advocate + architect review surfaced these. None breaks the core model; all are real gaps or decisions owed before building. Roughly in priority order.

1. **The ordering layer is undesigned — the big one.** §6 names a “single sequencer per namespace” but gives no design: *which* node sequences, how it is chosen, how it fails over, how non-sequencer nodes replicate and apply in order, where the log is persisted. A single sequencer is also a **single point of failure, crash-fault-only** (it can equivocate/censor/halt — the faults Brahms exists to resist), and “harden to consensus later” is a commit-path rewrite, not a knob. **Sketched in §13** (one Raft group per namespace, 1-voter→N, protocol pluggable behind a stable log API).
2. **Read-set is per-predicate, not per-fact.** The ported differ hashes a whole functor's clauses, so conflicts *and* notifications fire per predicate. Decide: build per-fact granularity, or accept and document per-predicate (false conflicts + notification fan-out).
3. **Cross-ontology atomicity default.** The sword move is the lost/duplicated-item hazard; “independent commit by default” is unsafe for conserved resources (transfers/trades/crafting) — the common case, not rare. “Compensation” has no general construction for effect-bearing writes. Decide: **coupled by default** for conserved resources; define the coupling protocol (deferred to Phase 4, but the motivating example needs it).
4. **Identity landed (A.3); signing is Phase B.** `node_id` is the node's Ed25519 **pubkey** and connections are bound to it via mutual TLS. Still pending: per-change **signatures** + the per-block **quorum certificate**, so ACL-enforced foreign writes (§5) and attestation (§6) — which need *signed* changes — remain gated on Phase B (§9).

5. **No "fast *and* agreed" lane.** Instance/gameplay facts are high-rate *and* need agreement, so they pay full ordering latency and can't use the lossy physics lane. Decide: add a fast-but-agreed mechanism (per-key single-owner serialization), or scope quod's log to structural facts only.
6. **Durable history vs. stateless deploy.** *Decided (§13):* keep the **full block list durably on disk** so the history is permanent and browsable (a ledger) — a deliberate exception to "no disk," for the history only. Current facts aren't saved separately (they replay from the block list). Cold boot = reload own history and/or catch up from the committee. Trimming old history is opt-in, later.
7. **CAP stance is implicit.** Ordered logs are CP — under partition the side that can't reach the sequencer/quorum is unavailable for that namespace. State it per fact class; reconsider keeping CRDT for the monotonic schema subset.
8. **Membership $\neq$ replica set.** Being in a namespace's Brahms view doesn't mean you host its log. The replica set (and the *f* in "*f*+1 attestation") needs its own definition.
9. **Cursor resource bounds.** Open :: dialogs hold full interpreter state on the answering node; need per-caller caps + idle timeout, plus a per-proof hop budget for the link-following fan-out (§4).
10. **erlog engine concurrency model.** Likely one *gen_server* per namespace serializing commit/apply, with read-only proofs on copy-on-write overlays — state it; getting it wrong is a rewrite.
11. **Cross-namespace read-set validation seam.** Connect "record the read height" (§2) to the notification index (§3) and commit-time validation (§6) into one explicit path — this is where cross-ontology serializability actually lives.
12. **Contention management.** OCC needs backoff + retry cap + a fallback to pessimistic single-owner for hot keys, or hot facts livelock under load.

13. The committee and the history — how an ontology is ordered and stored

Status: proposed, not finalized. This is the design for §12 #1 (and it settles #6 and #8). Prior art: one small agreement group per shard (TiKV, CockroachDB call it *multi-Raft*), the **Raft** recipe for a self-healing group, and **ra** (RabbitMQ's Raft for Erlang).

One committee per ontology

Each ontology is run by a small **committee** — say 3 or 5 machines — that holds it and agrees on every change. (“Committee” is the plain name; in Raft terms it's the group.)

- One member is **in charge** at a time. It takes incoming changes, bundles them into the next **block**, and sends that block to the others. A block becomes official once **more than half** the committee has written it down.
- If the one in charge dies or goes silent, the others **pick a new one among themselves** in a second or two. Automatic — no human, no config edit.
- The committee is the authority for *changing* the ontology. *Reading* it is a separate, much cheaper thing (see below).

The smallest committee is **one machine** — just “one machine in charge,” the simple dev setup. Grow the same committee to 3 or 5 and it survives some of them dying: **same code, more members**. So “simple now” and “robust later” are one thing, dialled by committee size.

Who's on the committee is stored in the ontology itself

Committee membership is **facts in the ontology** (as in bbsvx/onia), and a machine that wants to join is admitted by **proving a join-predicate** the ontology defines. Keep two kinds of change apart:

- **Who's in charge** changes by itself, only when the current one fails — not on a timer.
- **Who's on the committee** changes only *on purpose* — a machine joins (passing the join-predicate) or leaves/is removed — and every such change is **itself a recorded block**. So unlike Brahms (which constantly reshuffles its loose crowd), the committee is deliberately **stable**; that stability is what lets it keep an exact record. The first block names the starting committee; everything after is just more blocks — no chicken-and-egg.

Rotation is optional. If you want the committee to rotate so the same 3 machines don't hold an ontology forever (fairness, spreading load), that's a policy on top — e.g. onbrater's: every so many blocks, recompute the committee from the peer facts. Start stable; add rotation later if wanted.

Brahms vs the committee

Two different jobs, stacked. **Brahms = the crowd**: background gossip that keeps every machine *roughly* aware of who's around — big, loose, always shifting; it **finds**

machines but doesn't order changes or hold the ontology. **The committee = the small group** that actually holds one ontology and agrees on its changes. Brahms finds candidates; the committee is the small subset that actually runs the ontology, joined by a deliberate, recorded step. Being in the crowd $\neq$ being on the committee.

The history is the block list — kept in full, and browsable

Two separate things:

- **The current facts** — the ontology as it is now (what prove reads): the live Prolog database each member holds.
- **The history** — the ordered **block list**, every change ever made. The current facts are just this history replayed from the start.

Decision: keep the full history, permanently, and browsable (a ledger you can page through on a web page). So the block list is **saved durably** (to disk on the committee members) — a deliberate exception to quod's "no disk" goal, but only for the history. The current facts need no separate saving: they rebuild from the block list. **Rule: save the block list; derive everything else from it.** A web page reads the history straight off any committee member. *Trimming old history* (keep only a recent snapshot + changes since, to bound size) is an **option we add later** — default is keep everything.

Who holds it — first cut: only the committee. Each committee member keeps the full block list; read-copies hold current facts only, not the history. *Later (opt-in):* a `local_history(Node)` predicate in the ontology adds extra history-holders ("archive" nodes) beyond the committee — **committee members always store it; the predicate only adds more.** Being a predicate, it can be a rule (`local_history(N) :- has_big_disk(N)`). It expresses intent a node honors (usually self-declared — a remote ontology can't force your disk; gate who may assert it by ACL), and when a node becomes an archive it back-fills the past history once from a committee member. Not in the first version.

Reads don't go through the committee

The committee is the authority for **changing** an ontology. **Reading** it — calling a predicate to look something up — changes nothing and does **not** need the committee. A read only needs *a copy of the current facts*, and copies can be everywhere. This is the answer to "won't a popular ontology like root funnel everything through 3 nodes?":

- Other machines hold a **read-only copy** of root and answer reads locally — the 3 committee members are not in the path.

- Better, root's hot predicates (`isa`, `have_attribute`, class definitions) are **schema that barely changes**, so a caller keeps its **own copy** and is pinged only on the rare change. In steady state `root::isa(...)` is a **local lookup, zero network hops**.

Three tiers for a read: (1) your own cached copy — rarely-changing things like schema, no hops; (2) a read-copy near you — one hop; (3) the committee — only for a read that must be the guaranteed-latest value (rare). The committee's only real load is **edits**, and a root-type ontology is barely ever edited.

When the committee is a limit: only if an ontology gets many *edits* (not reads) — edits to one ontology are handled one at a time by its committee. Fix: **split an edit-heavy ontology into smaller ones**, each with its own committee → more edit capacity in parallel. (That's why we have one committee per ontology, not one global one.)

This read path is the “read-set is the subscription” idea (§3) doing double duty (a copy + notify-on-change). Honest caveats: caching *foreign* facts brings back the per-fact “where did this come from” tracking we deferred in §2; a cached copy can be a beat behind (fine for schema; go to the committee for guaranteed-latest); and the per-predicate notification granularity of §12 #2 applies (a root change pings every holder — fine because root rarely changes).

How a change commits (the edit interface)

Above the committee sits one small interface; *how* the committee agrees lives behind it, so the agreement can be made stronger later without touching anything above:

```
append(Ns, Change)          -> {ok, BlockIndex} | {error, not_in_charge, Hint}
% the committee then hands each official change, in order, to every member:
apply(BlockIndex, Change, Facts) -> Facts'          % deterministic
```

- A **change** is a committed transaction: `{TxId, CallerNs, Diff, ReadCheck}` — the asserts/retracts to apply, plus a note of what the proof relied on.
- **append** is how a write-transaction commits (§5): the one in charge orders it, copies it to the others, and it counts once a majority have it.
- **apply** runs on every member in the same order: **re-check what the proof relied on is still true; if so apply the diff; if not, reject it** (the caller retries). That is §6's “validate the read-set at the change's position,” located precisely. Same changes, same order = identical facts on every member.

We record the diff, not the goal. Members apply a concrete list of asserts/retracts; they do **not** re-run the Prolog (re-running isn't guaranteed identical across machines). Only the one in charge runs the proof.

How it plugs into the existing modules

```

quod_sup
├─ quod_quic                (transport – unchanged)
├─ quod_brahms_sup → quod_brahms per Ns  (discovery / the crowd – unchanged)
└─ quod_ns_sup      → per-ontology subtree                                ← NEW
    └─ quod_ledger   (Ns)  committee member: order + agree on blocks
        └─ quod_prolog (Ns)  the facts: apply blocks, answer proves

```

- Committee messages (who's in charge, catch-up, new blocks) ride `quod_link` streams on their **own channel {log, Ns}**, kept separate from Brahms's gossip so the two don't get stuck behind each other.
- **Brahms finds candidates; the committee list (in the ontology) is the authoritative one** (§12 #8).
- New `quod_reg` names: {`quod_ledger`, `Ns`}, {`quod_prolog`, `Ns`}.
- A member that restarts reloads its own on-disk history and/or **catches up from the others**. Losing one member is fine — that's the point of having several.

Make the agreement stronger later — same interface

The committee runs the **Raft** recipe to start (survives machines *crashing*). For ontologies where members might *lie*, the same interface can sit on a Byzantine agreement recipe later — the apply side and everything above are unchanged. Picked per ontology, by how much the members trust each other.

Still open

- **Hand-roll vs ra — decided: hand-roll a lean Raft over quod_link** (no Erlang distribution, matching how Brahms was built), using `ra` + the Raft paper as reference, not as a dependency.
- **Read freshness** — stale-copy-OK (default) vs go-to-committee for the guaranteed-latest value.
- **Read-copies** — who holds a read-only copy of a popular ontology, and how they're kept current (the §3 notify path).
- **One quod_prolog per ontology** serialises edits; read-only proofs run on cheap copies alongside.
- **Trimming history** — opt-in, later.
- Cross-ontology all-or-nothing edits stay a layer *above* this (§12 #3), Phase 4.

14. The event system — how a committed change notifies what reacts

Status: design for Phase 2 (Phase 1 handles only the narrow case at the end). Drawn from onia §14 (the D/P/E model) and §15 (the live-vs-replay split); bbsvx implements a looser version we improve on.

Three layers: the fact change, the derived views, the reactions

Every committed change runs in three layers, in order:

- **The fact change (D)** — the assert/retract into the kb. *This is what the ordering layer already applies (§13).*
- **Derived views (P)** — in-memory state computed from the facts: indexes, the "who-read-what" subscription table, (later) a scene graph. Rebuilt **synchronously, right after D**, so that by the time anyone is notified the derived views are already consistent.
- **Reactions (E)** — the outward/reactive stuff: `react_on` rules firing, messages sent, client/3D updates pushed. Runs **after commit, and only on a live commit**.

Order: commit → apply the diff (D) → refresh derived views (P) → fire reactions (E). D+P are synchronous and finished before anyone is notified; E is the async, outward part.

The one rule that matters: replay must not re-fire reactions

When a node **replays** committed history — catching up, restarting, a fresh committee member — it runs **only D** (rebuild the facts). It must **not** re-fire E: no re-sent messages, no re-pushed updates, no re-run `react_on`. Otherwise a node that falls behind and catches up would double-fire every side effect. (onia enforces this by delivering replay on a separate channel reactors don't listen to; **bbsvx gets this wrong** — it re-runs effects on replay and leans on idempotency, the wart we avoid.)

The ordering layer already draws this line: the **live** apply path fires reactions; the **replay** path (`ordering-layer-spec.md §4.6`) runs D only. Phase 2's notification hooks onto the live apply path, never replay.

react_on and effects are facts

A reaction is declared as content — `react_on(Pattern, Goal)` (stored as an `event_handler` fact); an operation's effects as `effect/4` facts. Fits the "config is facts + predicates" spine. The discipline that keeps replay safe: **effect bodies may only touch the kb; the outward stuff lives in the reaction (E) handlers fired on live delivery.**

This is how §3's "read-set is the subscription" actually works

"Notify the proved scopes that read a changed fact" is the E layer: after a live commit, match the changed facts against the read-sets / `react_on` patterns and fire the matching reactions. So §3's notification isn't a separate subsystem — it's event delivery on the live apply path. (Caveat: today's read-set is per-predicate, §12 #2, so notifications fire per predicate, not per fact.)

One change → one notification; and a loop guard

A single change that cascades into many diffs fires reactions **once, at the end** (one notification envelope), not per-op. And a reaction triggered by a notification must not freely write back and re-trigger — bbsvx uses a "firewall" (a subscription-triggered handler can't itself write) to stop amplification loops; we'll need the same.

Phasing

- **Phase 1 (already in the build spec):** the only reactions are a *write's own deferred effects* — they fire **once, on the submitting node, at commit** (parked by `tx_id`), never on other members, never on replay (at-most-once). The narrow, safe case; the spec implements it.
- **Phase 2:** generalize to **reactors** — `react_on` rules, the read-set→notification index, cross-ontology notification — all on the live apply path, with the loop guard. Out of scope for the first build.

Ordering layer — Phase 1 build spec (quod_ledger / quod_prolog)

This document is the single, unified Phase-1 build spec for quod's ordering/content layer, realizing doc/content-layer-design.md §13. The decisions it fixes: a **hand-rolled lean Raft** over quod_link streams (channel {log, Ns}), with **no Erlang distribution; one committee (one Raft group) per namespace**; a **full, durable, browsable block list** kept committee-only in this first cut; and an MVP scope of **own-namespace writes + cross-namespace reads** (foreign writes, BFT, and read-copy fan-out are out of scope). quod_ledger is the per-namespace Raft gen_statem that orders and replicates blocks; quod_prolog is its sibling gen_server that deterministically applies committed blocks and serves proofs. Both live under a two-level supervision tree (quod_ns_sup → quod_ns → {quod_ledger, quod_prolog}).

Module map

```
quod_sup (one_for_one)
├─ quod_quic                                (transport, existing)
├─ quod_brahms_sup (simple_one_for_one)      membership, unchanged -
discovery only
│   └─ quod_brahms (per Ns)
├─ quod_ns_sup    (simple_one_for_one) THIS spec – subtree root, {quod_ns_sup, node}
│   └─ quod_ns     (per Ns, rest_for_one) {quod_ns, Ns}
│       └─ quod_ledger (Ns)                {quod_ledger, Ns} Raft committee member
│       └─ quod_prolog (Ns)                {quod_prolog, Ns} fact store + prove engine
└─ quod_metrics
```

- **quod_ledger** — gen_statem, one per namespace; orders + agrees on blocks; owns the durable log.
- **quod_prolog** — gen_server, one per namespace; applies committed blocks; serves prove.
- **quod_ledger_store** — library module (no process); on-disk persistence for quod_ledger.
- **quod_diff** — pure library; differ write-set (op-log) + per-functor read-set hashes.
- **quod_ns** — per-Ns rest_for_one sub-supervisor over {quod_ledger, quod_prolog}.
- **quod_ns_sup** — simple_one_for_one parent of quod_ns instances.

Channel: term_to_binary({log, Ns}, [deterministic]) everywhere (the single wire channel, distinct from Brahms's {channel, Ns}).

Reg keys: {quod_ns_sup, node}, {quod_ns, Ns}, {quod_ledger, Ns}, {quod_prolog, Ns}, plus the channel property {channel, term_to_binary({log, Ns}, [deterministic])}.

Shared header include/quod_ledger.hrl is the single source of truth for term_no(), log_index(), server_id(), clause(), op(), read_check(), #entry{}, #transaction{}, and the six Raft RPC records. include/quod_ledger_store.hrl -includes it and defines only the private #store{} / handle().

Table of contents

1. [quod_ledger state machine](#)
2. [Wire protocol over quod_link](#)
3. [Persistence \(quod_ledger_store\)](#)
4. [quod_prolog apply + prove](#)
5. [quod_ns_sup lifecycle + membership](#)
6. [Build order + test plan](#)
7. [Open questions / out of scope for Phase 1](#)

Shared header include/quod_ledger.hrl (canonical types + records)

Every type and record below is defined **once**, here, and -included by quod_ledger, quod_ledger_store (transitively), quod_prolog, and tests. Field names are snake_case throughout.

```
%% include/quod_ledger.hrl
-type term_no() :: non_neg_integer().           %% Raft Term, starts 0
-type log_index() :: non_neg_integer().          %% 0 = empty-log / snapshot sentinel; entries
  ↳ 1..N
-type pubkey() :: binary().                      %% Ed25519 public key (32 bytes)
-type node_id() :: pubkey().                    %% a node's STABLE identity == its pubkey
  ↳ (built; A.3)
-type endpoint() :: {inet:hostname(), inet:port_number()}.  %% where a node is dialed (a
  ↳ routing hint)

%% a Prolog clause; identity is content only (#1)
-type clause() :: {Head :: term(), Body :: term()}.  %% Body == true for a plain fact

%% the differ's write-set: ordered op-log of asserts/retracts (#3)
-type op() :: {assert, clause()} | {retract, clause()}.

%% read-set: per-functor content hash, per-predicate (#4)
-type read_check() :: #{ {Functor :: atom(), Arity :: non_neg_integer()} => integer() }.
```

```

%% the committed change record – FIXED shape (#24).
%% `author`/`sig` are RESERVED for signing (identity readiness, see below): Phase 1
%% sets author = self node id, sig = none, and verification is a pass-through stub.
%% Reserving them now keeps the wire + on-disk format stable when signing turns on.
-record(transaction, {tx_id      :: binary(),           %% unique per transaction (ulid)
                      caller_ns  :: binary(),           %% CallerNs (#5), first-class (#2)
                      diff       :: [op()],             %% concrete asserts/retracts (#25)
                      read_check :: read_check(),       %% what the proof relied on (#24)
                      author     :: server_id(),         %% who submitted it (Phase 1: node id;
                      ⇨ later: pubkey())
                      sig = none :: binary() | none}).    %% Ed25519 sig over canonical bytes;
                      ⇨ none in Phase 1

%% Raft log entry (§E.0). `data` is a #transaction{} for blocks (or `noop` for the
⇨ election marker).
-record(entry, {index :: log_index(),
               term  :: term_no(),
               kind  :: block | config,
               data  :: #transaction{} | noop | {add, server_id()} | {remove,
               ⇨ server_id()}}).

%% the six Raft RPC records – exact fields (snake_case)
-record(request_vote, {term      :: term_no(),
                      candidate_id :: server_id(),
                      last_log_index :: log_index(),
                      last_log_term  :: term_no()}).
-record(request_vote_reply, {term      :: term_no(),
                             vote_granted :: boolean()}).
-record(append_entries, {term      :: term_no(),
                        leader_id  :: server_id(),
                        prev_log_index :: log_index(),
                        prev_log_term  :: term_no(),
                        entries      :: [#entry{}],    %% [] for heartbeat
                        leader_commit :: log_index()}).
-record(append_entries_reply, {term      :: term_no(),
                              success    :: boolean(),
                              match_index :: log_index()}).    %% success: matched idx;
                              ⇨ fail: conflict hint
-record(install_snapshot, {term      :: term_no(),
                          leader_id  :: server_id(),
                          last_included_index :: log_index(),
                          last_included_term  :: term_no(),
                          config          :: [server_id()],    %% committee as of
                          ⇨ snapshot
                          data           :: binary()}).         %% already
                          ⇨ term_to_binary'd state
-record(install_snapshot_reply, {term :: term_no()}).

```

`read_check()` values are erlang:phash2-style **integers** (one hash per {Functor, Arity}), produced by `quod_diff:functor_hash/3` and compared by `quod_prolog`'s OCC re-check. Producer and validator use the same integer representation.

Identity & signing readiness

Updated (identity milestone A.3, 2026-06-30). Identity is now the node's **Ed25519 pubkey** (`node_id()`), not its address. The bullets below describe the model as built; the signing half (`#transaction.sig`, quorum certificates) is Phase B, still pending.

A committee member's identity is its **pubkey** (`node_id() = pubkey()`), generated on first boot and persisted; the address `{Host, Port}` (`endpoint()`) is demoted to “where you dial it” — a routing hint resolved on connect. This survives a host move (the durable membership is pubkey-only) and lets **mutual TLS** prove who a node is.

- **#transaction.author / #transaction.sig** — author is the submitting node's pubkey (set at every `#transaction{}` build site). `sig` is the Ed25519 signature over the canonical bytes `term_to_binary({tx_id, caller_ns, diff, read_check}, [deterministic])` — **still none until Phase B**; `verify_change/2` is a pass-through stub until then. The byte-size guard and `tx_id` correlation are unaffected.
- **Identity vs. address (built).** `node_id()` is the `pubkey()`; `{Host, Port}` is an `endpoint()` hint. Committee config entries (`{add, NodeId}`) and `voted_for/leader_id` carry the **key**, not the address. The address is learned from the authenticated link header (`{Pubkey, Addr}`) into a resolver cache (`quod_quic`), and a redirect resolves the leader's key → endpoint. (We did **not** adopt onbrater's `peer_admitted/4` recorded admission fact — the address stays a volatile hint; Brahms gossiping `{pubkey, addr}` for sparse-seed topologies is a later refinement.)
- **Block-level proof (commit certificate).** A subscriber verifying a *relayed* block (P2 epidemic dissemination) needs each committed block to carry the quorum's signatures. **Phase B** adds it (a per-entry quorum certificate + a verify API) — superseding the earlier “BFT-only, not reserved” framing; full per-voter Byzantine validation is still a later milestone.

What identity built (A.3) vs. Phase B: the node keypair is generated at boot, persisted, and **used** — it is the `node_id` and the mutual-TLS cert, and the transport binds the proven peer pubkey. Still **stubbed until Phase B**: `#transaction.sig` (none) and `verify_change/2` (pass-through), and the per-block quorum certificate. The no-identity/test path keeps using the address as the id transitionally.

1. quod_ledger state machine

Module: /home/yan/src/quod/src/quod_ledger.erl — one `gen_statem` per ontology namespace, the committee member that orders and agrees on blocks. Hand-rolled lean Raft over `quod_link` streams on channel `term_to_binary({log, Ns}, [deterministic])`, no Erlang distribution. It is the append/replicate side; `quod_prolog` is the apply/prove side, driven by this module's `commit→apply` loop.

1.1 Assumptions

1. **`node_id()` = `pubkey()`** is the committee-member id used as `voted_for/leader_id/candidate` and as map keys (its `address_endpoint()` is a resolvable hint). (A.3: *was {Host, Port}; the no-identity/test path still uses an endpoint as the id transitionally.*)
2. **Durability is delegated to `quod_ledger_store`** (§3). Every `fsync`-ordering rule is expressed as "call `quod_ledger_store:*` and only on its `ok/{ok, Store1}` return send the network reply / count toward commit." The handle is threaded through `#d.store`.
3. **Block payload is opaque to `quod_ledger`**. The application command is a `#transaction{}` (§ shared header); `quod_ledger` carries it as `#entry.data` for `kind=block` and never inspects it. The election no-op is the one block entry whose data is the atom `noop`.
4. **config re-derivation is a whole-log fold** (`derive_committee/1`) on every `append/truncate` touching a config entry. Cheap at committee sizes 1/3/5.
5. **InstallSnapshot + compaction arrive in M3**, not M1/M2. Until M3 no compaction occurs and `snap_idx=0`, so the §8 "`prev_log_index < snap_idx`" overlap case never triggers. The `#install_snapshot{}` records are defined from the start (harmless) but only sent/received from M3.
6. **Channel binary** `term_to_binary({log, Ns}, [deterministic])` is computed once in `init/1` and cached in `#d.chan`.

1.2 Behaviour, exports, callback_mode

```
-module(quod_ledger).
-moduledoc """
Per-namespace lean **Raft** committee member: orders and replicates the
ontology's block list over the dedicated `{log, Ns}` `quod_link` channel.
One `gen_statem` per ontology. States `follower` | `candidate` | `leader`.
""".
-behaviour(gen_statem).
-include("quod_ledger.hrl").

%% API
-export([start_link/2,
         append/2, add_member/2, remove_member/2,
```

```

        status/1, committee/1, stats/1, namespaces/0,
        replay/1, snapshot/1, history/3]).
%% gen_statem
-export([init/1, callback_mode/0, terminate/3]).
%% states
-export([follower/3, candidate/3, leader/3]).

-ifdef(TEST).
-export([last_log_index/1, last_log_term/1, term_at/2, quorum/1,
        derive_committee/1, up_to_date/4, advance_commit/1,
        truncate_append/2, encode/1, decode/1]).
-endif.

callback_mode() -> [state_functions].

```

start_namespace/2 does **not** live on quod_ledger — it lives on quod_ns_sup (§5).
 quod_ledger:start_link/2 is invoked by quod_ns.

Each state is Name(EventType, Event, D) with a catch-all delegating to a shared common/3:

```

follower(EventType, Event, D) -> common(EventType, Event, D).
candidate(EventType, Event, D) -> common(EventType, Event, D).
leader(EventType, Event, D)    -> common(EventType, Event, D).

```

1.3 The #d{} record — PERSISTED vs VOLATILE

```

-record(d, {
    %% ---- scaffolding (mirrors quod_brahms #d{}) ----
    ns      :: binary(),
    self    :: server_id(),           %% own NodeId {Host,Port} (= server_id)
    cfg     :: map(),                 %% maps:merge(?DEFAULTS, Config)
    chan    :: binary(),              %% term_to_binary({log, Ns}, [deterministic])
    conns   = #{ } :: #{server_id() => {pid(), reference(), out | in}}, %% NodeId =>
    ↪      {LinkPid, Mon, Origin}
    outbox  = #{ } :: #{server_id() => [binary()]}, %% per-peer FIFO queued while a link
    ↪      opens
    rx      = #{ } :: #{server_id() => #rx{}},      %% per-peer chunk reassembly (§2)

    %% ===== PERSISTED (fsync via quod_ledger_store before reply) =====
    cur_term = 0      :: term_no(),
    voted_for = none :: server_id() | none,
    log       = []    :: [#entry{}],              %% index 1..N, no gaps – IS the durable
    ↪      block list
    snap_idx  = 0      :: log_index(),
    snap_term = 0      :: term_no(),
    snap_cfg  = []     :: [server_id()],          %% committee as of snapshot

```

```

%% ===== VOLATILE (reconstructed on restart) =====
commit_index = 0 :: log_index(),
last_applied = 0 :: log_index(),
next_index   = #{ } :: #{server_id() => log_index()}, %% leader only
match_index  = #{ } :: #{server_id() => log_index()}, %% leader only
votes        = #{ } :: #{server_id() => boolean()}, %% candidate: votes this term
leader_id    = none :: server_id() | none,
cfg_uncommitted = false :: boolean(), %% one membership change in flight (#9.1)
pending      = #{ } :: #{log_index() => gen_statem:from()}, %% client appends awaiting
↳ commit

%% ---- handle + metrics ----
store      :: quod_ledger_store:handle(),
elections  = 0 :: non_neg_integer(),
appends    = 0 :: non_neg_integer(),
commits    = 0 :: non_neg_integer(),
snapshots_installed = 0 :: non_neg_integer(),
%% transport counters
msgs_sent  = 0 :: non_neg_integer(),
msgs_recv  = 0 :: non_neg_integer(),
bytes_sent = 0 :: non_neg_integer(),
chunks_sent = 0 :: non_neg_integer(),
rx_dropped = 0 :: non_neg_integer()).

```

Persistence contract. `cur_term`, `voted_for`, `log`, and the `snapshot` fields are durable, reloaded in `init/1`. Everything else is rebuilt (`commit_index/last_applied` reset to `snap_idx` on restart, then re-advance from heartbeats / re-application).

```

-define(DEFAULTS,
    #{node_id      => undefined, %% own NodeId; REQUIRED (valid_cfg rejects undefined)
      mode         => create,    %% create | join ($5)
      committee    => [],        %% bootstrap committee; [] => self-only 1-voter (#18)
      heartbeat_ms => 150,       %% leader-peers AppendEntries cadence (<< election)
      election_ms   => 1000,     %% base election timeout; randomized to [T, 2T]
      election_jit  => 1.0,
      max_batch     => 256,      %% max #entries per AppendEntries (kept under
      ↳ ?CHUNK_BYTES)
      max_pending   => 1024,     %% backpressure cap on in-flight client appends (M5)
      snapshot_every => 10000,   %% compaction cadence (M3); local policy
      max_snapshot_bytes => (64 bsl 20), %% reassembled snapshot cap ($2)
      data_dir      => undefined}). %% threaded from application:get_env(quod, data_dir)
-define(MAX_FRAME_BYTES, (1 bsl 20)). %% quod_link hard cap
-define(MAX_RAFT_BYTES, 65536). %% reassembled cap for non-snapshot msgs ($2)
-define(CHUNK_BYTES, 65536). %% per-frame chunk payload ($2)

```

1.4 init/1 — load durable state + 1-voter self-bootstrap

```

init({Ns, Config}) ->
  Cfg = maps:merge(?DEFAULTS, Config),
  case valid_cfg(Config, Cfg) of
    {error, Reason} -> {stop, {bad_config, Reason}};
    ok ->
      Self    = maps:get(node_id, Cfg),
      Chan    = term_to_binary({log, Ns}, [deterministic]),
      DataDir = maps:get(data_dir, Cfg),
      quod_reg:subscribe({channel, Chan}),
      {ok, Store} = quod_ledger_store:open(Ns, DataDir),
      D0 = #d{ns = Ns, self = Self, cfg = Cfg, chan = Chan, store = Store},
      D1 = load_or_bootstrap(D0, Cfg),           %% durable reload OR founding config
      D2 = open_committee_links(D1),
      D3 = D2#d{commit_index = D2#d.snap_idx, last_applied = D2#d.snap_idx},
      {ok, follower, D3, [election_timeout(D3)]}
  end.

```

load_or_bootstrap/2:

- **Restart path (#29):** quod_ledger_store:load(Store) returns #{cur_term, voted_for, log, snap_idx, snap_term, snap_cfg, snap_data} → populate the PERSISTED fields. Committee is derive_committee/1.
- **Founding path (mode=create, 1-voter, #18):** store empty AND committee ∈ {[], [Self]}. Seed a committed kind=config {add, Self} (equivalently snap_cfg = [Self] with empty log), durable. Yields committee() = [Self], quorum() = 1. It wins its first election uncontested.
- **mode=join:** store empty, **no genesis config**, election timer **not** armed — passive learner until it observes the committed {add, self} config entry (§5).
- **Multi-member bootstrap:** non-empty committee seeds snap_cfg to that list, empty log, term 0.

valid_cfg(RawConfig, MergedCfg) -> ok | {error, Reason}: rejects node_id == undefined, non-binary Ns, non-list committee, non-positive timers, heartbeat_ms >= election_ms.

1.5 Timers

```

election_ms(Cfg) ->
  T = maps:get(election_ms, Cfg), J = maps:get(election_jit, Cfg),
  T + round(rand:uniform() * J * T).
election_timeout(#d{cfg = Cfg}) -> {state_timeout, election_ms(Cfg), election}.
heartbeat_timeout(#d{cfg = Cfg}) -> {state_timeout, maps:get(heartbeat_ms, Cfg),
  ↪ heartbeat}.

```

follower/candidate arm {state_timeout, _, election} (firing → start election; reset on valid AppendEntries for current term or granting a vote). leader arms {state_timeout, heartbeat_ms, heartbeat} and never an election timer. state_timeout auto-cancels on any state transition.

1.6 RequestVote — request & reply

Receiver handle_request_vote(Peer, #request_vote{} = RV, State, D):

1. RV.term < cur_term → reply #request_vote_reply{term=cur_term, vote_granted=false}, {keep_state, D}.
2. RV.term > cur_term → step_down/2 (persist {cur_term := RV.term, voted_for := none}), continue at new term; resulting transition is to follower.
3. Grant iff voted_for ∈ {none, RV.candidate_id} **and** up_to_date(RV.last_log_term, RV.last_log_index, last_log_term(D), last_log_index(D)):

```
up_to_date(CandT, CandI, MyT, MyI) -> CandT > MyT otherwise (CandT == MyT andalso CandI
  ↪ == MyI).
```

4. **If granting:** D1 = D#d{voted_for = RV.candidate_id}, quod_ledger_store:write_meta(State, cur_term, candidate_id) — durable — then reply vote_granted=true, re-arm election timer, {next_state, follower, D1', [election_timeout(D1')]}.
5. Else reply vote_granted=false.

Candidate receiving #request_vote_reply{}: ignore if Reply.term < cur_term or no longer candidate. Reply.term > cur_term → step_down, {next_state, follower, ...}. If vote_granted → votes#{Peer => true}; if count(true votes incl. self) >= quorum(D) → **become leader** (§1.9).

1.7 AppendEntries — request & reply

Receiver handle_append_entries(Peer, #append_entries{} = AE, State, D):

1. AE.term < cur_term → reply #append_entries_reply{term=cur_term, success=false, match_index=0}, {keep_state, D}.
2. AE.term > cur_term → step_down/2 (persist).
3. Record leader_id = AE.leader_id and **transition to follower** ({next_state, follower, ...} from any prior state — a candidate at equal term steps down to this valid leader), re-arming the election timer.
4. **Consistency check** on AE.prev_log_index/prev_log_term:
 - prev_log_index == 0 → OK.
 - prev_log_index > last_log_index(D) → fail, reply match_index = last_log_index(D).
 - prev_log_index == snap_idx → require prev_log_term == snap_term.

- else `term_at(prev_log_index, D) /= prev_log_term` → fail, reply `match_index = prev_log_index - 1`.
 - On failure: `success=false`, re-arm election timer, stay follower.
5. **Truncate + append** (`truncate_append/2, pure`): same-index-different-term existing entry → drop it and the tail, append the suffix; same-index-same-term → skip (idempotent, **never truncate on a match**); no entry → append. If any touched entry is `kind=config` → re-derive committee, set/clear `cfg_uncommitted`. `quod_ledger_store:append(Store, NewEntries)` — durable — before replying `success=true`. On a truncation, **fail any pending From at an index > commit_index** with `{error, not_in_charge, leader_id}`.
 6. **Advance commit**: if `AE.leader_commit > commit_index`, set `commit_index := min(leader_commit, index_of_last_NEW_entry)`, then drive `apply($1.10)`.
 7. Reply `#append_entries_reply{term=cur_term, success=true, match_index = prev_log_index + length(AE.entries)}`, `{next_state, follower, D', [election_timeout(D')]}`.

Leader receiving `#append_entries_reply{}`:

- `Reply.term > cur_term` → `step_down, {next_state, follower, ...}`.
- Ignore if not leader or `Reply.term < cur_term`.
- **On success**: `match_index#{Peer => max(old, Reply.match_index)}`, `next_index#{Peer => Reply.match_index + 1}`. Run `advance_commit/1` (§1.10); if `next_index[Peer] <= last_log_index`, send the next batch (pipelining).
- **On failure**: `next_index#{Peer => max(1, Reply.match_index + 1)}` (never below `snap_idx+1`), re-send `AppendEntries` with the lower `prev_log_index`. If it would drop to `<= snap_idx`, that is the `InstallSnapshot` trigger (M3).

1.8 The replication loop (leader)

`replicate/1` runs on (a) heartbeat timeout, (b) just after a local append, (c) after a successful AE reply that left a peer behind. For each peer $P \in \text{committee()} \setminus \{\text{self}\}$:

```
PrevI = next_index[P] - 1,
PrevT = term_at(PrevI, D),
Entries = log_from(next_index[P], max_batch, D),    %% [] => pure heartbeat
AE = #append_entries{term=cur_term, leader_id=self,
                     prev_log_index=PrevI, prev_log_term=PrevT,
                     entries=Entries, leader_commit=commit_index},
send_raft(P, AE, D)
```

Heartbeats are sent unconditionally on the heartbeat timer, including with no peers (1-voter: the loop is over []). Each batch is capped at `max_batch` entries, kept under `?CHUNK_BYTES` so single-frame AE stays unchunked in the common case.

1.9 Becoming candidate → becoming leader

start_election/1 (election timeout in follower/candidate) — guarded by `lists:member(self, derive_committee(D))` (a removed server does not start elections):

```
T1 = cur_term + 1,
D1 = D#d{cur_term = T1, voted_for = self, votes = #{self => true}, leader_id = none},
ok = quod_ledger_store:write_meta(Store, T1, self),      %% DURABLE before any RPC
broadcast RequestVote{term=T1, candidate_id=self,
                      last_log_index=last_log_index(D1), last_log_term=last_log_term(D1)},
{next_state, candidate, D1#d{elections = elections+1}, [election_timeout(D1)]}
```

If `quorum(D1) == 1`, the self-vote satisfies quorum → fall straight through to `become_leader/1`.

become_leader/1 (on reaching quorum of granted votes):

```
LLI = last_log_index(D),
Next = maps:from_list([P, LLI + 1] || P <- peers(D)),
Match = maps:from_list([P, 0] || P <- peers(D)),
D1 = D#d{next_index = Next, match_index = Match, leader_id = self, votes = #{}},
%% MANDATORY current-term no-op (#7, Figure 8): append a no-op kind=block of the new term
↪ so
%% prior-term entries a majority holds can commit (indirectly), and followers can advance
↪ commit
%% even with no client write. Commits via the SAME advance_commit/1 rule; fsync'd before
↪ counting.
I = LLI + 1,
E = #entry{index = I, term = cur_term, kind = block, data = noop},
D2 = D1#d{log = D1#d.log ++ [E]},
{ok, Store1} = quod_ledger_store:append(D2#d.store, [E]),
D3 = case quorum(D2) of 1 -> advance_commit(D2#d{store=Store1}); _ -> D2#d{store=Store1}
↪ end,
{next_state, leader, D3, [replicate_now, heartbeat_timeout(D3)]}
```

1.10 Commit rule + in-order apply into quod_prolog

Commit rule (leader, advance_commit/1) — mandatory current-term guard:

```

advance_commit(D = #d{commit_index = C}) ->
  LLI = last_log_index(D),
  Ns = [N || N <- lists:seq(C+1, LLI),
        replicated_majority(N, D),
        term_at(N, D) == D#d.cur_term],
  case Ns of [] -> D; _ -> apply_committed(D#d{commit_index = lists:max(Ns)}) end.

replicated_majority(N, D) ->
  Count = 1 + length([P || P <- peers(D), maps:get(P, D#d.match_index, 0) >= N]),
  Count >= quorum(D).

```

A prior-term entry commits only *indirectly* — when a current-term entry above it commits (the §1.9 no-op guarantees one). Counting a prior-term entry committed by replica count alone is the Raft Figure-8 bug and is never done.

Apply loop (all servers, apply_committed/1) — drives quod_prolog:

```

apply_committed(D = #d{last_applied = LA, commit_index = CI}) when LA >= CI -> D;
apply_committed(D = #d{last_applied = LA}) ->
  I = LA + 1,
  #entry{kind = Kind, data = Data} = entry_at(I, D),
  D1 = case Kind of
    block ->
      Verdict = quod_prolog:apply_block(D#d.ns, I, Data), %% ok | {reject,
        ↪ conflict}
      D2 = reply_pending(I, Verdict, D), %% see below
      D2#d{commits = D2#d.commits + 1};
    config ->
      drop_departed(Data, clear_cfg_gate(Data, D))
  end,
  apply_committed(D1#d{last_applied = I}).

```

apply_block/3 is a **synchronous gen_server:call** into quod_prolog, returning ok | {reject, conflict}. The OCC read-set re-check happens at apply on **every** member; because every member re-checks deterministically against the same converged KB at the same index, the verdict is identical everywhere. quod_ledger surfaces the verdict to the parked client:

```

reply_pending(I, Verdict, D) ->
  case maps:take(I, D#d.pending) of
    {From, Pending1} ->
      Reply = case Verdict of ok -> {ok, I}; {reject, conflict} -> {error,
        ↪ conflict_retry} end,
      gen_statem:reply(From, Reply),
      D#d{pending = Pending1};
    error -> D
  end.

```

A rejected block stays committed in the log as a no-op (it changed no facts), preserving identical prefixes. The noop election marker applies as nothing. config apply clears the one-change gate and, on the leader, drops a removed peer's volatile next_index/match_index slots:

```
drop_departed({remove, Node}, D) ->
    D#d{next_index = maps:remove(Node, D#d.next_index),
        match_index = maps:remove(Node, D#d.match_index)};
drop_departed(_, D) -> D.
```

1.11 The append API + membership-change entries

```
-spec append(binary(), #transaction{}) ->
    {ok, BlockIndex :: log_index()} | {error, not_in_charge, Hint :: server_id() | none}
    | {error, conflict_retry} | {error, busy}.
append(Ns, Change) ->
    try gen_statem:call(quod_reg:via({quod_ledger, Ns}), {append, Change}, 5000)
    catch exit:_ -> {error, not_in_charge, unavailable} end.
```

Served only in **leader**:

```
leader({call, From}, {append, Change}, D) ->
    case maps:size(D#d.pending) >= maps:get(max_pending, D#d.cfg) of
        true -> {keep_state, D, [{reply, From, {error, busy}}]};
        false ->
            I = last_log_index(D) + 1,
            E = #entry{index = I, term = D#d.cur_term, kind = block, data = Change},
            {ok, Store1} = quod_ledger_store:append(D#d.store, [E]), %% DURABLE before
            ↪ counting own log
            D1 = D#d{log = D#d.log ++ [E], store = Store1,
                    appends = D#d.appends + 1,
                    pending = (D#d.pending)#{I => From}}, %% From replied from
            ↪ apply step
            D2 = case quorum(D1) of 1 -> advance_commit(D1); _ -> D1 end,
            {keep_state, D2, [replicate_now]}
    end;

follower({call, From}, {append, _}, D) ->
    {keep_state, D, [{reply, From, {error, not_in_charge, D#d.leader_id}}]};
candidate({call, From}, {append, _}, D) ->
    {keep_state, D, [{reply, From, {error, not_in_charge, none}}]}.
```

From is **not** replied here — it is parked in pending keyed by log_index and replied from the apply step (§1.10) so the reply reflects the OCC verdict. On leader step-down, fail all pending above commit_index with {error, not_in_charge, leader_id}. replicate_now is an internal event ({next_event, internal, replicate}).

Membership change (kind=config, one at a time):

```

add_member(Ns, Node)    -> gen_statem:call(quod_reg:via({quod_ledger, Ns}), {add_member,
↪ Node}, 5000).
remove_member(Ns, Node) -> gen_statem:call(quod_reg:via({quod_ledger, Ns}),
↪ {remove_member, Node}, 5000).

leader({call, From}, {Op, Node}, D) when Op == add_member; Op == remove_member ->
  case D#d.cfg_uncommitted of
    true  -> {keep_state, D, [{reply, From, {error, config_in_flight}}]};
    false ->
      Data = case Op of add_member -> {add, Node}; remove_member -> {remove, Node}
      ↪ end,
      I    = last_log_index(D) + 1,
      E    = #entry{index = I, term = D#d.cur_term, kind = config, data = Data},
      {ok, Store1} = quod_ledger_store:append(D#d.store, [E]),
      D1 = D#d{log = D#d.log ++ [E], store = Store1, cfg_uncommitted = true},
      D2 = D1#d{next_index = ensure_peer(Node, D1)}, %% adopt-on-append
      D3 = open_committee_links(D2),
      D4 = case quorum(D3) of 1 -> advance_commit(D3); _ -> D3 end,
      {keep_state, D4, [{reply, From, {ok, I}}, replicate_now]}
  end.

```

- **Adopt-on-append:** committee/1 derives from the log, so a new member counts in quorum()/commit math the instant the entry is appended. Never wait for commit to adopt.
- **One in flight:** cfg_uncommitted=true blocks the next change until the apply loop clears it.
- **Leader removing itself** ({remove, SelfNodeId}): stays leader until the entry commits (under the new config excluding it), then steps down. start_election/1 is guarded by lists:member(self, derive_committee(D)).

```

committee(Ns) ->
  try gen_statem:call(quod_reg:via({quod_ledger, Ns}), get_committee, 1000) catch exit:_
  ↪ -> [] end.

derive_committee(#d{snap_cfg = Base, log = Log}) ->
  lists:foldl(fun(#entry{kind = config, data = {add, S}}, Acc) -> [S | Acc -- [S]];
              (#entry{kind = config, data = {remove, S}}, Acc) -> Acc -- [S];
              (_, Acc) -> Acc
            end, Base, Log).

quorum(D) -> (length(derive_committee(D)) div 2) + 1.
peers(D)  -> derive_committee(D) -- [D#d.self].

```

get_committee/status/stats are served in common/3 (any state). replay/1 (§4) resets last_applied := snap_idx and re-drives apply_block/3 over snap_idx+1 .. commit_index for a freshly-restarted quod_prolog. snapshot/1 returns {ok, snap_idx, snap_data} | none for KB seeding. history/3 (§3) serves paged browsing.

1.12 Edge cases

- **Stale replies** (old term / wrong state): validated against `cur_term/state`; ignored.
- **Duplicate/delayed AppendEntries**: same-index-same-term entries skipped (never truncate on a match).
- **Foreign-channel / undecodable**: dropped in `common/3` / pre-decode (§2).
- **Link death** (`{'DOWN', ...}` / `{link_error, ...}`): drop the cached conn; next heartbeat re-opens.
- **CP under partition (#16)**: a minority leader cannot reach quorum → `advance_commit` never advances → writes hang (caller times out → `{error, not_in_charge, unavailable}`). Deliberate CP choice.
- **Restart mid-term**: `init/1` reloads durable state; `commit_index/last_applied` reset to `snap_idx`. Re-application into `quod_prolog` must be idempotent on `quod_prolog`'s side.

1.13 Conventions

`logger:info("quod[~s]: <msg>", [Ns | Args]); catch exit:_ around gen_statem:call; [safe] size-capped decode; side-effect returns ignored with _ =; stats/1 returns one snake_case key set (§1.14); terminate/3 does quod_reg:unsubscribe({channel, Chan}) in a catch plus quod_ledger_store:close(Store).`

1.14 stats/1

One `snake_case` key set (the union; consumed by `quod_metrics` via **partial** map match so adding keys is safe):

```
stats(Ns) ->
  try gen_statem:call(quod_reg:via({quod_ledger, Ns}), get_stats, 1000) catch exit:_ ->
    ↪ undefined end.

%% get_stats reply:
#{cur_term => T, commit_index => C, last_applied => A, log_len => L,
  committee_size => CS, is_leader => IL, role => follower|candidate|leader,
  elections => E, appends => AP, commits => CM, snapshots_installed => SI,
  pending_appends => PA,
  msgs_sent => MS, msgs_recv => MR, bytes_sent => BS, chunks_sent => CHS, rx_dropped => RD}
```

2. Wire protocol over quod_link

This is the transport/codec layer of quod_ledger: record shapes (shared header), encoding, chunking, link bookkeeping, and the three transport operations.

2.1 Channel

```
Chan = term_to_binary({log, Ns}, [deterministic]),
quod_reg:subscribe({channel, Chan}),
```

[deterministic] is mandatory: quod_quic:open_link/2 and quod_reg:subscribe/1 key on the binary; a non-deterministic encoding would split the channel across nodes/OTP versions. terminate/3 mirrors with quod_reg:unsubscribe({channel, Chan}) in catch _:_ -> ok. {channel, Chan} (a binary-keyed gproc property) and {quod_ledger, Ns} (a gproc name) do not collide.

2.2 Records & types

server_id(), term_no(), log_index(), clause(), op(), read_check(), #entry{}, #transaction{}, and the six RPC records live in include/quod_ledger.hrl (\$ shared header), -included at the top of quod_ledger.erl **before** #d{}/#rx{} so server_id()/#entry{} are in scope. install_snapshot.data is an already-serialized binary(); the whole record is encode/1'd once like any other record (no separate re-wrap of data; the concatenation of its chunk parts is exactly encode(#install_snapshot{})).

2.3 #rx{} reassembly record (defined before #d{})

```
-record(rx, {msg_id :: reference(),
            total  :: pos_integer(),
            got    :: #{pos_integer() => binary()},
            bytes  :: non_neg_integer()}).
```

2.4 Encoding / decoding

```
encode(Msg)      -> term_to_binary(Msg).
decode(Bin)      -> try binary_to_term(Bin, [safe]) of T -> T catch _:_ -> error end.
decode_record(Bin) -> try binary_to_term(Bin)           of T -> T catch _:_ -> error end.
```

The wire **envelope** (`{raft|raft_chunk, Ns, ...}`) holds only known atoms + binaries, so it decodes with **[safe]** (`decode/1`) — refusing unknown atoms / fun / pid / port. But the **inner record** is an `#append_entries{}` carrying a `#transaction{}` whose diff is arbitrary Prolog clauses — i.e. atoms the receiver *has not seen yet* (the fact's own functor/args). `[safe]` would refuse those legitimately-new atoms and drop every fact-bearing block, so a follower could never learn a new fact (it only learns the atom by applying it — chicken-and-egg). The inner record therefore decodes **without [safe]** (`decode_record/1`), bounded by the `?MAX_RAFT_BYTES` size cap. This is sound for Phase 1's **trusted single-operator committee**; the later identity/BFT layer re-tightens it with signed, schema-validated changes. (Original spec said `[safe]` was mandatory everywhere — corrected here: it cannot be, for the application payload.) Every record is a tagged tuple; dispatch is a pattern match with a final `_ -> D` drop clause.

2.5 Max payload, chunking & reassembly

```
-define(MAX_RAFT_BYTES, 65536).    %% reassembled cap for NON-snapshot msgs
-define(CHUNK_BYTES,    65536).    %% 64 KiB payload per quod_link frame - << 1 MiB ceiling
%% max_snapshot_bytes is the cfg knob (default 64 MiB), read from D#d.cfg - NOT a macro.
```

Why `?CHUNK_BYTES == ?MAX_RAFT_BYTES`: if `?CHUNK_BYTES` were larger, a non-snapshot message between the two would pass the single-frame test, be sent as one `{raft, ...}`, then be silently dropped at the receiver by the `?MAX_RAFT_BYTES` cap. Keeping them equal means any non-snapshot message exceeding its reassembled cap is chunked, and the reassembled result is re-checked against `?MAX_RAFT_BYTES` (dropping, not mis-sending, anything genuinely oversized). Snapshots are the one record allowed past `?MAX_RAFT_BYTES`, bounded by `max_snapshot_bytes`.

Envelope (the unit handed to `quod_link:send/2`):

```
{raft, Ns, Bin}                %% single-frame: Bin = encode(Record)
{raft_chunk, Ns, MsgId, Seq, Total, Part}  %% one slice of a multi-frame message
```

```
frames(Ns, Record) ->
  Bin = encode(Record),
  case byte_size(Bin) <= ?CHUNK_BYTES of
  true  -> [encode({raft, Ns, Bin})];
  false ->
    Parts = chunkify(Bin, ?CHUNK_BYTES),
    Total = length(Parts),
    MsgId = make_ref(),
    [ encode({raft_chunk, Ns, MsgId, Seq, Total, P})
      || {Seq, P} <- lists:zip(lists:seq(1, Total), Parts) ]
  end.
```

Reassembly: `#d.rx` holds at most one `#rx{}` per peer; a new `MsgId` replaces any in-flight one (channel is in-order per peer-pair). Memory cap: if bytes would exceed `max_snapshot_bytes`, drop and bump `rx_dropped`. Non-snapshot messages re-check `?MAX_RAFT_BYTES` after reassembly. Reassembly keyed `{Peer, MsgId}`.

2.6 The three transport operations

(a) Open links. For each committee member `M` not in `conns`: `quod_quic:open_link(M, Chan)`. On `{link_up, M, Chan, LinkPid}` cache `conns#{M => {LinkPid, monitor(process, LinkPid), out}}` and **flush the per-peer outbox FIFO in order**. On `{link_error, M, Chan}` drop the queued frames (Raft re-sends fresh on the next heartbeat). On `{'DOWN', Ref, ...}` drop the conn. All in `common/3`, matching on `Chan`.

Outbox divergence from Brahms (load-bearing): Brahms's outbox is last-wins single-payload. Raft RPCs are not last-wins, so `quod_ledger`'s outbox is a **per-peer FIFO list** flushed in order on `link_up`.

(b) Send to ONE member (unicast) — the only natural primitive:

```
send_raft(M, Record, D = #d{ns = Ns, conns = Conns, outbox = Outbox}) ->
  Frames = frames(Ns, Record),
  case maps:get(M, Conns, undefined) of
    {LinkPid, _Ref, _Origin} ->
      [ _ = quod_link:send(LinkPid, F) || F <- Frames ],
      bump_sent(Frames, D);
    undefined ->
      _ = quod_quic:open_link(M, D#d.chan),
      Q = maps:get(M, Outbox, []),
      bump_sent(Frames, D#d{outbox = Outbox#{M => Q ++ Frames}})
  end.

bump_sent(Frames, D) ->
  Bytes = lists:sum([byte_size(F) || F <- Frames]),
  D#d{msgs_sent = D#d.msgs_sent + 1, chunks_sent = D#d.chunks_sent + length(Frames),
    bytes_sent = D#d.bytes_sent + Bytes}.
```

Broadcast (`RequestVote` / `heartbeat`) is a fold over `committee() \ {self}`, encoding per-peer (per-peer `prev_log_index`/entries differ — no shared-bytes fan-out):

```
broadcast(MakeMsg, Committee, D) ->
  lists:foldl(fun(M, A) -> send_raft(M, MakeMsg(M, A), A) end, D, Committee --
    ↪ [D#d.self]).
```

(c) Receive. Inbound shape `{quod_message, {Peer, LinkPid}, Chan, Payload}`; `Peer` is the sending `server_id()`. Handled per role-state guarded on own `Chan`, foreign channels dropped in `common/3`:

```

follower(info, {quod_message, {Peer, _LinkPid}, Chan, Payload}, D = #d{chan = Chan}) ->
    {keep_state, handle_inbound(Peer, Payload, D)};
common(info, {quod_message, _, _OtherChan, _}, D) -> {keep_state, D}.

handle_inbound(_Peer, Payload, D) when byte_size(Payload) > ?CHUNK_BYTES + 64 -> D;
handle_inbound(Peer, Payload, D0) ->
    D = bump_recv(Payload, D0),
    case decode(Payload) of
        {raft, Ns, Bin} when Ns == D#d.ns -> dispatch_record(Peer, Bin, D);
        {raft_chunk, Ns, MsgId, Seq, Total, Part} when Ns == D#d.ns ->
            reassemble(Peer, MsgId, Seq, Total, Part, D);
        _ -> D
    end.

bump_recv(_Payload, D) -> D#d{msgs_recv = D#d.msgs_recv + 1}.

dispatch_record(Peer, Bin, D) ->
    case decode(Bin) of
        #install_snapshot{} = M -> raft_event(Peer, M, D);    %% bypasses
        ↪ ?MAX_RAFT_BYTES
        #install_snapshot_reply{} = M -> raft_event(Peer, M, D);
        Other when byte_size(Bin) <= ?MAX_RAFT_BYTES ->
            case Other of
                #request_vote{} = M -> raft_event(Peer, M, D);
                #request_vote_reply{} = M -> raft_event(Peer, M, D);
                #append_entries{} = M -> raft_event(Peer, M, D);
                #append_entries_reply{} = M -> raft_event(Peer, M, D);
                _ -> D
            end;
        _ -> D#d{rx_dropped = D#d.rx_dropped + 1}
    end.

```

raft_event/3 is the only bridge into the §1 state-machine logic.

2.7 Separation from Brahms

Distinct channel ({channel, term_to_binary({log, Ns}, [deterministic])} vs {channel, Ns}), distinct reg name ({quod_ledger, Ns} vs {quod_brahms, Ns}), distinct envelope tags ({raft, ...}/{raft_chunk, ...} vs gossip tuples). One quod_link per channel per connection → the {log, Ns} and Brahms streams are independent on the same QUIC connection.

3. Persistence (quod_ledger_store)

quod_ledger_store is the only quod code that touches disk. A **plain library module** (no process, no reg, no supervisor child), called synchronously in-line from inside the quod_ledger gen_statem callbacks so an fsync provably completes before the triggering network reply leaves. It holds no state beyond an opaque handle threaded through #d.store.

Assumption (flagged): the data-dir convention (QUOD_DATA env + application:get_env(quod, data_dir, ...)) is wired through quod_app:apply_env/0 exactly like QUOD_PORT. Default data_dir and per-namespace base64url encoding flagged for review.

3.1 Directory layout

\$QUOD_DATA/	(default: filename:basedir(user_data,"quod") ++ "/data")
<nstoken>/	nstoken = base64url(Ns)
meta.term	cur_term + voted_for (atomic tmp+rename, fsync'd)
log.0001	append-only log segment (entries = the block list)
log.0001.idx	{Index, Term, ByteOffset, ByteLen} per entry
snapshot.<Idx>.<Term>	materialized snapshot (atomic via tmp+rename)
snapshot.<Idx>.<Term>.cfg	committee config as of that snapshot

The first cut writes one growing segment log.0001; the segment number is reserved so rollover needs no format change.

3.2 The store handle

include/quod_ledger_store.hrl -includes quod_ledger.hrl (so #entry{} is the shared definition — **not** redefined here) and defines only the private store record + opaque handle type:

```
%% include/quod_ledger_store.hrl
-include("quod_ledger.hrl").

-record(store, {dir           :: file:filename_all(),
               ns            :: binary(),
               log_fd        :: file:io_device(),      %% [read,write,raw,binary]
               idx_fd        :: file:io_device(),      %% [read,write,raw,binary]
               seg           :: pos_integer(),
               last_index    :: non_neg_integer(),
               last_term     :: non_neg_integer(),
               base_offset   :: non_neg_integer(),      %% next append offset
               snap_index    :: non_neg_integer(),
```

```

        snap_term    :: non_neg_integer()).
-opaque handle() :: #store{}.
-export_type([handle/0]).

```

The log/idx fds are opened [read, write, raw, binary] (**not** [append, ...]): in append mode every write is forced to EOF regardless of file:position/2, breaking conflict truncation and post-truncation re-append. The handle tracks base_offset as the next write position; each append seeks there, writes, then advances it.

3.3 On-disk encoding (per file)

```

frame = <<Magic:32, Len:32, CRC:32, Payload:Len/binary>>
Magic = 16#9151_06AA
CRC    = erlang:crc32(Payload)
Payload = term_to_binary(Term, [deterministic])

```

- **meta.term** — single frame of #{cur_term => Term, voted_for => ServerId | none}. **Always** written atomically: write meta.term.tmp, file:datasync/1 it, file:rename/2 over meta.term, then file:datasync/1 the **containing directory** (rename is durable only after a dir fsync on POSIX). There is **no** in-place truncate-and-rewrite path (it would expose a torn-write window that could let a restarted node double-vote in one term).
- **log.NNNN** — append-only at the Raft level (never rewritten in place). The only byte mutation is tail truncation (follower conflict / torn-tail recovery): seek to a recorded offset, file:truncate/1, file:datasync/1.
- **log.NNNN.idx** — fixed-width <<Index:64, Term:64, Offset:64, Len:32>> per entry. A **derived cache**: no fsync on the critical path; only the log frame must be durable before append/2 returns. Rebuilt by a forward frame-scan if inconsistent on boot. The log is the source of truth.
- **snapshot.<Idx>.<Term> + .cfg** — tmp+datasync+rename+dir-fsync.

3.4 Public API (handle-threaded)

All functions are synchronous and complete their required fsync before returning. A write that cannot fsync raises (crashing the quod_ledger statem → supervisor restart → clean cold boot) — the one place the “never crash on transient failure” convention is deliberately inverted.

```

-spec open(Ns :: binary(), DataDir :: file:filename_all()) -> {ok, handle()}.
-spec close(handle()) -> ok.

-spec read_meta(handle()) -> {Term :: non_neg_integer(), VotedFor :: server_id() | none}.
%% {0, none} if meta.term absent (fresh namespace).

```

```

-spec write_meta(handle(), Term :: non_neg_integer(), VotedFor :: server_id() | none) ->
  ↳ ok.
%%   Atomic tmp+datasync+rename+dir-fsync. Called BEFORE a vote-granted / term-bump reply
  ↳ (§E.10.1/10.4).

-spec append(handle(), [#entry{}]) -> {ok, handle()}.
%%   Contiguous indices == last_index+1.. ; seek to base_offset, write frames+idx, single
  ↳ LOG datasync
%%   per batch, advance last_index/last_term/base_offset. Returns before quod_ledger
  ↳ counts own log (§10.2/10.3).
-spec truncate_from(handle(), Index :: pos_integer()) -> {ok, handle()}.
%%   Delete entry Index and after; file:truncate log+idx, datasync, recompute. Never <=
  ↳ snap_index.

-spec read_at(handle(), Index :: pos_integer()) -> {ok, #entry{}} | not_found.
-spec read_range(handle(), From :: pos_integer(), To :: pos_integer()) -> {ok,
  ↳ [#entry{}}}.
-spec last(handle()) -> {Index :: non_neg_integer(), Term :: non_neg_integer()}.
-spec term_at(handle(), Index :: non_neg_integer()) -> non_neg_integer() | undefined.

-spec write_snapshot(handle(), LastIdx, LastTerm, Config :: [server_id()], Data :: term())
  ↳ -> {ok, handle()}.
%%   Requires LastIdx <= commit_index. tmp+datasync+rename+dir-fsync. Compaction is a
  ↳ SEPARATE step –
%%   write_snapshot never deletes log entries, so a crash mid-compaction still has both
  ↳ copies.
-spec read_snapshot(handle()) -> none | {ok, LastIdx, LastTerm, Config :: [server_id()],
  ↳ Data :: term()}.
-spec install_snapshot(handle(), LastIdx, LastTerm, Config, Data) -> {ok, handle()}.
%%   write_snapshot (durable) FIRST, then replace the live log; caller jumps
  ↳ commit_index/last_applied.

```

load/1 is a convenience over read_meta/read_snapshot/scan returning #{cur_term, voted_for, log, snap_idx, snap_term, snap_cfg, snap_data} for quod_ledger:init/1.

3.5 Cold-boot reload sequence

Called once from quod_ledger:init({Ns, Config}):

1. DataDir = maps:get(data_dir, Cfg); {ok, Store0} = quod_ledger_store:open(Ns, DataDir). Inside open/2: a. filelib:ensure_path/1 the namespace dir. b. Open log.NNNN/.idx [read, write, raw, binary]; scan the idx; base_offset = log file size. c. **Torn-tail recovery:** if the final frame's Len+CRC don't fully fit/check, seek to the last good boundary, file:truncate/1 log+idx, file:datasync/1, set base_offset. Safe: an un-fsync'd append was never acked / counted toward commit. d. Rebuild idx by frame-scan if missing/short/inconsistent. e. read_snapshot/1 → set snap_index/snap_term; set

- last_index/last_term from the last idx record or from snap_* if the log is empty.
2. {Term, VotedFor} = read_meta(Store0) → seed cur_term, voted_for ({0, none} if absent).
 3. If a snapshot exists, seed the materialized KB (handed to quod_prolog) from snap_data, last_applied := commit_index := snap_index; else both := 0.
 4. Recompute volatile state: committee derived (snap_cfg overlaid with in-log kind=config entries in index order, *appended* not just committed); next_index/match_index empty until an election win.
 5. Enter follower with a randomized election timeout. A 1-member committee self-elects on the first timeout.
 6. Brand-new namespace: no files → empty store, read_meta gives {0, none}, quod_ledger appends the genesis {add, self} config (create mode).

3.6 Browsability (full-history read path)

```
-spec history(Ns :: binary(), From :: pos_integer(), Count :: pos_integer())
    -> {ok, [#entry{}], NextFrom :: pos_integer() | done} | {error, term()}.
history(Ns, From, Count) ->
    try gen_statem:call(quod_reg:via({quod_ledger, Ns}), {history, From, Count}, 1000)
    catch exit:_ -> {error, unavailable} end.
```

Served by read_range(Store, From, min(From+Count-1, LastIndex)), Count clamped to ?HISTORY_PAGE_MAX. Entries already compacted into a snapshot (Index =< snap_index) return {error, compacted, snap_index}. History is committee-only in the first cut; read-copies hold current facts only.

3.7 Edge cases

- **Torn tail write** — CRC/Len check at open/2, truncate to last intact frame.
- **meta.term torn rewrite** — impossible (tmp+rename only).
- **idx out of sync** — rebuilt by frame-scan; log is source of truth.
- **Truncate below snapshot** — truncate_from/2 refuses Index =< snap_index (crashing assertion).
- **Append with gap/backward index** — append/2 asserts contiguity; violation crashes the statem.
- **Crash mid-compaction** — both snapshot and full log on disk; boot restores from the snapshot + suffix.
- **fsync failure / disk full** — propagated as a crash; restart → clean cold boot.
- **quod_app env wiring** — add data_dir to apply_env/0 (default file-name:basedir(user_data, "quod") ++ "/data", overridable by QUOD_DATA).

4. quod_prolog apply + prove

quod_prolog is the per-namespace fact engine: it owns the committed erlog KB for one ontology, applies committed blocks from quod_ledger in log order, and serves proofs. It **serialises writes**; read-only proofs run on copy-on-write overlays alongside, never blocking the writer (#22, #26). One per ontology.

4.1 Module, records

```
-module(quod_prolog).
-behaviour(gen_server).  %% one writer-serialiser per Ns; no protocol states (those live
    ⇨ in quod_ledger)
-include("quod_ledger.hrl").
```

erlog stack reused from bbsvx: quod_erlog_db_differ → quod_erlog_db_local_prove → quod_erlog_db_ets (ported from the bbsvx modules, dropping the db_federated and set_acl layers — deferred with foreign writes, #10).

```
-record(kb, {est          :: erlog_state(),          %% #est{} over differ->ets, committed
    ⇨ facts
              applied = 0 :: non_neg_integer()}).  %% highest log index folded in

-record(s, {ns           :: binary(),
    self           :: server_id(),
    cfg            :: map(),
    kb             :: #kb{},
    prove_pending  = #{} :: #{reference() => prove_ctx()}, %% in-flight prove
    ⇨ workers, by Ref
    parked        = #{} :: #{binary() => write_ctx()},      %% writes awaiting
    ⇨ commit, by tx_id
    applies       = 0 :: non_neg_integer(),
    rejects       = 0 :: non_neg_integer(),
    proves        = 0 :: non_neg_integer(),
    conflicts      = 0 :: non_neg_integer()}).

-type bindings() :: [map()] | fail.
-type prove_ctx() :: {From :: gen_server:from(), CallerNs :: binary(),
    HeightRead :: non_neg_integer(), MonRef :: reference(), TimerRef ::
    ⇨ reference()}.
-type write_ctx() :: {From :: gen_server:from(), Effects :: [term()]}
```

#transaction{} and the op()/read_check()/clause() types come from the shared header. read_check values are phash2 integers per {Functor, Arity} (per-predicate

granularity — same-functor different-fact changes cause accepted false conflicts; per-fact granularity deferred).

4.2 Start / register / supervise

```
start_link(Ns, Config) ->
  gen_server:start_link(quod_reg:via({quod_prolog, Ns}), ?MODULE, {Ns, Config}, []).

namespaces() -> gproc:select([{{n, l, {quod_prolog, '$1'}}, '_', '_'}, [], ['$1']}).

-define(DEFAULTS, #{node_id    => undefined,
                    prove_ms   => 30000,    %% read-proof worker watchdog (server-side)
                    call_ms    => 35000,    %% client call timeout in prove/3; MUST
                                     ↪ exceed prove_ms
                    max_diff_bytes => 1048576}).
```

Child of quod_ns (rest_for_one, after quod_ledger). `init({Ns, Config})` merges `?DEFAULTS`, validates, and rebuilds the KB (§4.6) before answering proves. `quod_prolog` does not subscribe to any channel; it receives committed blocks via direct call from `quod_ledger`.

4.3 Serving prove — copy-on-write overlays, no writer blocking

```
-spec prove(TargetNs :: binary(), Goal :: term(), CallerNs :: binary())
  -> {ok, Bindings :: [map()], ReadHeight :: non_neg_integer()}
    | {error, no_such_namespace} | fail.
prove(TargetNs, Goal, CallerNs) ->
  case quod_reg:where({quod_prolog, TargetNs}) of
    undefined -> {error, no_such_namespace};
    Pid -> try gen_server:call(Pid, {prove, Goal, CallerNs}, maps:get(call_ms,
      ↪ ?DEFAULTS))
          catch exit:_ -> fail end
  end.
```

`fail` is goal-failure; `{error, no_such_namespace}` is routing failure — distinct. The writer never blocks: `handle_call({prove, ...})` parks `From` and returns `{noreply, S}`; the reply is sent later (from `handle_cast({prove_result, ...})` for a read, or after commit for a write).

`handle_call({prove, Goal, CallerNs}, From, S):`

1. **Wrap the committed KB in a per-proof overlay:**

```
#kb{est = Committed, applied = HeightRead} = S#s.kb,
Wrapped = quod_erlog_db_local_prove:wrap_state(Committed, #{read_set => true}),
```

local_prove shadows asserts/retracts per functor in its own overlay; committed facts untouched.

2. **Record HeightRead = #kb.applied** into the parked prove_ctx() (#7 — record height read).
3. **Spawn + monitor the worker, arm a prove_ms watchdog**, stash {From, CallerNs, HeightRead, MonRef, TimerRef} in prove_pending under the worker Ref, return {noreply, S1}:

```
try erlog_int:prove_goal(Goal, Wrapped) of
  {succeed, #est{bs = Bs} = Final} ->
    Bindings = extract_bindings(Goal, Bs),
    Db       = Final#est.db,
    Changes  = quod_erlog_db_local_prove:get_local_changes(Db),
    ReadSet  = quod_erlog_db_local_prove:get_read_set(Db),
    Effects  = quod_erlog_db_local_prove:get_effects(Db),
    gen_server:cast(Self, {prove_result, Ref, {ok, Bindings, Changes, ReadSet,
      ↪ Effects}});
  {fail, _} -> gen_server:cast(Self, {prove_result, Ref, fail})
catch Class:Err -> gen_server:cast(Self, {prove_result, Ref, {error, {Class, Err}}})
after quod_erlog_db_local_prove:cleanup_read_set(Wrapped) end
```

4. **Classify after execution (#8)** in handle_cast({prove_result, Ref, Result}, S) — remove the prove_ctx(), cancel timer, demonitor:
 - fail / {error, _} → reply fail (resp. {error, prove_failed}).
 - {ok, Bindings, [], _, _} → **read** (empty write-set, #6). Reply {ok, Bindings, HeightRead}. Nothing asserted, nothing committed. Bump proves.
 - {ok, Bindings, Changes, ReadSet, Effects} with Changes /= [] → touched facts. **MVP: legal only if CallerNs == TargetNs == S#s.ns** (#10). If so, hand off to the writer path (§4.5), keep From parked. Else reject {error, foreign_write_unsupported}.

Concurrency: multiple workers run on independent overlays; apply (§4.4) mutates the committed cell on the gen_server mailbox between proofs, but a wrapped worker sees its own copy-on-write view. The single serialisation point is the mailbox; apply is the only mutator of the committed cell.

4.4 apply_block — deterministic OCC apply (#23, #25)

quod_ledger, once a block is committed, calls quod_prolog **in log-index order**:

```
-spec apply_block(Ns :: binary(), Index :: pos_integer(), Change :: #transaction{} | noop)
-> ok | {reject, conflict}.
apply_block(Ns, Index, Change) ->
  gen_server:call(quod_reg:via({quod_prolog, Ns}), {apply_block, Index, Change},
    ↪ infinity).
```

apply_block/3 is a **synchronous call**. To avoid the quod_prolog↔quod_ledger deadlock, quod_ledger:append from quod_prolog returns at **append/replication-accept** (not commit); the final OCC/commit verdict reaches the original prove caller via quod_prolog's own apply_block handler correlating by tx_id. So quod_prolog is never blocked in gen_server:call(quod_ledger, append) while needing to service apply_block.

handle_call({apply_block, Index, Change}, _From, S) runs the pure engine step, replies the verdict synchronously, updates kb, releases any parked write for the tx_id, bumps metrics:

```
%% apply(Index, Change, KB) -> {ok, KB'} | {reject, conflict, KB'}
apply(Index, noop, #kb{applied = A} = KB) ->
    Index := A + 1 orelse error({apply_out_of_order, Index, A}),
    {ok, KB#kb{applied = Index}};
apply(Index, #transaction{read_check = ReadCheck, diff = Diff}, #kb{est = Est, applied =
    ↪ A} = KB) ->
    Index := A + 1 orelse error({apply_out_of_order, Index, A}),    %% log order is the
    ↪ contract (#13)
    case validate_read_set(ReadCheck, Est) of
        ok ->
            {ok, Est1} = quod_diff:apply_ops(Est, Diff),            %% deterministic (#25)
            {ok, KB#kb{est = Est1, applied = Index}};
        {conflict, _Functor} ->
            {reject, conflict, KB#kb{applied = Index}}              %% block committed;
            ↪ effect is no-op
    end.
```

- **OCC re-check** (validate_read_set/2, ported): for each {Functor, Arity} => ExpectedHash, recompute quod_diff:functor_hash/3 against the committed KB at index A, compare. Any mismatch → {conflict, _}.
- **On ok**: quod_diff:apply_ops/2 replays the concrete ops (asserts dedup by content, retracts by content, #1). Deterministic; does not re-run Prolog. Bump applied := Index, applies++.
- **On {reject, conflict}**: do not apply the diff but still advance applied := Index (the block is consumed; effect is a no-op due to stale read). rejects++. The submitting node turns the reject into a retry (§4.5); other members no-op. Because every member runs apply over the same committed prefix, the verdict is identical everywhere — accept/reject is itself part of the replicated state machine.

4.5 Building a #transaction{} and submitting it (effects-after-commit)

A would-be writer (own-ns prove whose worker produced Changes /= []) becomes a transaction. From stays parked; nothing is replied until the block applies on this

node.

1. Proof already ran on the staging overlay, capturing Changes, ReadSet, Effects.
2. Build a `#transaction{}`:

```
Change = #transaction{tx_id      = ulid:generate(),
                      caller_ns  = CallerNs,
                      diff       = normalize_ops(Changes),  %% local_prove ops -> [op()]
                      ↪ by content
                      read_check = ReadSet}
```

`normalize_ops/1` strips clause Tags so the diff is content-only/node-portable (#1, #2). Guard `byte_size(term_to_binary(Change)) =< max_diff_bytes`; over-size → {error, change_too_large}, drop.

3. **Park {From, Effects} in parked under tx_id, then submit:**

```
case quod_ledger:append(S#s.ns, Change) of
  {ok, _BlockIndex}      -> ok;  %% accepted into the log (replication-accept)
  {error, not_in_charge, Hint} -> {error, {not_leader, Hint}}; %% reply now, unpark
  {error, conflict_retry}   -> {retry, conflict};             %% leader fast-fail
  ↪ (optional)
  {error, busy}            -> {error, busy}
end
```

`quod_prolog` does not apply its own write directly — it waits for the block via `apply_block/3` in committed order, applying via the same deterministic path as every member.

4. **Release on apply (effects-after-commit, #11).** When `apply_block` processes the block carrying this `tx_id`, look up parked:
 - applied → reply {ok, Bindings, HeightRead} to the parked From **and** fire the deferred Effects.
 - rejected → reply {retry, conflict}; discard Effects. Remove the `tx_id` entry either way. On other members the block has no parked entry, so `apply_block` just mutates the KB. `quod_ledger` tags each applied block with its `#transaction.tx_id` for the correlation.

4.6 Rebuild on start — replay the log / snapshot (#20, #29, #13)

`init/1` builds the committed KB **from the log**, never from a separately-persisted fact store, via the `quod_ledger:replay/1` handshake (the safe direction: the log is authoritative; prolog rebuilds from it, never vice-versa):

```
build_kb(Ns) ->
  {ok, Base} = erlog_int:new(quod_erlog_db_differ, {fresh_ets(Ns), quod_erlog_db_ets}),
  Est0 = load_builtins(Base),
  case quod_ledger:snapshot(Ns) of
    {ok, SnapIndex, SnapData} -> {SnapIndex, seed_from_snapshot(SnapData, Est0)};
```

```

    none                                -> {0, Est0}
end.

```

Then, on init, `quod_prolog` synchronously calls `quod_ledger:replay(Ns)`, which resets `quod_ledger's last_applied := snap_idx` (and hands snapshot data if any) and **re-drives `apply_block/3` over `snap_idx+1 .. commit_index`** into the fresh prolog, before any prove is answered. This is what makes a **lone `quod_prolog` crash** recover: without the reset, the surviving log (already at `last_applied = commit_index`) would push only future blocks and the fresh KB would stay empty.

Replay uses the **same `apply/3`** as the live path, so the OCC accept/reject sequence and the final applied counter are reproduced bit-for-bit — the rebuilt `#kb{}` (facts **and** applied) is identical to a continuously-running member's. `#kb.applied` is seeded to `snap_idx` so the `Index := applied+1` assertion holds from the first replayed block. No deferred effects fire on replay (committed-already).

4.7 Sync query API + metrics

```

stats(Ns) -> try gen_server:call(quod_reg:via({quod_prolog, Ns}), get_stats, 1000) catch
  _:_ -> exit:_ -> #{} end.

handle_call(get_stats, _From, S) ->
  {reply, #{applies => S#s.applies, rejects => S#s.rejects, proves => S#s.proves,
    conflicts => S#s.conflicts, applied => (S#s.kb)#kb.applied}, S}.

```

`quod_metrics:declare/0` gains `quod_prolog_applied_index`, `quod_prolog_apply_rejects_total`, `quod_prolog_proves_total`, labelled `[namespace]`, set in a `refresh_ns`-style loop driven off `quod_prolog:namespaces/0`.

4.8 Edge cases

Case	Handling
prove on unknown <code>TargetNs</code>	<code>quod_reg:where</code> → undefined → {e
Goal fails	worker reports fail; reply fail; o
Worker crashes / times out	DOWN or <code>prove_ms</code> watchdog → look
Foreign-ns write (<code>CallerNs</code> $\neq$ <code>TargetNs</code> , <code>Changes</code> $\neq$ <code>[]</code>)	{error, <code>foreign_write_unsupport</code>
<code>apply_block</code> out of order	<code>error({apply_out_of_order, ...</code>
OCC conflict at apply	{reject, conflict}: facts unchar

Case	Handling
Over-size #transaction{}	reject before quod_ledger:append
Not the leader on submit	{error, not_in_charge, Hint} →
1-voter committee	transparent: quod_ledger commit
Restart mid-flight write	a parked write whose block had n

5. quod_ns_sup lifecycle + membership

5.1 quod_ns_sup — subtree root (src/quod_ns_sup.erl)

Copy of quod_brahms_sup except the key and child module. simple_one_for_one; namespaces are created via start_namespace/2; the child is a per-Ns quod_ns **sub-supervisor** (not a worker), because a namespace owns two fate-shared workers.

```
-module(quod_ns_sup).
-behaviour(supervisor).
-export([start_link/0, start_namespace/2, stop_namespace/1, namespaces/0]).
-export([init/1]).

start_link() -> supervisor:start_link(quod_reg:via({quod_ns_sup, node}), ?MODULE, []).

-spec start_namespace(binary(), map()) -> supervisor:startchild_ret().
start_namespace(Ns, Config) ->
    supervisor:start_child(quod_reg:via({quod_ns_sup, node}), [Ns, Config]).

-spec stop_namespace(binary()) -> ok | {error, not_found}.
stop_namespace(Ns) ->
    case quod_reg:where({quod_ns, Ns}) of
        undefined -> {error, not_found};
        Pid -> supervisor:terminate_child(quod_reg:via({quod_ns_sup, node}), Pid)
    end.

namespaces() -> gproc:select([{{n, l, {quod_ns, '$1'}}, '_', '_'}, [], ['$1'])).

init([]) ->
    Flags = #{strategy => simple_one_for_one, intensity => 10, period => 10},
    Child = #{id => quod_ns, start => {quod_ns, start_link, []},
              restart => transient, type => supervisor},
    {ok, {Flags, [Child]}}.
```

start_namespace/2's [Ns, Config] are appended to the empty child MFA. namespaces/0 keys on {quod_ns, '\$1'}. stop_namespace/1 addresses the simple_one_for_one child by pid via quod_reg:where/1.

5.2 quod_ns — per-namespace sub-supervisor (src/quod_ns.erl)

The unit of **fate-sharing**. Registers {quod_ns, Ns}, supervises quod_ledger then quod_prolog with **rest_for_one**.

Authoritative direction: the durable log rebuilds the kb; the kb never rebuilds the log. If quod_ledger crashes, rest_for_one restarts it **and then** quod_prolog, which rebuilds from the disk-reloaded log; if quod_prolog crashes alone, only it restarts and rebuilds from the log's committed prefix via the §4.6 replay handshake.

```
-module(quod_ns).
-behaviour(supervisor).
-export([start_link/2]).
-export([init/1]).

start_link(Ns, Config) -> supervisor:start_link(quod_reg:via({quod_ns, Ns}), ?MODULE,
  ↳ {Ns, Config}).

init({Ns, Config}) ->
  Flags = #{strategy => rest_for_one, intensity => 10, period => 10},
  Children =
    [{#id => quod_ledger, start => {quod_ledger, start_link, [Ns, Config]},
      restart => permanent, type => worker},
     #{id => quod_prolog, start => {quod_prolog, start_link, [Ns, Config]},
      restart => permanent, type => worker}],
  {ok, {Flags, Children}}.
```

Children are permanent within the sub-sup; the sub-sup is transient under quod_ns_sup (a clean stop_namespace is not auto-restarted; a crash is).

5.3 quod_ledger $\rightleftharpoons$ quod_prolog apply coupling

The coupling is the Raft apply loop (§1.10): for each committed kind=block, quod_ledger calls quod_prolog:apply_block(Ns, Index, Change) -> ok | {reject, conflict} **strictly in index order**. It is a synchronous call; the OCC read-set re-check runs at apply on every member (§4.4) — the verdict is deterministic and identical everywhere, so this reconciles “apply re-checks” (#23) with determinism (#25). kind=config entries carry **no read_check** and are not read-validated.

Rebuild handshake: on quod_prolog:init/1 (empty kb), it calls quod_ledger:replay(Ns) -> {ok, AppliedUpTo} which resets last_applied := snap_idx (seeding from snapshot data if present) and re-drives apply_block/3 over snap_idx+1 .. commit_index before any prove is answered (§4.6). The same handshake covers the quod_ledger-crash path (prolog restarts after it).

5.4 Routing prove(TargetNs, Goal, CallerNs)

Reads do not go through the committee (#22). Routing is a gproc name lookup on {quod_prolog, TargetNs} (see §4.3 prove/3). TargetNs == CallerNs is own-namespace; TargetNs /= CallerNs is a cross-namespace read (MVP allows exactly these two, #10). Foreign writes are rejected at routing/classification time. The fact store is CP (#16): a node that cannot confirm commit_index is current cannot serve a guaranteed-latest read or any write.

5.5 Committee write path: append / apply

quod_ledger:append(Ns, #transaction{}) -> {ok, BlockIndex} | {error, not_in_charge, Hint} | {error, conflict_retry} | {error, busy} (§1.11). A non-leader replies {error, not_in_charge, Hint} (last-known leader). On the leader: append a kind=block entry, replicate, commit once a majority holds it, then the apply loop pushes it into every member's quod_prolog (which runs the OCC re-check at apply, surfacing {error, conflict_retry} to the client if the read-set changed). All committee RPCs ride the {log, Ns} channel (§2).

5.6 Namespace creation (genesis, 1-voter) vs join

Both enter through quod_ns_sup:start_namespace(Ns, Config); Config.mode ∈ {create, join} selects the path. The distinction is safety-critical: create bootstraps a 1-voter committee and may self-elect; join must **never** bootstrap a committee, **never** arm an election timer, and **never** self-elect until a real committee's leader has added it.

Create (genesis, 1-voter, #18): quod_ledger finds no durable state and mode=create → bootstraps a single committed kind=config {add, self} (1-voter committee), fsync'd. On election timeout it self-elects (quorum 1), becomes leader, is immediately writable — same code paths. Growth to 3/5 is the same code, one config change at a time (#19, §1.11).

Join: quod_ledger finds no durable state and mode=join → passive learner (empty log, no genesis config, election timer **not** armed). Then:

1. **Discover via Brahms** (discovery only, #28): quod_brahms:view(Ns) / quod_brahms:sample(Ns) (both exported, degrade to [] on exit:_). Brahms returns candidates, filtered against the in-log committee.
2. **Request join:** open {log, Ns} links to candidates, unicast {join_request, self, JoinArgs}; a non-leader replies {not_in_charge, LeaderHint}.

3. **Leader evaluates the join-predicate** against its own committed facts at its `commit_index` (`quod_prolog:prove(Ns, JoinPredicateGoal, Ns)`). Not proved $\rightarrow$ `{join_denied, Reason}`.
4. **Append a single {add, JoinerId} config entry**, refused if `cfg_uncommitted` (one-at-a-time, §1.11). Membership is adopted on append by every member that has the entry.
5. **Catch up + activate:** the leader brings the joiner's empty log up via `AppendEntries` back-up (or `InstallSnapshot`, M3). When the joiner receives the `{add, self}` config entry it adopts membership (now a voter) and **arms its election timer**.

5.7 Leaving / removal (#19, §1.11)

Voluntary leave submits a `kind=config {remove, ServerId}` to the leader (same one-at-a-time rule). A leader removing itself stays leader until the removal commits (under the new config excluding it), then steps down and its `quod_ns` subtree is torn down via `stop_namespace/1` (clean terminate, not auto-restarted). A removed server stops/disarms its election timer once it observes its own committed removal. A crash (not a removal) is fine (#29): the committee continues at reduced size; the crashed node's `quod_ns` restarts (transient $\rightarrow$ on crash), `quod_ledger` reloads and catches up; membership is unchanged.

5.8 quod_reg keys

Add four rows to the `quod_reg` moduledoc key table (no code change):

Key	identifies
<code>{quod_ns_sup, node}</code>	per-namespace content subtree root supervisor (singleton)
<code>{quod_ns, Ns}</code>	a namespace's content sub-supervisor
<code>{quod_ledger, Ns}</code>	a namespace's Raft committee member / log statem
<code>{quod_prolog, Ns}</code>	a namespace's Prolog fact store + prove engine

Channel property: `{channel, term_to_binary({log, Ns}, [deterministic])}` — distinct from Brahms's `{channel, Ns}`.

5.9 Integration with quod_brahms and quod_app boot

quod_brahms — discovery only (#28). Never started/stopped/restarted or read on the create path; only quod_ledger's join path reads quod_brahms:view/1 / sample/1, then filters against the in-log committee.

quod_sup wiring: replace the %% TODO line (quod_sup.erl:31) with:

```
{%id => quod_ns_sup, start => {quod_ns_sup, start_link, []}, type => supervisor}
```

placed after quod_brahms_sup under the unchanged one_for_one flags.

quod_app boot: a maybe_start_ns/0 after maybe_join/0, mirroring its env reads (including node_id / default_node_id()); add QUOD_NS_MODE (create|join, default create):

```
maybe_start_ns() ->
  case os:getenv("QUOD_NAMESPACE") of
    false -> ok; "" -> ok;
    NsStr ->
      Ns    = list_to_binary(NsStr),
      Self  = application:get_env(quod, node_id, default_node_id()),
      Mode  = case os:getenv("QUOD_NS_MODE") of "join" -> join; _ -> create end,
      Cfg   = #{node_id => Self, mode => Mode,
                seed_peers => parse_seeds(os:getenv("QUOD_SEEDS")), join_ns => Ns},
      case quod_ns_sup:start_namespace(Ns, Cfg) of
        {ok, _} -> logger:info("quod[~s]: content namespace up (~p)", [Ns, Mode]);
        Error   -> logger:error("quod[~s]: content namespace failed: ~p", [Ns,
                                   ↳ Error])
      end, ok
  end.
```

Called from start/2 after ok = maybe_join() so Brahms is up first for the join path.

5.10 House-style conformance

Supervisors copy quod_brahms_sup; workers register via quod_reg:via/1, init/1 takes {Ns, Config}, merges ?DEFAULTS, valid_cfg/2, {stop, {bad_config, Reason}}; quod[~s] logging; catch exit:_ / catch _:_ -> ok degradation; encode/decode with [safe] + size cap; OTP 27+ -moduledoc/-doc Markdown; metrics via get_stats-style maps + quod_metrics gauges driven off the namespaces/0 enumerators.

6. Build order + test plan

The Raft mechanics follow §1; integration follows the quod_brahms house style and §5 supervision. Each milestone is demonstrable on its own and depends only on earlier ones.

6.1 Modules added

module	role	reg key
quod_ledger	per-Ns Raft committee member; owns the durable log	{quod_ledger, Ns}
quod_prolog	per-Ns facts engine: apply/prove	{quod_prolog, Ns}
quod_ledger_store	on-disk persistence helper (library)	—
quod_diff	pure differ: write-set op-log + read-set hashes	—
quod_ns	per-Ns rest_for_one sub-sup	{quod_ns, Ns}
quod_ns_sup	simple_one_for_one parent of quod_ns	{quod_ns_sup, no

Shared types/records live in include/quod_ledger.hrl (§ shared header) and include/quod_ledger_store.hrl (§3.2). #d{} (§1.3) is the Raft gen_statem state.

Channel-match hazard (mandatory). Brahms matches inbound {quod_message, _, Ns, _} / {link_up, _, Ns, _} on Ns. quod_ledger uses Chan = term_to_binary({log, Ns}, [deterministic]), so **every** quod_ledger receive/link clause **MUST** bind/guard on Chan, not Ns. A verbatim copy from brahms that matches on Ns would silently drop every Raft message.

Assumption (flagged): the committee channel is reliable, in-order, point-to-point per peer-pair but not delivery-guaranteed (§E preamble). quod_link provides exactly this; a missing reply is "no reply", retried on the next heartbeat.

6.2 M1 — persistence + single-voter log + apply + prove (one node)

Goal: a fully working 1-voter namespace, exercising every Raft code path a single voter runs (#18).

Build:

1. **quod_ledger_store** (§3) — open/2, load/1, write_meta/3, append/2, truncate_from/2, write_snapshot/5, plus the readers. Every mutating call fsyncs before returning; one fsync per append/2 batch. (Single-chunk snapshots; chunking not built.)

2. **quod_diff** (pure, -ifdef(TEST) exported): functor_hash(Functor, Arity, Clauses) -> integer(), read_check(KB, ReferencedFunctors) -> read_check(), diff_to_ops(BeforeKB, AfterKB) -> [op()], apply_ops(KB, [op()]) -> KB'. Content-only identity; deterministic apply.
3. **quod_prolog** (§4): prove/3, apply_block/3 (ok | {reject, conflict}) wrapping the pure engine apply/3. prove/3 runs against the local converged KB, returns bindings (not auto-asserted, #6), classifies after execution (#8); empty diff = read (no append); non-empty = transaction. Effects deferred until after commit (#11).
4. **quod_ledger_gen_statem** (§1): init/1 (load durable, reconstruct volatile, subscribe on Chan, arm election timer); brand-new-Ns create bootstrap ({add, self}); append/2 ({error, conflict_retry} = OCC reject, {error, busy} = M5 backpressure); append path (leader, 1-voter: fsync, park From, commit on local fsync); apply loop (reply parked From from the apply step per OCC verdict; fail pending on tail loss / step-down); self-election as a degenerate normal election (#18).

Demonstrable: one node, start_namespace(Ns, #{node_id => Self, mode => create}); append = {ok, 1}; prove returns the asserted bindings; restart = log replays from disk, same facts (#20, #29); a read appends nothing (#6, #10); a second append whose read_check was invalidated returns {error, conflict_retry} and changes no facts (#14).

6.3 M2 — election + replication on 3 nodes

Goal: a real N=3 committee over loopback QUIC. No change to M1's commit/apply/persist paths (#18).

Build (extends quod_ledger only): link management to peers (§2.6 — match on Chan); send_rpc unicast + encode-once broadcast; receiving RPCs with the size-guard-before-decode and [safe] decode; RequestVote (§1.6); AppendEntries (§1.7, fsync appended entries before success, fail pending above a truncation); leader replication + commit with the mandatory current-term guard (§1.10); election & step-down mechanics (randomized [T, 2T], persisted self-vote, fail pending on step-down).

Demonstrable: 3 loopback nodes elect a leader; append on the leader commits once 2 of 3 have it (#17); append on a follower returns {error, not_in_charge, LeaderHint}; kill the leader => new leader in ~T, accepts appends; the three KBs are identical (#13, #25).

6.4 M3 — catch-up + snapshot (#29)

Plain catch-up via the AppendEntries back-up loop (no snapshot); compaction after snapshot_every committed entries (write_snapshot/5 fsync **before** truncating the live log, only the committed prefix); InstallSnapshot receiver + leader side (next_index[Peer] <= snap_idx → send #install_snapshot{}). The #install_snapshot{} records are already defined (shared header); InstallSnapshot and compaction land here, not earlier — so §1 and §3 do not contradict on whether compaction happens.

Demonstrable: append many blocks so the leader compacts, then start a fresh 4th node or restart a member whose log was compacted away ⇒ InstallSnapshot then AppendEntries, KB converges.

6.5 M4 — single-server membership change (#19)

add_member/2 / remove_member/2 (leader-only, one in flight via cfg_uncommitted); adopt-on-append; leader-removes-itself; new-member catch-up reuses M3. Brahms only finds candidates; the in-log committee is authoritative (#28).

Demonstrable: 3→5 via two sequential add_members (the second blocked until the first commits), serving appends throughout, then 5→3 including a leader removing itself with clean failover; surviving KBs identical.

6.6 M5 — metrics + backpressure

quod_ledger:stats/1 (§1.14, one snake_case key set); quod_metrics gauges (partial-match consumer); backpressure (max_pending → {error, busy}; max_batch kept under ?CHUNK_BYTES/1 MiB). CP availability: under partition the quorum-less side cannot append, surfaced as {error, not_in_charge, _}, not a crash.

Demonstrable: Prometheus per-Ns leader/term/commit gauges track failover live; flooding a leader returns {error, busy} past the cap.

6.7 Test plan

eunit (pure): quod_diff_tests (op ordering #3; content-only order-independent functor_hash #1/#4; apply_ops determinism #25). quod_prolog apply re-check (matching read_check = {ok, _}; changed functor hash = {reject, conflict} #14/#23; determinism #25). quod_ledger pure helpers: quorum/1 (1/3/5 → 1/2/3); term_at/2 incl. snapshot sentinel and index-0; the election-restriction truth-table

(§1.6); the **commit-rule current-term-guard** (refuses to commit a prior-term entry even at full replication; commits it only once a current-term entry above it commits — Figure 8); the AppendEntries truncate-vs-skip decision incl. the **idempotent same-term-skip that must NOT truncate on a delayed/duplicate RPC**. quod_ledger_store round-trip + **durability ordering** (each mutating call returns only after fsync).

Common Test over loopback QUIC:

1. raft_single_voter_SUITE (M1): self-elect, append $\Rightarrow$ {ok, 1}, prove returns bindings (not auto-asserted #6); restart $\Rightarrow$ replays, same KB; a read appends nothing; a stale-read_check append returns {error, conflict_retry} and leaves facts unchanged.
2. raft_safety_SUITE (M2) — core safety: no two leaders per term; **double-vote prevention across crash** (persist (cur_term, voted_for) before the grant reply); leader failover within bounded timeouts + pending failed {error, not_in_charge, _}; log convergence (byte-identical up to commit_index, equal KBs); election restriction; **prior-term commit (Figure 8 scenario)**.
3. raft_replay_SUITE: feed a captured committed log through quod_prolog's apply path on a fresh engine $\Rightarrow$ identical KB incl. identical accept/reject verdicts on OCC no-op blocks (#13, #25).
4. raft_catchup_SUITE (M3): fresh node back-up; compacted-member Install-Snapshot; restart-from-own-history delta catch-up.
5. raft_membership_SUITE (M4): 3 $\rightarrow$ 5 with the second add_member blocked until the first commits; 5 $\rightarrow$ 3; leader-removes-itself; no two-leader window.
6. occ_conflict_SUITE: A proves a #transaction{} with a non-empty read_check; before A's append commits, B mutates a functor in A's read_check; A's apply_block re-checks at its write's log position, rejects, A's caller gets {error, conflict_retry} and retries — committed KB reflects B then A's retry in log order. Negative: a disjoint read_check commits without abort. Per-predicate granularity: a same-functor different-fact change still aborts (accepted MVP false conflict #4).

Cross-cutting: committed prefixes identical across members; last_applied monotone, never a different entry at the same index; no quod_ledger crash on transient link drops ({'DOWN', ...} handled in common/3); no pending From left without a reply after its index commits, is truncated, or its leader steps down.

7. Open questions / out of scope for Phase 1

Out of scope (each has a design home, deferred):

- **Foreign / cross-namespace writes** — only own-namespace writes + cross-namespace reads (#10); owner- executes-through-its-own-rules + cross-node commit-coupling + ACL/: : authorization owed later.
- **Byzantine (BFT) agreement** — the committee runs Raft (crash-fault) only (#30); a BFT recipe under the same append/apply interface is later (#31).
- **Cryptographic identity & signing** — Phase 1 runs single-operator with `server_id() = {Host, Port}` and a keypair generated-but-unused; `#transaction.author/#transaction.sig` slots are reserved and verification is stubbed (see *Identity & signing readiness*). Real identity (`pubkey()` replaces `{Host, Port}` + signed admission record), signed changes, and BFT block commit certificates come with the survive-lies work.
- **Read-copies fan-out / the three read tiers** (#22) — Phase 1 reads hit a committee member's converged KB directly; replicating current facts out to non-committee read-copies is deferred.
- **local_history archive-holders** — full history is committee-only in the first cut (#21); read-copies hold current facts only.
- **History trimming** — the block list is kept in full permanently (#20); snapshotting (M3) compacts only the materialized live log, never the durable browsable block list.
- **Per-fact read-set granularity** — MVP hashes whole predicates (#4); same-functor different-fact changes cause accepted false conflicts; the read-set→notification index is also later.
- **Consistent cross-namespace snapshots** — read skew across namespaces tolerated (#7 caveat); no snapshots retained for reads.
- **Exactly-once effects across crashes** — effects-after-commit is at-most-once (no effect log).
- **Reactive event system** (`react_on` / D-P-E / live notifications, the read-set→notification index, cross-ontology notify) — Phase 2; see `content-layer-design.md` §14. Phase 1's only reactions are a write's own deferred effects, fired once on the submitting node at commit, never on other members, never on replay (§4.5–4.6, at-most-once).
- **Adaptive committee sizing / partition heal** — parked.

Open questions to resolve before/during build:

- Default `data_dir` and whether per-namespace dirs are base64url-encoded (assumed yes; flag for review).
- Whether `QUOD_NAMESPACE` doubles as both the Brahms membership id and the content namespace id (assumed yes for MVP; add `QUOD_CONTENT_NAMESPACE` if they ever diverge).
- Whether the leader keeps an optional pre-append OCC fast-fail (`quod_ledger:append` returning `{error, conflict_retry}` before replication) in addition to the authoritative apply-time gate, or relies solely on apply-time rejection.

Identity / signing milestone — kickoff brief

Paste the **Kickoff prompt** below into a fresh chat to start this milestone. Everything under it is the brief that prompt points at. Work happens in **plan mode** first — Yan reviews and comments the design before any code is written.

Kickoff prompt (paste this)

Start the identity / signing milestone for quod. Work in PLAN MODE: explore the code read-only and present a plan via ExitPlanMode for my approval/comments BEFORE writing any code — I want to review the design first.

Read these before planning:

- doc/identity-milestone-kickoff.md (the full brief – motivation, scope, decisions, d criteria)
- doc/deferred.md §1 + §2 (the identity gate + transport-hardening items it closes)
- CLAUDE.md and the memory index (conventions, versioning rule, no-backcompat, no-timeouts)

The brief lists the open architectural decisions I most want to weigh in on (node_id = pub address; QUIC mutual-TLS as the identity proof; id→address resolution; what gets signed; provisioning in the Nomad deploy; staging). Bring me a plan that takes a clear position on with the trade-offs, so I can comment.

Why now — the motivation (don't lose this)

We pivoted to identity **because of a concrete deploy blocker**, not as abstract hardening:

A node's identity is its address, `server_id() = {Host, Port}`. When a node moves hosts (a Nomad reschedule, `distinct_hosts` reshuffle, or host failure) its IP changes → its id changes → it falls out of its **own** durable committee config. That already broke the founder when climbing past the first deploy. And running 5 → 7 → N nodes on only 3–4 physical hosts means **multiple nodes per host**, which address-based identity handles only by fragile pinning.

So the milestone's first job is to **decouple identity from address**: a node keeps a stable, host-independent identity wherever it runs; the address becomes a mere routing hint that can change freely. That unblocks scaling the live deploy to 5/7/N. The security half (signed messages, commit certificates) is the same milestone because it's the same keypair — and it closes the hostile-net gaps that are currently only *bounded* (see `deferred.md §1`).

This is the named “big gate”: the reader/subscriber arc (P2 epidemic dissemination → P3 the millions tier) cannot proceed without it, because a subscriber must verify a **relayed** commit without trusting the relay.

Definition of done

1. **Host-independent identity.** A node has a keypair. `node_id` derives from the **pubkey**, not `{Host, Port}`. Membership config (the durable Raft log) records the stable id; the address is a resolvable hint, not the identity.
2. **Deploy scales.** A node survives a host move with its id intact; multiple nodes per host are trivially distinct (no static-port-one-per-host limit, no host pinning). Demonstrated: founder + joiner + replica stable across a redeploy, then grow to 5, then 7 nodes.
3. **The `deferred.md §1` security items close** (or are deliberately staged):
 - redirect authentication on the join path (signed `#join_reply` bound to a proven leader key);
 - pubkey-possession gate **before** `can_join` runs;
 - signed blocks + per-block **quorum certificate** + a verify API (the prerequisite for P2);
 - authenticated remote reads on the `{prove, Ns}` endpoint;
 - re-tighten the non-[safe] decode for anything accepted from a relay (`deferred.md §2`).
4. **Green + clean.** Existing suites unaffected (`quod_ledger_tests`, `join_SUITE`, `raft_safety_SUITE`, `replica_SUITE`, `quic_SUITE`); a new identity/signing suite; xref clean; no new dialyzer.

Staging is an open decision (see below) — a clean split is **(a) identity + addressing** (unblocks the deploy, ship + scale) then **(b) signing + certificates** (closes the security gaps).

Current state — accurate pointers

- `server_id() :: {inet:hostname(), inet:port_number()}` — `include/quod_ledger.hrl:11`.
Used as the key for conns, outbox, rx, next_index, match_index, votes,

voted_for, leader_id, every member_op(), and the vote/append records throughout src/quod_ledger.erl.

- Reserved-but-unused identity surface already in the records: pubkey() type + pubkey = none in #join_request (include/quod_ledger.hrl:78); #transaction.author (= node id today) + sig reserved with sig = none. No commit certificates exist.
- **QUIC already does TLS** with a single **shared static** cert/key — priv/certs/cert.pem / key.pem, loaded in src/quod_quic.erl:64-67 (load_cert/load_key). This is the clean hook: a **per-node keypair** → **self-signed cert** → **mutual TLS** authenticates the peer's pubkey at the transport layer, so "I'm talking to pubkey X" comes for free and the address is just where X happens to be.
- The dial path: quod_ledger keys outbound links by server_id (conns/outbox); the actual connect goes through src/quod_link.erl → src/quod_conn.erl → src/quod_quic.erl to an address. Today id is the address, so there's no resolution step — the milestone introduces one.
- Gossip carries {ip, port} as NodeId and does **not** verify identity — src/quod_brahms.erl:55-57 ("the gossip layer does not yet verify identity").
- Join path entry points to re-secure: dispatch_join_request/4, handle_join_reply, start_admission in src/quod_ledger.erl (deferred.md §1 names each).

Open architectural decisions — take a position, I'll comment

1. **node_id = pubkey (or a hash of it)?** And the **address** becomes a separate, dynamic hint. How is id→address resolved when we need to dial a peer we only know by id? Options to weigh: address hints carried alongside ids in membership/gossip; a resolution cache populated from inbound authenticated connections; Consul/seed lookup. (Membership must stay address-free so a move doesn't rewrite the log.)
2. **QUIC mutual-TLS as the identity proof.** Make each node's keypair its TLS identity so the handshake authenticates the peer pubkey (replacing the shared static cert). Recommended — it gives authenticated transport with no extra protocol — but confirm it fits quod_quic/quicer.
3. **What gets signed, and at what layer?** Per-#transaction author signature? Per committed **block** / **AppendEntries** quorum certificate (the P2 prerequisite)? Both? Define the verify API and where it runs (apply path vs relay-accept path).
4. **Key management / provisioning in the deploy.** Where do node keypairs come from on the Nomad cluster — generated on first boot and persisted to the per-node CSI volume, or provisioned via config/secret? This directly shapes deploy/quod.nomad and the volume layout.
5. **Staging.** One milestone in two shippable phases (identity+addressing, then signing) vs all at once. The deploy-scaling payoff lands at the end of phase (a).

6. **Membership trust.** Today `can_join` is default-open on root. With real identities, does admission still trust the operator seed/`can_join`, or does possession-of-pubkey + `can_join` together gate it (`deferred.md §1`)? Keep peer = validator ≠ user (no global peer/user directory) — see the `quod-state-and-next` memory.

Constraints (must honor)

- **Greenfield — NO backward compat.** Change `server_id`'s type, wire formats, records, the TLS cert scheme, and volumes freely. Wipe the CSI volumes for a clean re-found. No migration machinery.
- **Versioning rule.** Every commit bumps **only the patch (last) digit** of the version (the deploy image tag in `deploy/quod.nomad` + the docker build tag). Never touch major/minor — Yan owns those. HEAD is 0.6.2.
- **Threat model: hostile network.** This milestone is what lets several defenses move from *bounded* to *closed*.
- **Plan mode first.** No code until Yan approves the plan; present it via `ExitPlanMode`. He may take minutes to reply — **do not** use wakeups/short timeouts to chase him.
- **Conventions.** OTP 27+ `-moduledoc/-doc` Markdown (not `EDoc @doc`); real Markdown, public API only — see `CLAUDE.md`.

Read order

`doc/deferred.md (§1 + §2)` → `include/quod_ledger.hrl` → `src/quod_ledger.erl` (`server_id` usage, `dispatch_join_request`, `handle_join_reply`, `start_admission`, the `conns/dial` path) → `src/quod_quic.erl` (TLS cert load) → `src/quod_link.erl` + `src/quod_conn.erl` → `src/quod_brahms.erl` (gossip `NodeId`) → `deploy/quod.nomad` + `deploy/volumes/*.hcl` → `~/.claude/plans/delightful-giggling-reddy.md` (the reader arc this gate unblocks) → `CLAUDE.md` + the memory index.

Verification (end state)

```
rebar3 eunit
rebar3 ct --suite test/identity_SUITE          # new: keypair id, mutual-
TLS auth, signed-block verify
rebar3 ct --suite test/join_SUITE              # join/promote unaffected by the id change
rebar3 ct --suite test/raft_safety_SUITE
rebar3 ct --suite test/replica_SUITE
rebar3 xref && rebar3 dialyzer
```

Then the deploy-scaling proof: a node survives a host move with its id intact, and the cluster grows to 5 then 7 nodes with multiple nodes per host.

Deferred work — central registry

A living list of **deliberately-postponed work and known gaps**, with enough context to pick each up later. Cross-cutting and milestone-gated items live here; day-to-day TODOs stay in code as `AI: / REVIEW(...)` markers.

When you close an item, delete it from this file. When you defer something during a review, add it.

1. Identity / signing milestone — the big gate

The next major milestone. Today `server_id() = {Host, Port}` (spoofable), `#transaction.author/sig` are reserved but `sig = none`, and there are no commit certificates. Until nodes have keypairs and messages/blocks are signed, several hostile-network defenses can only be **bounded, not closed**.

Gated on it:

- **Redirect authentication (join path).** A joiner acts on any `{redirect, X}` reply. Only a signed `#join_reply` bound to a proven leader key closes it. *Bounded now:* only a not-yet-committed member reacts, and contacts is capped (`?MAX_CONTACTS`). → `quod_ledger:handle_join_reply`.
- **Pubkey-possession gate before can_join.** An unauthenticated joiner still triggers a `can_join` proof. Require it to prove possession of its advertised pubkey first. *Bounded now:* concurrent proofs capped at `?MAX_ADMITTING`. → `quod_ledger:start_admission`.
- **can_replicate policy.** A full-copy read-replica holds *every* fact, so admitting one is a **confidentiality** decision for access-controlled ontologies. Root is public, so reusing `can_join` is fine today; add a `can_replicate` rule before replicating private ontologies. → `quod_ledger:admit_replica`.
- **Authenticated + rate-limited remote reads.** The `{prove, Ns}` endpoint accepts a link from any node (reads are open; content is gated per-clause by `can_read`). A hostile network can flood links and proofs. *Bounded now:* `?MAX_INFLIGHT` proofs, `?MAX_FRAME_BYTES` per frame. → `quod_prove`.
- **Signed blocks + per-block quorum certificate.** Required for the **P2 epidemic dissemination** (§4): a subscriber must verify a *relayed* change without trusting the relay. Re-tighten the non-[safe] decode for any block accepted from a relay.

2. Transport hardening (hostile-net)

- **Mutual TLS is opportunistic at the lib level, but quod now binds it.** `verify => true` only *requests* a client cert (an empty cert still completes the handshake, `peer_cert = undefined`). **Closed in A.3:** `quod_conn:bind_ok/2` rejects an inbound connection whose link-header pubkey is a real 32-byte key but whose `quic:peer_cert/1` is missing or mismatched — so an unauthenticated / impersonating peer can't speak on a pubkey identity. *Still open:* the bind is skipped for non-pubkey (no-identity/test) ids, so it only bites once a node has a real keypair (the production path).
- **PEM-fallback badmatch on a missing cert file** (`quod_quic:identity_certkey/0` → `load_cert/load_key`). When the identity env (`identity_cert/identity_key`) is absent, the fallback does `{ok, Pem} = file:read_file(certfile)`, which **badmatches if the file is missing** and crashes the transport's `init/1` at boot. Pre-existing (the old code loaded the PEM unconditionally) and now *less* reachable; make it a clean fail-fast error once the legacy/test PEM path is retired (the production boot always sets the identity env via `quod_app:apply_identity`).
- **Non-[safe] decode** (`quod_ledger:decode_record`, `quod_prove:inbound`). Any on-channel speaker can deliver arbitrary terms (atom-table growth). Deliberate so fact atoms decode; closed by signed/validated payloads (§1). Size-bounded: `quod_prove` caps frames at 1 MiB, `quod_ledger` at `~?CHUNK_BYTES`.
- **Per-peer reassembly heap** (`quod_ledger` chunk reassembly). Spoofed peers each start an incomplete chunked message → up to $N \times 64$ MiB buffered. Needs a per-message timeout / a cap on concurrent reassemblies.
- **quod_prove large results.** A prove result > 1 MiB (`?MAX_FRAME_BYTES`) is dropped — no app-level chunking. Add chunking (like `quod_ledger`'s) if large read results are ever needed.
- **quod_prove outbox on link_error.** A buffered read is dropped and the caller times out (5 s) then can retry. Intended eventual behavior; an app-level retry/backoff in `remote/5` would fail faster.

3. Raft membership (M3 / M4 fast-follows)

- **remove_member / leader-removes-itself** (symmetric M4). **When it lands:** restart the join driver on loss of committed membership — a committed `{remove, Self}` would otherwise strand a node that already stopped asking (the join driver stops at committed-membership).
- **Snapshot / compaction / InstallSnapshot** (M3 heavy half). Only needed once history is trimmed; nothing compacts yet (`snap_idx = 0` always).
- **Ra-style round-based promotion.** `quod` promotes a learner once its `match_index` reaches a fixed target (its admission index); Ra (`rabbitmq/ra`) promotes only if a replication *round* completes within an election timeout —

avoids promoting a node that will perpetually lag a busy committee. Fine for low-write ontologies; revisit at high write rates.

- **Brahms-candidate discovery for joiners.** Joiners use the operator seed list as the contact path; Brahms view/sample-based discovery is a later refinement.

4. Reader/subscriber arc — the path to “millions read root”

P1 (read-replicas + remote-read) is built. Plan: `~/.claude/plans/delightful-giggling-reddy.md`.

- **Identity / signing** (§1) — the gate for everything below (can't safely gossip unsigned blocks to untrusted nodes).
- **P2 — epidemic dissemination.** Push-pull gossip + anti-entropy over the per-namespace Brahms overlay (Replicas/Subscribers join it); every block carries + is verified against its quorum certificate before re-push. Scales the change feed past the leader-star.
- **P3 — bounded-cache subscribers (the millions tier).** Predicate cache (warmup = root schema + system-ontology registry) + consume the P2 feed + invalidate touched predicates on *live* commit (never replay) + lazy-refetch via remote-prove (P1) on miss.
- **P4 — per-predicate read-set routing** (“read-set is subscription”) + cache GC (refcount + 60 s debounce, onia §10). The `quod_diff` functor-hash read-set already produces the per-predicate keys.

5. Parked (deliberately — don't reopen without a reason)

- **Adaptive view sizing** — SHELVED. The KMV $\hat{n}$ self-under-sizes at scale; the correct path (if ever needed at hundreds+ nodes) is a churn-hardened push-sum counter, not the KMV. `view_size = 16` fixed.
- **Partition heal** — a hard network split does not auto-recover (seeds read once at boot). Fix when needed: periodic re-seed from Consul.
- **Rolling-deploy ACK compat** — new-vs-old nodes churn during a rolling upgrade (the link ACK is a wire change). Non-issue in dev (redeploy all at once).

M1 content-layer — code review findings

Max-effort multi-agent review (2026-06-26) of the M1 content layer: 72 candidates → **29 verified** (22 CONFIRMED, 7 PLAUSIBLE). This is the complete record. Status key: **[FIXED]** addressed this session · **[M2]** pinned for the M2 pass (an inline %% REVIEW(M2) : note marks the site) · **[NOTE]** acknowledged / by design.

The unifying insight: M1 is single-node and single-threaded, so the **OCC layer never fires** and most concurrency findings are *dormant* until M2. The live M1 bugs are all in the **startup / restart / failure** paths — which the happy-path e2e test didn't exercise.

Real bugs (live in M1)

1. **[FIXED] quod_prolog.erl:119 — ETS read-set table leak.** The read-set table (owned by the long-lived gen_server) was deleted only on the {succeed,...} branch; fail/error/EXIT leaked one table each → eventual system_limit crash. The bare catch also hid crashes. → try/after cleanup on all paths; narrowed catch.
2. **[FIXED] quod_prolog.erl:164 + quod_ledger.erl:190 — apply_gap crash on a prolog-only restart with a concurrent write.** A new append's apply_block(N+1) could hit a fresh kb (applied=0) before the async rebuild reset the cursor → error({apply_gap}), and apply_loop had no try/catch so it crashed quod_ledger too. → readiness gate + apply_block returns {behind, Applied} and apply_loop resyncs instead of crashing.
3. **[FIXED] quod_prolog.erl:80 — prove served before rebuild completes.** Rebuild was a best-effort async cast with no readiness gate → a prove in the restart window ran against an empty kb. Spec §4.6 requires rebuild before serving proves. → ready flag; quod_ledger signals mark_ready after catch-up; prove returns {error, rebuilding} until then (this also removes a rebuild-vs-write deadlock).
4. **[FIXED] quod_prolog.erl:144 — a write “fails” but takes effect.** A 5s append timeout replied {error,...} *without parking*, but quod_ledger could still commit+apply. → park before submit; keep parked on an ambiguous timeout; a per-tx TTL delivers the verdict or a clean {error, timeout}.
5. **[FIXED] quod_prolog.erl:173 — parked caller never replied.** If the block never reached apply_block, From hung to its 35s client timeout (no TTL). → per-tx TTL eviction (same mechanism as #4).
6. **[FIXED] quod_ledger.erl:190 — commits counted rejected blocks; no try/catch around apply_block.** Verdict was discarded and commits bumped unconditionally. → bump only on ok; resync on {behind,_}; guard the call.

Correctness subtleties — dormant in M1, pinned for M2 (concurrency)

7. **[M2] quod_erlog_db_local_prove.erl:212 + :147 — OCC false-negatives.** The “skip if locally modified” rule, and the abolished branch never recording, mean a write that retracts/abolishes a functor records no read-dependency on it → a concurrent change goes undetected. Belongs with the M2 OCC + per-fact-granularity rework (design-doc §12 #2).
8. **[FIXED] quod_erlog_db_local_prove.erl:162 — OCC false-positive.** `get_procedure_type` (metadata) went through `get_procedure`, recording a content dependency for `predicate_property/current_predicate`. → type check reads the committed type directly, no read recording.

Robustness — mostly M2-coupled

9. **[M2] quod_ledger_store.erl:119 — write_meta omits the post-rename dir-fsync.** A power-cut can lose a vote → double-vote risk. Only matters once M2 has *real elections* (1-voter never has a contested vote).
10. **[FIXED] quod_prolog.erl:204 — global atom per namespace + named-table restart race.** `kb_table` minted a permanent atom and used a named ETS table (re-create race on rapid restart). → switched the committed `kb` to `erlog_db_dict` (functional, threaded through `#s.est`): no per-namespace named ETS table and no global atom at all.
11. **[M2] quod_ledger.erl:147 — become_leader skips the Figure-8 noop block.** Benign at 1-voter (no prior-term uncommitted entries); **required** for M2's multi-voter commit guard. Goes in with the real election path.
12. **[FIXED] quod_metrics.erl:94 — raw binary Ns as a prometheus label.** A non-printable namespace breaks `/metrics`. → label is a safe string.
13. **[M2] quod_ledger_store.erl:282 — scan_log trusts CRC-valid frames without checking index contiguity/monotonicity.** Defensive; low stakes at 1-voter.
14. **[M2] quod_erlog_db_local_prove.erl:91 — new/1 drops assert_hooks/ retract_hooks on the wrapped out_db.** No hooks are configured in M1; latent if `erlog` hooks are ever used.
15. **[M2] quod_erlog_db_local_prove.erl:123 — local/committed tag separation is by numeric range (1_000_000).** Collision only if one functor ever accumulates a million committed clauses; latent.

Efficiency — fine at M1 scale, M2

16. **[M2] quod_ledger.erl:115 — in-memory #d.log duplicates the store's index; the hot path is $O(n^2)$** (`++/lists:last/keyfind` per op). Rework when

logs grow (the store already has an $O(1)$ index).

17. **[M2] quod_erlog_db_local_prove.erl:113 — assertz ++ is $O(M^2)$ for M asserts to one functor; apply_ops re-fetches the full clause list per op.**
18. **[M2] quod_prolog.erl:119 — a fresh public ETS table is created/torn down per prove** (the read-set is tiny + single-process). Tie-in with the #1 leak fix; a per-engine reused table is the M2 form.

Cleanup

19. **[FIXED] quod_prolog.erl:59 — dead replay_reset/2** (left after the rebuild fix). Removed.
20. **[FIXED] quod_prolog.erl:149 — dead {error, conflict_retry}/{error, busy} clauses** in submit_write; quod_ledger:append never returns those. Removed.
21. **[FIXED] quod_prolog.erl:89 — proves counter not bumped on the {error, _} branch nor on writes.** → counted consistently.
22. **[FIXED] quod_diff.erl:80 — clause_present/5 + find_tag/5 are the same scan.** → clause_present defined in terms of find_tag.
23. **[FIXED] quod_diff.erl:52 — functor_of/1 re-implements erlog_int:functor/1** (already exported, same shape). → call erlog's.
24. **[FIXED] quod_ledger_store.erl:243 — hand-rolled URL-safe base64.** → OTP native `base64:encode(_, #{mode => urlsafe, padding => false})`.

By design / accepted

25. **[NOTE] quod_diff.erl:27 — op bodies carry erlog's well-formed {Body, HasCut} shape.** Deliberate (greenfield, no compat): the overlay produces it, apply_ops stores it, functor_hash consumes it — internally consistent. erlog is SHA-pinned.
26. **[NOTE] quod_erlog_db_local_prove.erl:68 — functor_ops collapses asserta/assertz order + duplicate multiplicity.** Correct for the content-identity (set) semantics quod uses for facts; if rule *ordering* ever matters, revisit (M2).
27. **[M2] quod_ledger_store.erl:197 — read_range silently drops a missing index** (a gap fails the generator). Add gap detection with the contiguity check (#13).

M2 content-layer — code review findings

Max-effort adversarial multi-agent review (2026-06-27) of the **M2** ordering layer (per-namespace Raft committee over loopback QUIC: election + replication, driving committed blocks into `quod_prolog`). 11 candidates → **8 confirmed** (3 rejected). Status key: **[FIXED]** addressed this session · **[DOC]** recorded with a concrete fix, out of M2's testable fault model.

All four `raft_safety_SUITE` multi-node tests, 18 `quod_ledger_tests`, and the full 96-test eunit suite pass after these fixes.

Real bugs (live in M2)

1. **[FIXED] `quod_ledger.erl` — chunked `AppendEntries` unconditionally dropped → replication stall.** `frames/2` chunks only when the encoded record exceeds `?CHUNK_BYTES` (64 KiB), but the receiver drops any reassembled non-snapshot message over `?MAX_RAFT_BYTES` (same 64 KiB) — so *every* chunked AE was discarded, never acked, and the leader re-sent the same oversized batch forever. Reachable because `log_from/3` batched by entry **count** (`max_batch=256`), so a catch-up or a stream of fact-bearing blocks could exceed 64 KiB. → batch AEs by encoded **bytes** (`?AE_BATCH_BYTES`, `take_under_bytes/2`) so an AE is always a single frame; the chunk path is reserved for M3 snapshots.
2. **[FIXED] `quod_conn.erl` / `quod_ledger.erl` — dead-but-dialable member → unbounded outbox.** A connection that never comes up (`quic:connect` timeout/closed) discarded its queued `{open_link, ...}`, so the waiter got neither `link_up` nor `link_error`; `quod_ledger`'s per-peer outbox then grew every 150 ms heartbeat forever (OOM) against an offline member. → `quod_conn:start_outbound` now drains queued opens and replies `link_error` on every connect-failure path; `send_raft` **coalesces** the outbox to the latest message per peer (each Raft RPC carries full current state, so newer supersedes older).
3. **[FIXED] `quod_ledger.erl` — multi-voter restart marked `quod_prolog` ready over an empty kb.** On restart `commit_index` resets to `snap_idx` and is only re-learned from the first `AppendEntries`/election; the old rebuild handler marked ready immediately, so a restarted multi-voter member briefly served fail/stale answers for durably-committed facts (1-voter was accidentally safe). → readiness is now gated on `caught_up/1` (applied up to a *re-learned* commit point), and `mark_ready` rides the same FIFO cast channel as `apply_block`, so it always lands after the applies. Enabled by #4.

4. **[FIXED] quod_ledger.erl / quod_prolog.erl — synchronous append↔apply_block deadlock.** quod_prolog:submit_write calls quod_ledger:append synchronously while quod_ledger's apply loop calls back into quod_prolog:apply_block synchronously — a call cycle. M1 dodged it for the leader's *own* write (reply the commit-ack before applying), but the rebuild handler and any 2nd concurrent writer close the cycle into a mutual block (self-heals only at the 5 s append timeout). → **apply_block is now an async cast**: quod_ledger never blocks on quod_prolog, so it always stays free to service append. The OCC verdict goes straight to the parked client; a forward gap asks quod_ledger to re-drive. (Also what makes #3's readiness gating deadlock-free.)

Robustness — production-hardening beyond M2's loopback fault model

5. **[DOC] quod_ledger.erl — a silently half-open cached link is never recovered.** If a follower's host crashes or a one-way partition forms such that the QUIC connection stops delivering *without* emitting a close (so no DOWN), the leader keeps sending into the dead link; its 150 ms heartbeats keep the connection "active", so QUIC's idle timeout never fires either. **Not reachable on loopback** (a killed node closes cleanly → DOWN → re-dial, which leader_failover exercises). **Fix when hardened**: leader tracks heartbeats-since-last-reply per peer (a follower replies to *every* AE), and on a threshold tears down the whole connection (a re-introduced quod_quic:drop_conn, killing the quod_conn so the next send dials a fresh one) — now safe because the symmetric client-only transport no longer produces the phantom that made an earlier attempt churn. Deferred only because it cannot be validated in M2's loopback setup, not because it's hard.

Lower-severity / latent — fixed opportunistically

6. **[FIXED] quod_ledger.erl:apply_committed — badmatch if $I \leq \text{snap_idx}$.** entry_at/2 holds only the live tail (snap_idx+1..); a {behind,A} resync (now: a post-rebuild reset) could leave last_applied < snap_idx, making the next entry_at return false → badmatch outside the apply try/catch. Unreachable in M2 (snap_idx=0), live once M3 snapshots land. → start the loop at max(last_applied, snap_idx)+1.
7. **[FIXED] quod_ledger.erl:reassemble — duplicate chunk Seq double-counted bytes.** A retransmitted chunk overwrites in got (size unchanged) but was added to bytes twice, able to spuriously trip the reassembly cap. → count a part's bytes only on first sight of its Seq. (The chunk path is now AE-free per #1, so this is snapshot-only/M3, but cheap.)

8. **[FIXED] quod_ledger_store.erl:scan_log — silent truncation past mid-log corruption.** scan_log trimmed (deleted) the file at the first bad-CRC/discontinuous frame, treating it as a torn tail. But a crash can only damage the *final* frame, so a corrupt frame **followed by a valid one** is mid-log bit-rot — trimming it silently discarded durably-committed entries (and could let the node re-append divergent indices peers already hold). → trim_or_fail/7 peeks past the bad frame: a valid ?MAGIC following = **fail-stop** ({log_corruption, ...} — recover from peers); nothing following = torn tail, trim as before. Covered by t_interior_corruption_fail_stops / t_torn_tail_bad_crc_trims.

Second pass — max-effort /code-review (10 angles, 2026-06-27)

A separate 10-angle line-by-line review (50 raw → 38 deduped → 15 verified) over the same diff, including the fix code above. All addressed except two non-bugs:

- **[FIXED] quod_ledger.erl:send_raft — duplicate link_up closed the LIVE link (HIGH).** send_raft re-called open_link on every send to a not-yet-connected peer (no in-flight guard); each dial added a waiter in quod_conn, which then notified all waiters with the SAME LinkPid — the duplicate link_up fell into the “already linked” branch and quod_link:closed the just-adopted live link (self-DOWN → drop → re-dial → stall). → dial only when not maps:is_key(M, Outbox) (no open already in flight).
- **[FIXED] chunk path (MEDIUM) — inbound guard ?CHUNK_BYTES + 64 dropped a full chunk frame** (the envelope + Ns + ref overflow it); a single oversized #transaction can chunk in M2. → guard + 1024, and a REASSEMBLED message routes under ?MAX_SNAPSHOT_BYTES (not the single-frame ?MAX_RAFT_BYTES).
- **[FIXED] reassemble badkey crash (LOW)** — an out-of-range Seq/Total from a corrupt peer made the completion path maps:get a missing key and crash the statem. → reject out-of-range Seq/Total up front.
- **[FIXED] efficiency — advance_commit re-derived the committee $O(N) \times \log$ per AE reply** (hoist peers/quorum once); **apply_committed did a registry lookup per entry** (hoist once); **last_log_index/term were lists:last ($O(n)$) on every hot-path call** → cached tail (last_idx/last_term, refreshed via with_log/2 at every log mutation), and log_from fast-paths the caught-up/heartbeat case to $O(1)$.
- **[FIXED] quod_conn — dead {conn, Peer} registration** (leftover after removing connection adoption; nothing reads it, a mutual-dial pair clashed silently) → removed reg_conn.
- **[FIXED] submit_write — {error, busy} backpressure double-wrapped to {error, {error, busy}}** → handled as a clean retry signal; append/2 spec now lists {error, busy}.

- **[FIXED] trim_or_fail trusted the corrupt frame's length** for the next-frame peek (#8 follow-up) → now scans the tail for the magic, so a corrupt length can't make the interior-corruption check miss.
- **[NOTE] link_error drops the outbox with no immediate re-dial** — by design: Raft tolerates the loss (next heartbeat/election re-sends; with the `send_raft` fix the cleared outbox re-dials on the next send).
- **[NOTE] encode/1+decode/1 mirror quod_brahms's** — left as-is: trivial `term_to_binary` wrappers, and `quod_ledger`'s codec has diverged (`decode_record` without `[safe]`); a shared module would over-abstract two one-liners.

Rejected (verified false alarms)

- **next_index[Peer] regression on a reordered AE reply** — not reachable: `match_index` in a reply is monotone (`prev_log_index + length(entries)`), and the in-order per-peer `quod_link` channel can't reorder; self-healing even if it could.
- **5 s statem freeze from apply_block during a "deadlock window"** — refuted: the commit-ack is replied before the apply loop runs (now moot anyway: `apply_block` is async).
- **fail_pending_above on a follower** — dead code, not a bug: a follower never holds pending (only a leader parks client appends).

Module reference

Application & supervision

`quod_app`

quod application entry point.

Configuration is a **HOCON file** (`config/quod.conf`, or wherever `QUOD_CONF` points; a release ships `priv/quod.conf`). The file is the primary source; OS environment variables prefixed `QUOD_` override individual keys, using `__` to descend the path — e.g. `QUOD_CONTENT__MODE=join` overrides `content.mode`, `QUOD_NODE__PORT=15000` overrides `node.port`. See `quod_schema` for the shape.

`load_config/0` bridges the file onto the application env the transport reads (`listen_port`, `metrics_port`, `node_id`) and returns the content section, which drives Brahms membership and the content namespace this node founds (create) or joins. It also **load-or-creates the node's Ed25519 identity** (a side effect: persists `node.key` under the identity dir and sets `node_pubkey/identity_cert/identity_key`). **With no config file present, the app starts in legacy mode** — `sys.config / application:set_env` drive the transport, no content namespace is auto-started, and no identity is minted (this is what the multi-node test SUITE relies on).

`quod_sup`

quod top-level supervisor.

Boots the QUIC transport. Brahms/Tendermint/Prolog layers get added as children here.

QUIC transport

`quod_quic`

QUIC transport over the pure-Erlang quic library: the **server** + dialer and the **connection authority**.

It starts the quic server (each accepted connection is handed to a `quod_conn` owner via `connection_handler`), and serializes outbound connection creation so there is **one connection per peer** (no dial race). Streams/links and message delivery live in `quod_conn / quod_link`.

Upper layers use one async call:

```

quod_quic:open_link(NodeId, Channel)      %% -> caller gets {link_up, NodeId, Channel,
↳ LinkPid}
quod_link:send(LinkPid, Payload)          %% direct, non-blocking
%% messages arrive on the gproc property {channel, Channel}
%% erlang:monitor(LinkPid) -> 'DOWN' is the disconnect

```

Why pure Erlang quic is a process-per-connection pure-Erlang stack — no quicer/msquic NIF, so no C toolchain or from-source build, and a small, predictable image. (The per-node footprint is governed by the BEAM, not the QUIC backend: cap the port table with +Q in vm.args or a container's huge default nofile preallocates ~1.5 GB of port_table.)

Functions

resolve/1 Resolve a target to a dialable endpoint: an endpoint dials direct; a pubkey via the cache.

learn/2 Record a Pubkey => Endpoint resolution hint (learned from a header / gossip).

open_link(Target, Channel) Open (or reuse) a link to Target for Channel. Target is either a **node_id()** (a pubkey — resolved to an address via the hint cache; link_error if not yet known) or a **{Host,Port}** endpoint (dialed directly — for seeds/contacts whose pubkey is not yet known). Asynchronous: the caller receives {link_up, Target, Channel, LinkPid} (or {link_error, Target, Channel}).

quod_conn

A **connection**: one process per peer, **owning** one pure-Erlang quic connection and routing its streams to quod_link processes.

The quic owner model delivers every stream's events to this one process, so quod_conn is the demux point: {quic, Conn, {stream_data, StreamId, Bin, Fin}} is forwarded to the link registered for StreamId (a peer-opened stream gets a fresh inbound link on first sight). It opens outbound streams on {open_link, Channel, _} and learns an inbound peer's node id from the link's header.

Links are **linked** to the connection, so when the connection drops every link dies with it — and each link's death is the disconnect signal its holder monitors.

Functions

open_link(ConnPid, Channel, ReplyTo) Ask this connection to open (or reuse) a link for Channel, replying to ReplyTo.

start_inbound(Conn, Self) Own an accepted connection Conn (the quic listener transfers ownership to us).

start_outbound(Host, Port, Peer, Self, ALPN, Cert, Key) Dial Host:Port (known node id Peer), become the connection owner, serve links. We present our own Cert/Key so the peer (the TLS server) can authenticate us via mutual TLS. `verify => false` skips validating the *peer's* self-signed server cert (no CA chain); the peer authenticates US, and we authenticate it when it dials back — every directed pair is server-verifies-client.

quod_link

A **link**: one process per (peer, channel) = one QUIC stream's framing.

Unlike a NIF stream, a pure-Erlang quic connection delivers *all* of its streams' data to a single owner (quod_conn). So a link does **not** own its stream — quod_conn routes inbound bytes for the link's StreamId to it as {data, Bin, Fin}, and the link sends with `quic:send_data/4` directly (any process holding the connection pid may send). A link:

- **frames** an opener's first write as a **header** (<<NLen:16, NodeId, CLen:16, Channel>>), then length-prefixed payloads (<<PLen:32, Payload>>),
- **publishes** each payload as {quod_message, {Peer, self()}, Channel, Payload} on the gproc property {channel, Channel},
- **dies** when its stream/connection drops — its death is the disconnect signal its holder `erlang:monitors`.

Liveness handshake (link_up requires a peer ACK)

Over a connectionless (UDP/QUIC) transport a *send* is **not** proof the peer is alive — a dead peer's connection lingers and a write to it silently succeeds (this is the SWIM rule: only a received reply proves liveness). So the **opener does not declare link_up until the peer ACKs its header**: it writes the header, then waits for the peer's first frame (an empty ACK frame the peer emits as soon as it reads the header). No ACK within `?ACK_TIMEOUT_MS` $\Rightarrow$ the link **dies** and its holder gets `link_error` (never a phantom link_up). The receiver, having read the header, already has proof

the opener exists, so it `link_ups` immediately *and* sends the ACK back. This makes a link to a dead/unreachable peer fail to come up, so the membership layer's probe correctly evicts it.

Functions

close(LinkPid) Close this link and reset its stream (its holder's `erlang:monitor` then fires DOWN).

send(LinkPid, Payload) Queue Payload to be sent on this link (non-blocking).

start_inbound(Conn, Sid, ConnProc) A peer opened Sid; read its header (from forwarded data) to learn (peer, channel).

start_outbound(Conn, Sid, Peer, Channel, Self, ConnProc) We opened Sid to Peer for Channel; announce Self (the header), then wait for the peer's ACK before declaring `link_up`. No ACK $\Rightarrow$ the link dies (`link_error` to the holder) — a write alone is never treated as liveness.

Membership (Brahms gossip)

quod_brahms

Brahms membership for one namespace (ontology) — the View V + round driver.

One `gen_statem` per namespace N_s , registered `{quod_brahms, N_s}`, holding the view V and the `quod_brahms_sampler` as inline state. Gossip for N_s flows on channel N_s .

A round

`idle` arms a jittered timer; on tick (`do_round/1`) it pushes its own id to $\alpha \cdot \ell$ peers of V and pulls views from $\beta \cdot \ell$ **disjoint** peers, then collects responses and **reconstructs** V from $\alpha \cdot \ell$ pushes, $\beta \cdot \ell$ pulls, and $\gamma \cdot \ell$ (always ≥ 1) sampler ids.

Byzantine handling

- **Push is unsolicited** → *limited-push reception*: every push is counted across the whole round (idle + collecting, since the last view rebuild), since peers push on their own schedule, not ours; once past `push_limit` the push contribution is dropped for that round (pull + sample still rebuild V, so the sampler keeps healing it).
- **Pull is solicited** → a `pull_resp` is accepted only from a peer we actually pulled this round, and is capped to the pull quota; unsolicited responses are dropped.
- **Self-exclusion, never collapse to empty, defensive + size-capped decode, link cache pruned & closed** to the view each round.

Each cached link to a view peer is `erlang:monitored`; its `DOWN` is the disconnect signal and evicts the peer at once (see `quod_link`).

Sample validation (failure detector)

The min-wise sampler is **sticky** by design — a sampled id is retained until a lower-hash id beats it — so a sampled node that dies (or, with dynamic ports, restarts under a *new* id) would otherwise be re-dialed forever. Brahms closes this with **sample validation**: each round we probe a few sampled ids whose liveness is unknown (probe_fanout ids that are in the sample but not currently linked) by opening a link to them. An id that does **not** come up within `probe_rounds` is declared dead — every sampler slot holding it is **reset to a fresh key** (`quod_brahms_sampler:invalidate/2`) and it is evicted from V. A `link_up` (or any message from the peer) is the liveness proof. This is the eviction half of the sampler's guarantee; without it, the very stickiness that makes the sample attack-resistant would pin dead ids in place.

Tombstones (beyond Brahms — SWIM-style)

Resetting our sample removes a dead id from *us*, but a peer that hasn't evicted it yet re-gossips it back, so it re-enters, is re-probed, and is re-evicted — wasteful churn until the whole fleet converges. Brahms does not address this (its analysis is asymptotic; it only guarantees convergence). We borrow the **SWIM** idea: a just-evicted id is **tombstoned** for `tombstone_rounds` and refused re-admission from *third-party* gossip (push / pull bodies) during that window. Only a **direct message that announces the id as its own sender** lifts the tombstone (SWIM **refutation**). Brahms identifies peers by their **address** (the `Addr` half of the link header) and trusts it as-announced — the gossip layer does not verify identity (see `quod_brahms_sampler`'s `sybil` caveat), so refutation is only as strong as that announced address. (The committee/ledger layer **DOES** authenticate — it binds the header's pubkey to `quic:peercert/1` — but Brahms discovery deliberately

stays address-based; gossiping the authenticated pubkey is a later refinement.) The eviction it guards is sound w.r.t. Brahms: the detector evicts only on a probe unanswered for `probe_rounds`, which a live node clears well within, so a live id is evicted (and tombstoned) only on a rare timing miss, and that self-heals the moment it next contacts us.

Deferred PUSH uses reliable streams; that optimisation comes later.

Functions

namespaces() All namespaces with a running Brahms statem on this node.

stats(Ns) Counters for namespace Ns: view/sample sizes, live links, rounds driven.

sample(Ns) Current uniform sample of namespace Ns (degrades to []).

view(Ns) Current view V of namespace Ns (degrades to [] if unavailable).

start_link(Ns, Config) Start the per-namespace statem (called by `quod_brahms_sup`, not directly).

start_namespace(Ns, Config) Start (join) Brahms membership for an ontology namespace Ns. Config needs `node_id`; `seed_peers` is optional. Spawns a supervised per-namespace statem.

quod_brahms_sup

Supervises one `quod_brahms` instance per namespace (ontology), on demand.

`simple_one_for_one`: namespaces are started via `quod_brahms:start_namespace/2` and are independent. Each child is a per-namespace Brahms membership statem.

quod_brahms_nest

Network-size estimator ($\hat{n}$) — a windowed **KMV** (k-minimum-values) distinct counter over the stream of node ids the Brahms layer gossips.

Brahms prescribes per-node structure sizes of $\Theta(\sqrt[3]{n})$ and notes a node can estimate n from its own sample (each id is seen with probability $1/n$). This module turns that into a concrete estimate so quod_brahms can later size the view/sample from it. **Phase 1 only exposes $\hat{n}$ as a metric; nothing is sized from it yet.**

How it works

Hash every observed id to a 64-bit value with a *secret-keyed* HMAC (like quod_brahms_sampler, so a flooder cannot bias it), and keep the K smallest **distinct** hashes. The K -th smallest, v_K , concentrates around K/n , giving the unbiased estimator $\hat{n} = (K-1) \cdot 2^{64} / v_K$. While fewer than K distinct ids have been seen the count is **exact** (so at small scale $\hat{n}$ is just the live id count). Relative error is $\approx 1/\sqrt{K}$.

Windowing

A plain KMV would also count *departed* ids forever. So the sketch is windowed: rotate/1 (called once per nest_window rounds by quod_brahms) shifts the current window to prev and starts a fresh cur; estimate/1 reports over $\text{cur} \cup \text{prev}$. Live ids are re-observed every round and stay; a departed id ages out within $\sim$ two windows.

Functions

estimate/1 Estimated network size over the current $\cup$ previous window.

rotate/1 Close the current window: it becomes prev, a fresh cur begins.

observe_all(Ids, N) Feed a batch of ids (order-independent).

observe/2 Feed one id into the current window.

new(K) A fresh estimator keeping the K smallest hashes, with a secret key.

quod_brahms_sampler

Brahms min-wise sampler — the uniform, flood-resistant half of Brahms.

A sampler is K independent **slots**. Each slot holds a *secret* random key that defines a hash h_i ; for every id observed, the slot keeps the id with the minimum $h_i(id)$ seen so far. Because the key is secret and HMAC behaves like a random oracle, the retained id is $\sim$ uniform over the **distinct** ids observed, *independent of how many times each appears*. So a Byzantine node that floods its id a million times is no likelier to be sampled than an honest id seen once.

This is a **pure data structure** (no process): the per-namespace `gen_statem` holds it as state and feeds it every id it sees.

Scope of the guarantee

It resists **multiplicity** flooding (one id presented many times). It does NOT resist **sybil** flooding (many *distinct* attacker ids) — every distinct id gets equal weight by design, so bounding the number of distinct attacker ids is the job of the gossip/admission layer above, not the sampler.

Security note The hash MUST be a secret-keyed, oracle-like function. Using `phash2` (or any public/predictable hash) lets a flooder craft ids that always win the min and breaks the guarantee. We use HMAC-SHA256 keyed by a per-slot secret.

`undefined` is reserved as the empty-slot marker and must not be used as a node id.

Functions

slots(Slots) Number of slots K .

invalidate(Id, Slots) A sampled id failed liveness — reset every slot holding it so it re-samples from scratch. Resetting mints a **new key** (i.e. a new hash function), which is what Brahms specifies for a sampler reset.

sample(Slots) The currently sampled ids as a **multiset** in slot order (empty slots skipped; duplicates kept — two slots legitimately collide). The caller dedupes if it wants a set.

observe_all(Ids, Slots) Present a batch of ids (order-independent).

observe(Id, Slots) Present Id to every slot; each keeps its own min-hash id.

new(K) A sampler of K empty slots, each with a fresh secret key.

quod_reg

gproc registration nomenclature for quod.

Every quod process registers a unique **name** and posts its events to a matching **property**, both keyed by the same Key = {Type, Id}:

gproc entry	form	use
name	{n, l, Key}	unique, addressable via {via, gproc, {n,l,Key}}
property	{p, l, Key}	subscribable; the process posts its events here

Key conventions

Key	identifies
{transport, node}	the QUIC transport gen_server (singleton)
{sup, node}	the root supervisor
{conn, NodeId}	a peer connection process (one per peer)
{quod_brahms, Ns}	a namespace's Brahms statem (one per Ns)
{channel, Name}	a pub/sub channel (property only, no owner)

Example

```
%% an interested process subscribes to a channel's messages...
quod_reg:subscribe({channel, Ns}),
%% ...and a link on that channel posts one:
quod_reg:publish({channel, Ns}, {quod_message, {Peer, self()}, Ns, Payload}).
```

Functions

unsubscribe(Key) The **current** process stops receiving Key's events.

subscribe(Key) The **current** process starts receiving Key's events.

publish(Key, Event) Post Event to every process subscribed to Key's property.

reg(Key) Register the **current** process under Key's name. Use from spawned loops that aren't started via {via, gproc, _}.

where(Key) Pid currently owning Key's name, or undefined.

via(Key) A {via, gproc, _} ref for Key — pass to gen_server/supervisor.

prop(Key) The gproc **property** entry for Key.

name(Key) The gproc **name** entry for Key.

Namespaces

quod_ns

Per-namespace sub-supervisor — the unit of **fate-sharing** for one ontology's content processes. Supervises quod_ledger then quod_prolog with **rest_for_one**.

The durable log rebuilds the kb, never the other way round: if quod_ledger crashes, rest_for_one restarts it *and then* quod_prolog (which rebuilds from the reloaded log); if quod_prolog crashes alone, only it restarts and rebuilds from quod_ledger's committed prefix via the rebuild handshake. See doc/ordering-layer-spec.md §5.2.

quod_ns_sup

Root of the content-layer subtree: a simple_one_for_one parent that starts one quod_ns per ontology namespace. Mirrors quod_brahms_sup. See doc/ordering-layer-spec.md §5.1.

Prolog engine & proving

quod_prolog

Per-namespace fact engine: owns the committed erlog knowledge base for one ontology, serves `prove/3`, and applies committed blocks from `quod_ledger` in log order. One `gen_server` per namespace.

- **Reads** run on a copy-on-write overlay (`quod_erlog_db_local_prove`) so the committed kb is never touched; the answer is bindings, returned to the caller.
- **Writes** (a proof that staged asserts/retracts) become a `#transaction{}` submitted to `quod_ledger`; the caller is parked and replied to when the block applies (or reaped by a per-tx TTL if the verdict never arrives).
- **apply_block/3** is the deterministic state machine `quod_ledger` drives on every member: re-check the read-set against the committed kb (OCC), then apply the diff or reject — identical verdict on every member.

Proves are gated until an initial **rebuild** completes (ready), so a freshly (re)started engine never answers from a half-built kb. The kb is built with the erlog flag `unknown = fail`. See `doc/ordering-layer-spec.md §4`.

Functions

genesis_diff(File) Build the genesis write-set from a `.pl` file.

Reads the file with the erlog parser, then turns each clause into a write-set `op()` in the SAME compiled form the live write path produces — so the genesis commits and replays through the normal apply path. erlog's `assertz` compiles the body (`well_form_body`, yielding `{Body, HasCut}`); we capture the result with the `quod_erlog_db_local_prove` overlay, exactly as `run_proof/2` does for a live write. Hand-building `{Head, true}` would store a malformed clause and crash on the first prove — the body must be the compiled form, not raw `true`.

Used at create only: the founder calls this once and commits the result into the ledger as the genesis block. A missing/unparseable file throws `{genesis_failed, _}`, which `quod_ledger:init/1` turns into `{stop, _}` (fail-fast — a node with no root is useless).

mark_ready(Ns) Signal that the initial rebuild is complete and proves may be served.

apply_block(Ns, Index, Change) Apply a committed entry (called by quod_ledger, strictly in index order). **Async (cast) on purpose:** quod_ledger calls this while it may itself be the target of a synchronous quod_ledger:append from this very process (the write path). A synchronous apply_block would close that call cycle into a deadlock (each waits on the other). As a cast, quod_ledger never blocks on us, so it stays free to service append. The OCC verdict is delivered straight to the parked client here; a forward gap asks quod_ledger to re-drive.

applied(Ns) The committed log index this kb has applied (the freshness height for a read).

prove_ro(TargetNs, Goal, CallerNs) Read-only prove: like prove/3 but a write goal is refused ({error, read_only}).

prove(TargetNs, Goal, CallerNs) Prove Goal (emitted from CallerNs) against namespace TargetNs.

quod_prove

Per-namespace **remote-read** endpoint — the path by which a node that is NOT on a namespace's Raft committee reads it without going through consensus (the Subscriber/Neither read path of the reader/subscriber layer).

Two halves in one gen_server, riding a dedicated **{prove, Ns}** quod_link channel (separate from quod_ledger's {log, Ns} — the channel-match hazard):

- **Responder** (on a Member/Replica that holds the kb): serves a {prove_req, ...} against the local committed kb via quod_prolog:prove_ro/3 (READ-ONLY — a write goal is refused, so a remote reader can never write through a Member). Each request runs in a worker so a slow prove never blocks the endpoint; concurrency is capped (?MAX_INFLIGHT) as a hostile-net bound.
- **Client** (any reader): remote/4,5 sends a {prove_req, ...} to a contact Replica/Member and awaits the {prove_resp, ...}.

Freshness contract: the response carries the committed log height the answer was proved at. Reads are eventual / bounded-stale; MinHeight gives read-your-writes — a responder whose applied height is below MinHeight replies stale so the caller retries a fresher peer.

Trust (P1): reads are open (content is gated per-clause by can_read); the inner record decodes without [safe] — same trusted-fleet posture as quod_ledger today. Authenticated/rate-limited remote reads on a hostile network arrive with the identity milestone.

Functions

remote(Ns, Goal, CallerNs, MinHeight, Contact) As remote/4 but against an explicit contact Replica/Member {Host, Port}.

remote(Ns, Goal, CallerNs, MinHeight) Remote-read Goal against Ns via a default contact. MinHeight = read-your-writes floor.

quod_erlog_db_local_prove

A **staging overlay** for the erlog database, used to run a proof against the committed knowledge base without touching it.

Wrap a committed erlog state with wrap_state/1,2, run the goal, then extract:

- get_local_changes/1 — the **write-set**: the asserts/retracts the proof made, as content-only op()s.
- get_read_set/1 — the **read-set**: one content hash per {Functor, Arity} the proof read from the committed db (for the apply-time OCC re-check).

Asserts/retracts are shadowed per functor; reads merge asserta ++ committed ++ assertz. The committed db (out_db) is read-only throughout. This is a slimmed port of bbsvx's bbsvx_erlog_db_local_prove (federation/ACL/provenance dropped).

Ported against erlog's #est{}/#db{} (see erlog_int.hrl); read-set hashing is shared with quod_diff so the producer here and the validator in quod_prolog agree bit-for-bit.

Functions

cleanup_read_set/1 Drop the read-set table for a finished proof (pass the final #est{}).

get_read_set/1 The read-set: #{ {Functor, Arity} => content-hash } of what the proof read.

get_local_changes/1 The write-set: the proof's asserts/retracts as content-only ops.

wrap_state(St, Opts) As `wrap_state/1`; `#{read_set => true}` also tracks the read-set.

wrap_state/1 Wrap a committed `#est{}` so a proof stages onto a private overlay.

quod_schema

HOCON config **schema** for quod.

The config **file** (`config/quod.conf`, overridable by `$QUOD_CONF`) is the primary source of configuration. OS environment variables prefixed `QUOD_` override individual keys, using `__` to descend the path — e.g. `QUOD_CONTENT__MODE=join` overrides `content.mode`, `QUOD_NODE__PORT=15000` overrides `node.port`. Env override is applied during `hocon_tconf` check (see `quod_app:load_config/0`).

String values are `binary()` (HOCON's native string), converted to lists in the boot code where the rest of the system expects `{Host, Port}` / file paths.

Content layer (ledger & diff)

quod_ledger

Per-namespace lean **Raft** committee member: orders and replicates the ontology's block list (the durable history) over the dedicated `{log, Ns}` `quod_link` channel, and drives committed blocks into `quod_prolog`. One `gen_statem` per namespace; states `follower` | `candidate` | `leader`. Hand-rolled Raft, **no Erlang distribution** — every committee RPC rides a `quod_link` stream, exactly as Brahms gossips. See `doc/ordering-layer-spec.md §1–§2`.

Milestones

- **M1** — the single-voter degenerate case: a sole-member committee wins instantly, commits each append on its local `fsync`, applies in order. No networking.
- **M2** — a real `N=3` committee over loopback QUIC: `RequestVote` + `AppendEntries` on the `{log, Ns}` channel, randomized elections, leader replication, and the multi-voter commit rule with the mandatory **Figure-8 current-term guard**. The commit/apply/persist core from M1 is unchanged (#18).

- **Join** (this layer) — a mode=join node grows the committee: it dials a contact (#join_request{}), the leader proves the can_join admission rule and admits it as a non-voting **learner** ({add_learner}), the existing AppendEntries back-up loop catches it up, and once it reaches its target the leader **promotes** it to a voter ({promote}) under the full single-server safety gate (one config change in flight + a current-term commit). Voters = the quorum/election set; learners receive entries but never vote.

append → commit → apply is deadlock-free (load-bearing)

quod_prolog:submit_write calls quod_ledger:append/2 synchronously, and this module's apply step calls **back** into quod_prolog:apply_block/3 synchronously. To avoid the two blocking on each other, append/2's From is parked in pending and replied {ok, Index} the instant the entry **commits** (a majority holds it) — *not* at apply time. That commit ack unblocks quod_prolog; the apply loop (a deferred internal event) then re-enters it freely, and the OCC verdict reaches the prove-client through quod_prolog's own tx_id-keyed park/release path. On step-down or a truncation that drops an uncommitted entry, the parked From is failed {error, not_in_charge, Hint}, which submit_write already handles. (This deliberately keeps the M1 split rather than the design doc's "reply the verdict from the log statem", which would re-introduce the deadlock.)

Channel-match hazard

Every receive/link clause guards on Chan = term_to_binary({log, Ns}, [deterministic]), **never** on Ns — Brahms matches its own gossip on Ns, and a clause matching Ns here would steal/own the wrong stream.

Functions

rebuild(Ns) Ask the log to (re)drive committed blocks into a freshly-started quod_prolog.

append(Ns, Change) Submit a change. Blocks until the entry commits ({ok, Index}) or this node loses leadership.

quod_ledger_store

Durable on-disk store for one namespace's Raft log — the **block list**.

This is the only quod code that touches disk. It is a plain library (no process, no registration): every function is synchronous and completes its required fsync before returning, and the handle is threaded by the caller (quod_ledger). See doc/ordering-layer-spec.md §3.

Layout, under DataDir/<base64url(Ns)>/:

file	holds
meta.term	one frame of #{cur_term, voted_for} (atomic tmp+rename)
log.0001	append-only CRC-framed #entry{} records — the block list

Each frame is <<Magic:32, Len:32, CRC:32, Payload:Len/binary>> with Payload = term_to_binary(Term, [deterministic]) and CRC = erlang:crc32(Payload). The in-memory index (#store.idx) is rebuilt by scanning log.0001 at open/2; a truncated final frame (a crash mid-append that never fsync'd) is detected by the length/CRC check and trimmed, which is safe because such an entry was never acknowledged.

Snapshots/compaction are deferred to milestone M3; the snapshot API is present but read_snapshot/1 returns none until then.

Functions

install_snapshot(S, LastIdx, LastTerm, Config, Data) Install a leader snapshot then drop the live log. (M3.)

write_snapshot/5 Write a snapshot file atomically. (Exercised from M3; present for the API.)

read_snapshot/1 Latest snapshot payload, or none (no snapshots are written before M3).

term_at/2 The term of the entry at Index (0/snapshot sentinels handled), or undefined.

last/1 {LastIndex, LastTerm} of the live tail (or the snapshot point if empty).

read_range/3 Read entries From..To (clamped to the live tail), in index order. The live log is contiguous, so a missing index in range is corruption, not an empty slot: read_range raises rather than silently returning a short list (review #27).

read_at/2 Read the entry at Index, verifying its CRC.

truncate_from/2 Delete entry Index and everything after it (follower conflict resolution). Refuses to truncate into a snapshot. datasync'd before returning.

append/2 Append contiguous entries (indices last_index+1 ..). One datasync for the batch; returns only after it completes (the entries are durable before quod_ledger counts them toward commit).

write_meta/3 Persist {Term, VotedFor} atomically (tmp + datasync + rename). MUST complete before quod_ledger grants a vote or replies at a bumped term.

read_meta/1 The persisted {cur_term, voted_for}; {0, none} for a fresh namespace.

load/1 Reload everything needed to reconstruct quod_ledger's durable state: the persisted term/vote, the full log (from snap_index+1 up), and the snapshot metadata.

close/1 Close the store's file handle.

open(Ns, DataDir) Open (creating if needed) the on-disk store for Ns under DataDir.

quod_diff

Pure helpers over the committed erlog database for the content layer.

- **functor_hash/3** — a content hash of one predicate {Functor, Arity} (the {Head, Body} list, tags stripped). The read-set captured by quod_erlog_db_local_prove uses *this* function, and quod_prolog's apply-time OCC re-check uses it too, so producer and validator agree exactly.
- **validate/3** — re-check a read-set against the committed db: ok if every predicate still hashes to the recorded value, else {conflict, Functor}.

- `apply_ops/2` — apply a `#transaction.diff ([op()])` to the committed erlog state, with content-identity dedup (asserting an identical fact is a no-op; retract is by content).

`op()` and `clause()` are defined in `quod_ledger.hrl`; `#est{}/#db{}` in `erlog_int.hrl`.

Functions

`apply_ops/2` Apply a `diff ([op()])` to a committed erlog state, with content-identity dedup.

`validate(ReadCheck, Mod, Ref)` ok if every predicate in `ReadCheck` still hashes as recorded, else the first `{conflict, F}`.

`functor_hash(Mod, Ref, F)` Content hash of predicate `F` in the db `Mod:Ref` (`{Head,Body}` list, tags dropped).

Identity & signing

`quod_identity`

A node's cryptographic **identity**: an Ed25519 keypair whose **public key is the node_id** (the address is only a routing hint). Generated once on first boot and persisted to the per-node data dir, so a node keeps its identity across restarts **and across a move to a new host** — the whole point of the identity milestone.

What is stored

Only the 32-byte Ed25519 **private seed**, at `<dir>/node.key` (mode 0600). The public key and a self-signed certificate are *derived* deterministically from the seed on every load (`crypto:generate_key/3 + mint_cert/1`), so there is one secret to guard and nothing else to keep in sync.

The cert and mutual TLS

`mint_cert/1` produces a minimal **self-signed** X.509 cert carrying the Ed25519 public key. The transport (`quod_quic`) presents it on every connection and reads the *peer's* via `quic:peercert/1`; `pubkey_of_cert/1` recovers the 32-byte key. There is **no CA** — a peer key is trusted because the committee admitted it (`can_join`), not because of a chain. Verification rests on TLS 1.3's `CertificateVerify` (proof the peer holds the key), which `quic` checks without a chain, so a self-signed per-node cert authenticates cleanly.

Signing (Phase B) `sign/2 / verify/3` (Ed25519 over canonical bytes) and the per-block quorum certificate land with the signing phase; this module owns the keypair they use.

Functions

short(Pub) A short, log-readable rendering of a public key: `kp_` + the first 4 bytes as hex (e.g. `kp_ab12cd34`). For display only — the canonical id is the full key.

pubkey_of_cert(DER) Recover the 32-byte Ed25519 public key from a DER cert (e.g. a `quic:peercert/1` result). Returns error on a malformed/unexpected cert — the input is the *peer's* cert on the TLS path, so a bad one must be rejected, never crash the caller.

mint_cert/1 Mint a minimal **self-signed** Ed25519 certificate (DER) for a keypair. Deterministic (fixed serial + validity), so re-minting from the same seed yields the same identity.

key_term/1 The signing key term (`public_key/quic #'ECPrivateKey' {}` form) for a keypair.

generate() Generate a fresh Ed25519 keypair {PubKey, Seed} (not persisted).

ensure(Dir) Load this node's identity from `Dir`, generating + persisting a fresh keypair on first boot (no `node.key` yet). Returns the derived public key (the `node_id`), the self-signed cert, and the signing key term — see `identity/0`.

Observability

quod_metrics

Prometheus metrics for quod, served at GET `/metrics` on `metrics_port` (default 14568).

Per-namespace Brahms gauges (`quod_brahms`) are refreshed on a short timer and labelled by namespace:

metric	meaning
<code>quod_up</code>	1 while the node is up
<code>quod_brahms_view_size{namespace}</code>	size of the view V
<code>quod_brahms_sample_size{namespace}</code>	size of the uniform sample
<code>quod_brahms_links{namespace}</code>	live cached links to peers
<code>quod_brahms_rounds{namespace}</code>	rounds driven so far
<code>quod_brahms_evictions{namespace}</code>	dead peers evicted by sample validation (cumulative)
<code>quod_brahms_tombstones{namespace}</code>	current tombstone entries (bounded; drains to 0)
<code>quod_brahms_estimated_n{namespace}</code>	estimated network size $\hat{n}$ (KMV; exact below k)